Typst changes from origin/main (77e07947) to 0edeac81

Typst changes from origin/main (77e07947) to 0edeac81 1
thesis/content/01-introduction.typ 1
thesis/content/02-background.typ 12
thesis/content/03-program-model.typ 16
thesis/content/04-traces.typ 23
thesis/content/05-domains.typ 30
thesis/content/06-analysis-interface.typ 37
thesis/content/07-cooperation.typ 41
thesis/content/08-equations.typ 45
thesis/content/09-solving.typ 48
thesis/content/10-results.typ 52
thesis/content/11-instances.typ 56
thesis/content/12-executable.typ 60
thesis/content/13-evaluation.typ 64
thesis/content/14-related.typ 71
thesis/content/15-conclusion.typ 76
thesis/content/abstract.typ 80
thesis/content/ai-use.typ 82
thesis/content/appendix-tooling.typ 86
thesis/content/reading-guide.typ 88
thesis/thesis.typ 90

thesis/content/01-introduction.typ

77e07947 → 0edeac81

611 old lines, 585 new lines

73% similar lines **+148** **-174** **49 changed blocks**

Old	77e07947	New	0edeac81
1	#import "../lib/code.typ": (	1	#import "../lib/code.typ": (
2	fixture, isaconst, isalocale, isathm, isatype, listing, oblig, rebase, site,	2	fixture, isaconst, isalocale, isathm, isatype, listing, oblig, rebase, site,
3	)	3	)
4	#import "../lib/figures.typ": check-row, partref, verdict	4	#import "../lib/figures.typ": check-row, partref, verdict
5	#import "../lib/sources.typ": proved	5	#import "../lib/sources.typ": proved
6	#import "../lib/math.typ": *	6	#import "../lib/math.typ": *
7	#import "../lib/theme.typ": metro, vb	7	#import "../lib/theme.typ": metro, vb
8		8	
9	#let metro-line(c, body) = text(fill: c, weight: "bold", body)	9	#let metro-line(c, body) = text(fill: c, weight: "bold", body)
10	#import "../lib/claims.typ": claim-ref	10	#import "../lib/claims.typ": claim-ref
11		11	
12	= Introduction <ch:intro>	12	= Introduction <ch:intro>
13		13	
14	A static analyzer answers questions about every execution of a program at once:	14	A static analyzer answers questions about every execution of a program at once.
15	whether a divisor can be zero at some point, whether a program point is	15	It can answer, for instance, whether a divisor can be zero at some program
16	reachable at all, or whether a check always holds. Users act on these answers	16	point, whether a point is reachable at all, or whether a condition always holds
17	without running the program, so the answers have to be right. The analyzer,	17	there. The developers who run the analyzer act on these answers without running
18	however, is itself a program, and its errors produce confident answers that	18	the program, so the answers have to be right. The analyzer, however, is itself a program, and its errors produce

Old	77e07947	New	0edeac81
19	real executions contradict. In Goblint	19	confident answers that real executions contradict. The issue tracker of
20	#link("https://github.com/goblint/analyzer/issues/1156")[issue 1156], Goblint	20	Goblint, an analyzer for C programs @vojdani16 @seidl26, lists more than a
21	claimed that `c % 2 == 1` holds for $c in {-5, -7}$, although the	21	hundred #link("https://github.com/goblint/analyzer/issues?q=label%3Aunsound")[issues labeled as unsound]
22	truncating remainder of C11 @iso-c11[§6.5.5p6] gives -1 for both values. The	22	that have been opened since 2021. In
23	congruence domain had ignored the sign of the dividend, and	23	#link("https://github.com/goblint/analyzer/issues/1156")[issue 1156], for
		24	instance, Goblint claimed that `c % 2 == 1` holds for $c in {-5, -7}$, although
		25	the truncating remainder of C11 @iso-c11[§6.5.5p6] gives -1 for both values.
		26	Goblint's congruence domain had ignored the sign of the dividend, and
24	#link("https://github.com/goblint/analyzer/pull/1161")[pull request 1161] fixed	27	#link("https://github.com/goblint/analyzer/pull/1161")[pull request 1161] fixed
25	it @goblint1161. An error of this kind is worse than imprecision. An imprecise	28	it @goblint1161. An error of this kind is worse than imprecision. An imprecise
26	answer leaves the check undecided, whereas a wrong answer gives a user who	29	answer leaves the question undecided, whereas a wrong answer gives a developer
27	trusts it no reason to look for the error.	30	who trusts it no reason to look for the error.
28		31	
29	Fixing one such bug does not rule out the next. Ruling out all of them requires	32	Fixing one such bug does not rule out the next. Ruling out every such bug
30	a proof that the analyzer's answers hold for every execution of every program,	33	requires a proof that the analyzer's answers hold for every execution of every
31	and that proof must itself be checked. Formal verification states such a claim	34	program. Such a proof is too long to check reliably by reading it. In a proof
32	as a proposition over explicit definitions, and a proof assistant checks its	35	assistant such as Isabelle/HOL @nipkow14, definitions and theorems are stated in a formal logic,
33	proof @nipkow14. Isabelle/HOL, one such proof assistant, constructs every theorem	36	and a small trusted kernel checks every inference step of every proof
34	through a small trusted inference kernel, following the LCF approach @paulson19. Trusting a proof	37	@paulson19. An error in a proof therefore stops the proof, and the faulty
35	therefore means trusting this kernel and the system it runs on, not the tactics	38	theorem is never established.
36	that produced the proof.		
37		39	
38	The kernel, however, checks a proof only against the definitions it is given.	40	This thesis presents Voblint, short for verified Goblint. Voblint is an
39	Whether those definitions model the intended objects, and whether the theorem	41	analyzer modeled on Goblint's architecture and formalized in Isabelle/HOL,
40	expresses the intended claim, is left to human review. For example, a review of parts of an	42	together with a machine-checked proof that its answers hold for every execution
41	Isabelle/HOL formalization of Munkres' general topology found	43	of the analyzed program. It analyzes VIMP, a small C-like language with integer
42	definitions that are logically weaker than the textbook's. They are harmless	44	variables and recursive procedures. Isabelle's code generator exports the
43	only because each theorem assumes the missing constraints again	45	proved analysis function to OCaml, where a command-line tool runs it. The same
44	@bryant26munkres[§8.1]. Verification projects therefore state	46	code runs in a #link(site("playground.html"))[browser playground], a web page
45	explicitly what lies outside the proof. The seL4 proof, for example, relates	47	that analyzes a program the reader enters. The soundness theorem starts from
46	the kernel's C implementation to an abstract specification in Isabelle/HOL and	48	_source executions_, the executions of a program under the small-step semantics #isaconst("pstep")
47	names the compiler, assembly code, boot code, cache management, and hardware as	49	of VIMP. Every later representation, from the control-flow graph to the
48	assumptions. It also assumes that its formal model of C reflects the C standard	50	solver's result, is connected to source executions by proof. The kernel,
49	@klein09.	51	however, checks proofs only against the definitions it is given. Whether
50		52	#isaconst("pstep") describes the intended language, its _adequacy_, therefore
51	This thesis builds Voblint, an executable analyzer whose answers are	53	lies outside the proof. @sec:vimp-vs-c argues it by comparing VIMP with C11.
52	proved to hold for every execution of the analyzed program. Like these		
53	projects, it names the assumptions under which its proof holds. Voblint draws		

Old	77e07947	New	0edeac81
54	its boundary at		
55	the source language. Its end-to-end soundness theorem starts from _source executions_, the		
56	executions of a program under the small-step semantics #isaconst("pstep") of		
57	Voblint's input language VIMP. Every later representation, from the		
58	control-flow graph to the solver's result, is connected to source executions by		
59	proof. Of the semantics, only #isaconst("pstep") therefore needs an adequacy		
60	argument instead of a proof. That argument states which fragment of C it models, where it departs		
61	from C11 (@sec:vimp-vs-c), and why no existing verified semantics can be used		
62	instead.		
63		54	
64	== Sound static analysis and its verdicts <sec:intro-static>	55	== Sound static analysis and its verdicts <sec:intro-static>
65		56	
66	Software is usually checked by testing. A failing test demonstrates a bug, but	57	Software is usually checked by testing. A failing test shows a bug, but
67	a passing test suite has observed only finitely many executions	58	a passing test suite has observed only finitely many executions
68	@rival20[§1.4.1]. A static analysis reasons about all	59	@rival20[§1.4.1]. A static analysis reasons about all
69	executions instead. At each program point, it computes an _abstract state_. An	60	executions instead. At each program point, it computes an _abstract state_. An
70	abstract state is a finite representation of a set of program states, for	61	abstract state is a finite representation of a set of program states, for
71	instance, an interval	62	instance, an interval
72	that contains every value a variable takes whenever execution reaches the	63	that contains every value a variable takes whenever execution reaches the
73	point. The analysis is _sound_ if the abstract state contains every state that	64	point. The analysis is _sound_ if the abstract state contains every state that
74	occurs in a real run, although it may contain more. Such a result is an	65	occurs in a real run, although it may contain more. Such a result is an
75	_over-approximation_. If a sound abstract state	66	_over-approximation_. If a sound abstract state
76	contains no state that violates a check, no execution violates it, tested or	67	contains no state that violates a condition, no execution violates it, tested or
77	not (@fig:intro-runs). The converse does not hold.	68	not (@fig:intro-runs). The converse does not hold.
78	An abstract state can contain violating states that no execution reaches, and	69	An abstract state can contain violating states that no execution reaches, and
79	the analysis then cannot decide the check, even when every execution satisfies	70	the analysis then cannot decide the condition, even when every execution satisfies
80	it.	71	it.
81		72	
82	Suppose the analysis computes $x in [43, +infinity)$ at a point that divides	73	Suppose the analysis computes $x in [43, +infinity)$ at a point that divides
	20 unchanged lines hidden		
103	For a general-purpose language, this incompleteness is unavoidable. By Rice's	94	For a general-purpose language, this incompleteness is unavoidable. By Rice's
104	theorem, no algorithm decides a nontrivial semantic property for every program	95	theorem, no algorithm decides a nontrivial semantic property for every program
105	of a Turing-complete language @rice53 @rival20[§1.3.3]. A sound analysis may	96	of a Turing-complete language @rice53 @rival20[§1.3.3]. A sound analysis may
106	therefore raise false alarms and leave checks undecided, but it never makes a	97	therefore raise false alarms and leave questions undecided, but it never makes a
107	false claim. It is	98	false claim. It is
108	useful as long as it decides the checks of interest often enough	99	useful as long as it decides the questions of interest often enough
109	@rival20[§1.3.5]. Since every sound analysis over-approximates the same	100	@rival20[§1.3.5]. Since every sound analysis over-approximates the same
110	executions, analyses can specialize. One tracks intervals, another parities, and	101	executions, analyses can specialize. One tracks intervals, another parities, and
111	another relations between variables. If any one of them, or their combination,	102	another relations between variables. If any one of them, or their combination,
112	proves a check, the check holds in every run that reaches it. An analysis can also separate the	103	proves a condition, the condition holds in every run that reaches it. An analysis can also separate the
113	activations of a procedure by their _calling context_, an abstraction of how	104	activations of a procedure by their _calling context_, an abstraction of how

114	the call was reached, such as the call site or the abstract state at entry. It	105	the call was reached, such as the call site or the abstract state at entry. It
115	then keeps one abstract state per program point and context, and it may analyze	106	then keeps one abstract state per program point and context, and it may analyze
116	one call in several contexts, for instance, one per abstract state that reaches	107	one call in several contexts, for instance, one per abstract state that reaches
117	the call. A richer domain or more contexts	108	the call. A richer domain or more contexts
118	can decide more checks, but they also make the analysis more expensive. If there	109	can decide more conditions, but they also make the analysis more expensive. If there
119	are	110	are
120	many contexts, the analysis may not finish in practice.	111	many contexts, the analysis may not finish in practice.
121		112	
122	Voblint answers every check that the user writes into the source program as	113	A user of Voblint states such a condition by writing a _check_
123	`__voblint_check(e)`. It also warns at every division and remainder where it cannot	114	`__voblint_check(e)` into the program, and Voblint answers each check with a verdict. It also warns
124	exclude a zero divisor. @fig:intro-answers shows a run in the	115	at every division and remainder where it cannot exclude a zero divisor.
125	#link(site("playground.html"))[browser	116	@fig:intro-answers shows a run in the playground together with what each answer
126	playground] together with what each answer guarantees. #verdict("PROVED") and	117	guarantees. #verdict("PROVED") and
127	#verdict("REFUTED") are the _definite_ verdicts, and #verdict("UNKNOWN") claims	118	#verdict("REFUTED") are the _definite_ verdicts, and #verdict("UNKNOWN") claims
128	nothing. Only #verdict("DEAD") makes a claim about reachability, and	119	nothing. Only #verdict("DEAD") makes a claim about reachability, and
129	#verdict("REFUTED") is not a verified counterexample. Voblint offers several abstract domains and context policies	120	#verdict("REFUTED") is not a verified counterexample. Voblint offers several abstract domains and context policies
130	(@ch:instances), and the run shown uses intervals and the unit context policy.	121	(@ch:instances), and the run shown uses intervals and the unit context policy, which gives every
		122	call the same context.
131		123	
132	#let _ans-warn = check-row("intro-answers", cond: "WARNING")	124	#let _ans-warn = check-row("intro-answers", cond: "WARNING")
133	#let _ans-proved = check-row("intro-answers", cond: "0 <= p && p <= 100")	125	#let _ans-proved = check-row("intro-answers", cond: "0 <= p && p <= 100")
	31 unchanged lines hidden		
165	table(	157	table(
166	columns: (auto, auto, 1fr),	158	columns: (auto, auto, 1fr),
167	align: (right + horizon, left + horizon, left + horizon),	159	align: (right + horizon, left + horizon, left + horizon),
168	stroke: none,	160	// Rules between the body rows keep each guarantee with its line.
		161	stroke: (_, y) => if y > 1 { (top: 0.3pt + vb.muted.lighten(50%)) },
169	inset: (x: 3pt, y: 2.5pt),	162	inset: (x: 3pt, y: 2.5pt),
170	table.hline(stroke: 0.5pt),	163	table.hline(stroke: 0.5pt),
171	[*Line*], [*Answer*], [*Guarantee*],	164	[*Line*], [*Answer*], [*Guarantee*],
	32 unchanged lines hidden		
204	caption: [One analyzer run on a clamp function, as the browser playground	197	caption: [One analyzer run on a clamp function, as the browser playground
205	shows it, with the interval domain and the unit context policy. The analysis	198	shows it, with the interval domain and the unit context policy. The analysis
206	covers every value of `t` at once. The screenshot opens the run in the	199	covers every value of `t` at once. The screenshot opens the run in the
207	playground (claim #claim-ref("intro-answers"), fixture	200	playground. The program is the regression test
208	#fixture("24-site-figures/precision/48-clamp_every_answer.vimp")).],	201	#fixture("24-site-figures/precision/48-clamp_every_answer.vimp").],
209	) <fig:intro-answers>	202	) <fig:intro-answers>
210		203	
211	== What existing analyzers and proofs leave open <sec:state-of-art>	204	== Soundness of existing analyzers <sec:state-of-art>
212		205	
213	Most abstract interpreters in practical use have no machine-checked soundness	206	Most abstract interpreters in practical use have no machine-checked soundness

214	proof and rely on testing instead. Livshits et al. know of no realistic	207	proof and rely on testing instead. Livshits et al. know of no realistic
	10 unchanged lines hidden		
225	such analyses to state explicitly	218	such analyses to state explicitly
226	the nature and extent of the unsoundness @livshits15[p. 45]. Goblint,	219	the nature and extent of the unsoundness @livshits15[p. 45]. Goblint,
227	for instance, documents such assumptions. It assumes that pointer arithmetic	220	for instance, documents such assumptions. It assumes that pointer arithmetic
228	does not overflow and, by default, that array reads stay in bounds, and it notes	221	does not overflow and, by default, that array reads stay in bounds, and calls this list likely incomplete @goblint-assumptions.
229	that this list is likely incomplete @goblint-assumptions.		
230		222	
231	A documented assumption tells the user where a soundness claim ends. Within	223	A documented assumption tells the user where a soundness claim ends. Within
232	that boundary, however, the claim still rests on an implementation that no one	224	that boundary, however, the claim still rests on an implementation that no one
233	has proved correct. Even the analyzers in practical use that are designed to be	225	has proved correct. Even the analyzers in practical use that are designed to be
234	sound are tested rather than proved (@sec:rel-practice), and testing an	226	sound are tested rather than proved (@sec:rel-practice), and testing an
235	analyzer is incomplete in the same way as testing any other program	227	analyzer is incomplete in the same way as testing any other program
236	(@sec:intro-static). Since its first	228	(@sec:intro-static). Since its first
237	participation in SV-COMP in 2021, a competition on benchmark programs with	229	participation in SV-COMP in 2021, a competition on benchmark programs with
238	known verdicts, Goblint has never produced a false negative on the	230	known verdicts, Goblint has never produced a false negative on the
239	competition's benchmark suite @seidl26. Still, its issue tracker lists more	231	competition's benchmark suite @seidl26, although its unsoundness issues were
240	than a hundred #link("https://github.com/goblint/analyzer/issues?q=label%3Aunsound")[issues labeled as unsound]	232	opened during the same years. A test finds an error only when someone thinks
241	that have been opened since 2021.	233	of exactly the scenario that triggers it, and every other error waits until it
242	The two observations do not contradict each other. A test finds an error only when	234	misleads a user who then reports it. The remainder defect of issue 1156
243	someone thinks of exactly the scenario that triggers it, and every other error	235	needed a negative dividend and was reported only after Goblint had claimed a
244	waits until it misleads a user who then reports it. The Goblint defect at the	236	false result. A proof covers every input instead. If every abstract operation
245	start of this chapter needed a negative dividend in the congruence domain and	237	is proved sound, a congruence remainder that ignores the sign of the dividend
246	was reported only after Goblint had claimed a false result. A proof does not	238	cannot pass, and errors of this kind are ruled out completely.
247	depend on the right scenario. If every abstract operation is proved		
248	sound for every input, a congruence remainder that ignores the sign of the		
249	dividend cannot pass, and errors of this kind are ruled out completely.		
250		239	
251	A proof about single operations, on paper or in a proof assistant, however, protects the	240	A proof about single operations, on paper or in a proof assistant, however, protects the
252	user only if a theorem connects these operations to the tool the user runs. Between the operations and the	241	user only if a theorem connects these operations to the tool the user runs. Between the operations and the
253	reported verdict, the analyzer compiles the program, assigns calling contexts,	242	reported verdict, the analyzer compiles the program, assigns calling contexts,
254	and solves equations, and each of these steps could lose executions. CompCert	243	and solves equations, and each of these steps could lose executions. CompCert
255	shows that such a theorem is possible for a realistic tool. It proves that its	244	shows that such a theorem is possible for a realistic tool. It proves that its
256	compiled code behaves as the source semantics specifies, and it lists the	245	compiled code behaves as the source semantics specifies, and it lists the
257	components that remain trusted @leroy09. Verasco carries the approach to a	246	components that remain trusted @leroy09. Verasco carries the approach to a
258	static analyzer for C @jourdan15, and several other abstract interpreters have	247	static analyzer for C @jourdan15, and several other abstract interpreters have
259	machine-checked soundness proofs as well. None of the compared works, however,	248	machine-checked soundness proofs as well. None of these approaches
260	combines recursive procedures, calling contexts, and a verified solver	249	(@sec:rel-mechanized), however, combines recursive procedures, calling
261	(@sec:rel-mechanized). Goblint's architecture uses the first	250	contexts, and a verified solver. Goblint's architecture uses the first
262	two and a generic solver, whose top-down algorithm has since been verified. It	251	two and a generic solver, whose top-down algorithm has since been verified. It

```
analyzes recursive programs and keeps one abstract state per program point and
calling context.
```

12 unchanged lines hidden

```
contexts. Under some context policies, a callee's contexts become known only
while the system is solved, so each call contributes its entry state to the
callee instead of the callee listing its callers in advance (@sec:side-effects). For the
solver itself, a proof already exists. The top-down (TD)
```

```
solver that Goblint uses, including its extension to side effects, has been formalized
and proved partially correct in Isabelle/HOL @stade24 @tilscher26. When it
terminates, it returns values that satisfy the inequalities of the queried
unknown and of every unknown it transitively reads
(#isaconst("part_post_solution", thy: "Basics_side")). Whether the equation
system describes the program, and whether the verdicts read off its solution
hold, lies outside that theorem.

The solver theorem, however, says nothing about programs. To connect a program
to its equation system, every abstract state must be compared with the
```

```
solver algorithm, including an extension to side effects, has been formalized
and proved partially correct in Isabelle/HOL @stade24 @tilscher26. When it
terminates, it returns values that satisfy the inequalities of the queried
local unknown and of every local unknown it transitively reads
(#isaconst("part_post_solution", thy: "Basics_side")). Whether the equation
system describes the program, and whether the verdicts computed from its solution hold, lies
outside that theorem.

The solver theorem, however, says nothing about programs. To connect a program
to its equation system, every abstract state must be compared with the
```

16 unchanged lines hidden

```
== Contributions <sec:contributions>

The contribution of this thesis is the Isabelle/HOL formalization of Voblint and
its machine-checked soundness proof, available at
```

```
== Contributions <sec:contributions>

The soundness proof of Voblint closes the gaps of @sec:state-of-art with a
chain of inclusions at every program point. The chain leads from source
executions to the verdicts of the _report_ that the analysis function returns
(@fig:intro-nest). Each step may add stores that no execution reaches, which
costs precision, but it must not lose a reachable store. A compiler that drops
an edge or a return to the wrong caller would lose such a store, and the proof
excludes both errors. The Isabelle/HOL formalization and its proof are available at
```

```
#link(
  "https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis",
)[`github.com/ManuelLerchner/Voblint-Verification-Master-Thesis`]
and archived on Zenodo @voblint. The proof closes the gaps of
@sec:state-of-art with a chain of inclusions at every program point, from
source executions to the reported verdicts (@fig:intro-nest). Each step may add
stores that no execution reaches, which costs precision, but it must not lose a
reachable store. Errors such as a compiler that drops an edge or a return to
the wrong caller would lose such a store, and the proof excludes them.

#figure(
  {
```

```
#link(
  "https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis",
)[`github.com/ManuelLerchner/Voblint-Verification-Master-Thesis`]
and archived on Zenodo @voblint.

#figure(
  {
```

20 unchanged lines hidden

```
        vb.accent,
        [Report],
```

```
        vb.accent,
        [Report],
```

```
        [stores the states #isaconst("report_states_at") lists at $v$ describe,
          which cover the collecting semantics because the solver's result is a
          post-solution of the equations (@ch:equations to @ch:results)],
        ring(
          vb.cong,
          [Collecting semantics],
```

20 unchanged lines hidden

```
  },
  kind: image,
  placement: none,
  caption: [Soundness at a program point $v$ as nested sets. A valid trace
    records, besides the stores of a graph execution, which call created each
    activation, and every graph execution is represented by one. Splitting the node
    collecting semantics by context loses no store. The end-to-end soundness
    theorem chains these inclusions for every report the analyzer returns
    (#isathm("run_voblint_source_sound"), @ch:results).],
) <fig:intro-nest>

Voblint closes these gaps by combining existing ideas, some of them already
formalized and others so far only described on paper or implemented. Its
overall design mirrors Goblint's architecture inside Isabelle/HOL, without
formalizing Goblint's implementation or C (@sec:rel-goblint). To the author's
knowledge, no prior mechanized analyzer connects a source semantics, through an
activation collecting semantics, indexed by context, to a side-effecting constraint system
and
its verified solver (@sec:rel-summary). Within this design, the formalization makes five
contributions.

- _End-to-end soundness._ The theorem connects the obligations of single
  operations to the tool the user runs. Whenever the analysis function
  #isaconst("run_voblint") returns a report, its definite verdicts
  (#verdict("PROVED") and #verdict("REFUTED")) hold for every source execution
  from an initial store with zeroed globals that reaches the corresponding
  program point, under every analysis configuration the function offers
  (#isathm("run_voblint_source_sound"), @sec:headline). Companion theorems
  justify #verdict("DEAD") and the absence of arithmetic warnings
  (@ch:results).
- _A concrete semantics of calling contexts._ Activation traces give the
  per-context states of an analysis a concrete counterpart. Each trace follows
  one procedure activation and records the activation that called it, so the
  contexts of a call can be read off the execution. Splitting the concrete
  stores by context loses no execution (@sec:contract), and every report covers
```

```
        [stores the states #isaconst("report_states_at") lists at $v$ describe,
          which cover the collecting semantics because the solver's result
          satisfies the inequalities of the constraint system (@ch:equations to
@ch:results)],
        ring(
          vb.cong,
          [Collecting semantics],
```

```
  },
  kind: image,
  placement: none,
  caption: [Soundness at a program point $v$ as nested sets. An activation trace
    records, besides the stores of a graph execution, which call created each
    activation, and every graph execution is represented by one. Splitting the node
    collecting semantics by context loses no store. The end-to-end soundness
    theorem chains these inclusions for every report the analyzer returns
    (#isathm("run_voblint_source_sound"), @ch:results).],
) <fig:intro-nest>

The chain combines existing ideas, some of them already formalized and others
so far only described on paper or implemented. Voblint follows Goblint's
architecture, and it formalizes neither Goblint's implementation nor C
(@sec:rel-goblint). To the author's knowledge, no prior mechanized analyzer
connects a source semantics, through an activation collecting semantics indexed
by context, to a side-effecting constraint system and its verified solver
(@sec:rel-summary). The formalization makes five contributions.

- _End-to-end soundness._ The theorem connects the obligations of single
  operations to the tool the user runs. Whenever the analysis function
  #isaconst("run_voblint") returns a report, its definite verdicts
  (#verdict("PROVED") and #verdict("REFUTED")) hold for every source execution
  from an initial store with zeroed globals that reaches the corresponding
  program point, under every analysis configuration the function offers
  (#isathm("run_voblint_source_sound"), @sec:headline). Companion theorems
  justify #verdict("DEAD") and the absence of arithmetic warnings
  (@ch:results).
- _A concrete semantics of calling contexts._ Activation traces give the
  per-context states of an analysis a concrete counterpart, because the chain of
  calling activations determines the contexts of a call. Splitting the concrete
  stores by context loses no execution (@sec:contract), and every report covers
```

```
  each context separately (#isathm("run_voblint_context_covers")). One call may have several contexts,
  which lets an analysis split it into cases, for instance one per
  path-sensitive state.
- _Separately verified components._ Each component discharges local soundness
  obligations, and generic theorems compose them into the end-to-end guarantee
  for every configuration. Domains prove their primitive operations sound,
  context policies prove that their contexts agree with the concrete
  semantics, cooperating analyses prove conditions on the answers they
  exchange (@ch:cooperation), and the solver enters the proof only through
  proved facts about its result (@sec:certificate).
- _An executable analyzer._ Isabelle's code generator @haftmann10 exports
  #isaconst("run_voblint"), the constant the soundness theorem is about, to
  OCaml. A command-line tool and a browser playground run it on VIMP programs.
  The playground also draws the control-flow graph and replays the solver's steps.
- _Needed assumptions and nonempty results._ The kernel cannot tell whether a
  theorem assumes too much or concludes too little. Further theorems therefore
  show that the analyzer gives definite verdicts on concrete programs
  (@sec:nonvacuity). Dropping the obligation #oblig("TOTAL") admits answers that concrete executions
  contradict (@sec:contract), and one
  domain obligation rejects a defect that occurred in Goblint
  (@sec:eval-1161). Concrete programs show what each mechanism gains
  (@sec:eval-precision).

== Scope and limits of the guarantee <sec:intro-scope>

Not every part of the delivered analyzer is covered by the proof
(@fig:intro-trust). _End-to-end_ means that the proof reaches from source
executions under #isaconst("pstep") to the result of the analysis function,
while the parser, the handwritten OCaml for printing, and the trusted foundation lie outside
it. Nor does the proof cover Goblint itself, only the parts of its architecture
that Voblint isolates, over a small language with recursive procedures.

#figure(
  {
```

53 unchanged lines hidden

```
    )
  },
  placement: none,
  caption: [Where the proof starts and stops, for accepted programs whose solve
    terminates. The adequacy of #isaconst("pstep") is
```

```
  each context separately (#isathm("run_voblint_context_covers")). One call may
  have several contexts, which lets an analysis split it into cases.
- _Separately verified components._ Each component discharges local soundness
  obligations, and generic theorems compose them into the end-to-end guarantee
  for every configuration. Domains prove their primitive operations sound,
  context policies prove that their contexts agree with the concrete
  semantics, cooperating analyses prove conditions on the answers they
  exchange (@ch:cooperation), and the solver enters the proof only through
  proved facts about its result (@sec:certificate).
- _An executable analyzer._ The command-line tool and the playground run code
  generated @haftmann10 from #isaconst("run_voblint"), the same constant the
  soundness theorem is about. The playground also draws the control-flow graph
  and replays the solver's steps.
- _Needed assumptions and nonempty results._ The kernel cannot tell whether a
  theorem assumes too much or concludes too little. Further theorems therefore
  show that the analyzer gives definite verdicts on concrete programs
  (@sec:nonvacuity). The obligation #oblig("TOTAL") requires
  that every call gets at least one calling context. Dropping it admits answers
  that concrete executions contradict (@sec:contract), and one
  domain obligation rejects a defect that occurred in Goblint
  (@sec:eval-1161). Concrete programs show what each mechanism gains
  (@sec:eval-precision).

== Scope of the soundness theorem <sec:intro-scope>

_End-to-end_ means that the proof reaches from source executions under
#isaconst("pstep") to the result of the analysis function (@fig:intro-trust).
The list after the figure collects the limits of the theorem.

#figure(
  {
```

```
    )
  },
  placement: none,
  caption: [Where the proof starts and stops, for well-formed programs whose solve
    terminates. The adequacy of #isaconst("pstep") is
```

496	argued (@sec:vimp-vs-c).],
497	) <fig:intro-trust>
498	
499	The theorem has the following limits.
500	
501	- _Language and properties._ VIMP has no pointers, heap, or threads. Its
502	integers are unbounded, division by zero is defined, and a callee's locals
503	start at 0. Therefore, a verdict
504	does not always hold for a C program with the same text (@sec:vimp-vs-c).
505	Voblint covers checks (`__voblint_check`) and division by zero, while Goblint also checks
506	overflows, data races, memory
507	safety, memory leaks, and program termination @saan26phd[§3.3].
508	- _Adequacy and trusted components._ It is argued, but not proved, that #isaconst("pstep") models the intended language (@sec:vimp-vs-c). The parser, the code generator, and the target
509	toolchains lie outside the proof (@fig:intro-trust).
510	- _Partial correctness._ The theorem covers every report the analyzer returns.
511	Termination of the solve is not proved for every program (@sec:certificate).
512	- _Precision and cost._ No completeness or general precision ordering is
513	proved (@sec:eval-precision). The executable uses the same data
514	representations as the proofs and was not optimized beyond one fixed
515	performance defect (@sec:codegen). Running time is measured only to show
516	that the analyzer finishes in acceptable time on the corpus
517	(@sec:eval-runtime).
518	- _Goblint and evidence._ The correspondence to Goblint is architectural, and
519	no theorem transfers to Goblint's implementation (@sec:rel-goblint). The
520	regression corpus is small and was written or adapted for this work
521	(@sec:eval-corpus).
522	
523	== Thesis outline along the proof chain <sec:roadmap>
524	
525	The nested sets of @fig:intro-nest say what the theorem claims. @fig:metro
526	shows how the proof establishes it. Each line groups the results of one layer
527	of the development, and each interchange is a theorem that uses the results of
528	the lines that meet there. The labels name the theorems, and the caption gives
529	the chapter that presents each line.

480	argued (@sec:vimp-vs-c).],
481	) <fig:intro-trust>
482	
483	
484	- _Language and properties._ VIMP has no pointers, heap, or threads. Its
485	integers are unbounded, division by zero is defined, and a callee's locals
486	start at 0. Therefore, a verdict
487	does not always hold for a C program with the same text (@sec:vimp-vs-c).
488	Voblint covers checks (`__voblint_check`) and division by zero, while Goblint also checks
489	overflows, data races, memory
490	safety, memory leaks, and program termination @saan26phd[§3.3].
491	- _Adequacy and trusted components._ The adequacy of #isaconst("pstep") is
492	argued in @sec:vimp-vs-c. The parser, the handwritten OCaml for printing, the
493	code generator, and the target toolchains lie outside the proof
494	(@fig:intro-trust).
495	- _Partial correctness._ The theorem covers every report the analyzer returns.
496	Termination of the solve is not proved for every program (@sec:certificate).
497	- _Precision and cost._ No completeness or general precision ordering is
498	proved (@sec:eval-precision). The executable uses the same data
499	representations as the proofs and was not optimized beyond one fixed
500	performance defect (@sec:codegen). Running time is measured only to show
501	that the analyzer finishes in acceptable time on the regression corpus
502	(@sec:eval-runtime).
503	- _Goblint and evidence._ The correspondence to Goblint is architectural, and
504	no theorem transfers to Goblint's implementation (@sec:rel-goblint). The
505	regression corpus is small and was written or adapted for this work
506	(@sec:eval-corpus).
507	
508	== Structure of the proof <sec:roadmap>
509	
510	@fig:intro-nest states what the end-to-end soundness theorem claims, and
511	@fig:metro draws its proof as a metro map. A line collects the results of one
512	layer of the development, for example the compiler and its simulation proof.
513	Where several lines meet, a theorem combines their results, and the label at
514	the interchange names that theorem. The result line leaves the central
515	interchange and splits into one branch for policies that choose the context
516	without reading abstract states and one for entry-state policies. The branches
517	rejoin at one common theorem. From there, the coverage of the stores joined
518	over all contexts leads directly to the terminals for #verdict("DEAD")
519	(#isathm("run_voblint_dead_check_unreached")) and for SAFE, which says that a
520	division without a #verdict("WARNING") never divides by zero

		521	(#isathm("run_voblint_arithmetic_safe")). The terminal for #verdict("PROVED")
		522	(#isathm("run_voblint_check_sound")) goes through the end-to-end soundness
		523	theorem #isathm("run_voblint_source_sound"), which also uses the compiler line
		524	to place a source execution at a graph node. A second branch uses the compiler
		525	line in the same way for the chain that keeps the contexts apart.
530		526	
531	#figure(	527	#figure(
532	image("/shared/generated/svg/metro.svg", width: 100%),	528	image("/shared/generated/svg/metro.svg", width: 100%),
533	placement: auto,	529	placement: auto,
534	caption: [A map of the proof with its interior definitions omitted. The	530	caption: [A map of the proof with its interior definitions omitted. The
535	#link(	531	#link(
536	site("index.html#proof-map"),	532	site("index.html#proof-map"),
537	)[project site]	533	)[project site]
538	shows it interactively, with every station linked to its theory. Each line	534	shows it interactively, with every station linked to its theory. Each line
539	groups the results of one layer: #metro-line(metro.sem)[Semantics], the	535	groups the results of one layer: #metro-line(metro.sem)[Semantics], the
540	source and trace semantics (#partref(<part:over-approx>), @ch:program-model	536	source and trace semantics (@ch:program-model and @ch:traces), #metro-line(metro.comp)[Compiler], the compiler and its
541	and @ch:traces), #metro-line(metro.comp)[Compiler], the compiler and its	537	simulation (@ch:program-model),
542	simulation (#partref(<part:over-approx>), @ch:program-model),		
543	#metro-line(metro.dom)[Domain], what a domain proves and how the analyses	538	#metro-line(metro.dom)[Domain], what a domain proves and how the analyses
544	cooperate (#partref(<part:analyses>), @ch:domains to @ch:cooperation, and the	539	cooperate (@ch:domains to @ch:cooperation and the
545	order analysis of @ch:instances), #metro-line(metro.fw)[Framework], the	540	order analysis of @ch:instances), #metro-line(metro.fw)[Framework], the
546	analysis contract, routing and coverage (@ch:traces, @ch:analysis-interface	541	analysis contract, routing and coverage (@ch:traces, @ch:analysis-interface
547	and @ch:equations), #metro-line(metro.sol)[Solver], the verified TD solver	542	and @ch:equations), #metro-line(metro.sol)[Solver], the verified TD solver
548	(#partref(<part:verdicts>), @ch:solving), and #metro-line(metro.res)[Result],	543	(@ch:solving), and #metro-line(metro.res)[Result],
549	from the solved equations to the verdicts (#partref(<part:verdicts>),	544	from the solved equations to the verdicts (@ch:equations and @ch:results).],
550	@ch:equations and @ch:results). Wide markers are interchanges, where a theorem uses the		
551	results of the lines that meet. The result line leaves the central		
552	interchange, splits for functional and entry-state context policies, and		
553	rejoins at one common theorem. From there the context-free coverage leads		
554	directly to the terminals for #verdict("DEAD")		
555	(#isathm("run_voblint_dead_check_unreached")) and for SAFE, which says that a division without a		
556	#verdict("WARNING") never divides by zero (#isathm("run_voblint_arithmetic_safe")). The terminal		
557	for #verdict("PROVED") (#isathm("run_voblint_check_sound")) goes through the		
558	end-to-end soundness theorem		
559	#isathm("run_voblint_source_sound"), which also uses the compiler line to		
560	place a source execution at a graph node. A second branch uses the compiler line		
561	in the same way for the chain that keeps the contexts.],		
562	) <fig:metro>	545	) <fig:metro>
563		546	
564	The proof map selects the results that connect the semantic layers.	547	The proof map selects the results that connect the semantic layers.

```
@fig:intro-corpus shows the surrounding formal development as a treemap of
exported Isabelle entities. It groups the declarations by session and theory,
including facts generated by Isabelle's definition packages. The area of a
block therefore measures its share of the exported corpus, not the difficulty
of its proofs. The interactive view links each declaration to its documentation.

#figure(
  link(site("proof-map.html?view=treemap"), image("/assets/proof-map-treemap.png", width: 100%)),
  placement: auto,
  caption: [The formalization's exported declarations, captured from the
    #link(site("proof-map.html?view=treemap"))[interactive treemap].
    Blocks group sessions and theories; their areas follow entity counts.
    Colors identify session dependency layers. Gold diamonds mark thesis
    citations, and red stars mark selected source-level soundness results.
    This is a corpus overview; it does not depict proof dependencies.],
) <fig:intro-corpus>

#partref(<part:problem>) consists of this introduction and @ch:background,
which recalls abstract interpretation, constraint systems, calling contexts,
side-effecting constraint systems, and the verified TD solver.

#partref(<part:over-approx>) fixes what an analysis must over-approximate:
source programs and their graphs (@ch:program-model) and activation traces with
the coverage contract (@ch:traces). The contract reduces soundness to five
local obligations about a _cover_, one set of stores per node and context, and
every later part works toward a cover that meets them.

#partref(<part:analyses>) states what domains, analyses, and cooperating analyses
must prove (@ch:domains to @ch:cooperation). A domain proves its primitives
sound once, an analysis supplies abstract transfer functions with
the analysis contract, and cooperating analyses prove conditions on the answers
they exchange.

#partref(<part:verdicts>) builds the equations, solves them, and composes the
end-to-end soundness theorem (@ch:equations to @ch:results). Every post-solution of the
equations induces a cover that meets the contract (@ch:equations), the verified TD
solver returns such a post-solution whenever it terminates (@ch:solving), and
the report read off it is sound for every source execution (@ch:results).

#partref(<part:instances>) instantiates the theorem (@ch:instances), follows
#isaconst("run_voblint") to the delivered tools and their trust boundary
(@ch:executable), and evaluates it (@ch:evaluation).
```

```
@fig:intro-corpus shows the surrounding formal development as a treemap of
exported Isabelle entities. It groups the declarations by session and theory,
including facts generated by Isabelle's definition packages. The area of a
block therefore measures its share of the exported corpus, not the difficulty
of its proofs. The interactive view links each declaration to its documentation.

#figure(
  link(site("proof-map.html?view=treemap"), image("/assets/proof-map-treemap.png", width: 90%)),
  placement: auto,
  caption: [The formalization's exported declarations, captured from the
    #link(site("proof-map.html?view=treemap"))[interactive treemap].
    Blocks group sessions and theories; their areas follow entity counts.
    Colors identify session dependency layers. Gold diamonds mark thesis
    citations, and red stars mark selected source-level soundness results.
    This is a corpus overview; it does not depict proof dependencies.],
) <fig:intro-corpus>

#partref(<part:problem>) introduces the problem and recalls abstract interpretation,
constraint systems, and the verified TD solver (@ch:background).

#partref(<part:over-approx>) defines source programs and their graphs
(@ch:program-model), then activation traces and the coverage contract
(@ch:traces). The later parts establish this contract.

#partref(<part:analyses>) develops the soundness obligations for domains
(@ch:domains), analyses (@ch:analysis-interface), and their cooperation
(@ch:cooperation).

#partref(<part:verdicts>) builds the equations (@ch:equations), solves them
(@ch:solving), and derives sound source-level verdicts (@ch:results).

#partref(<part:instances>) presents the analysis instances (@ch:instances),
the executable and its trust boundary (@ch:executable), and the evaluation
(@ch:evaluation).
```

607		582	
608	#partref(<part:assessment>) compares Voblint with prior work (@ch:related) and	583	#partref(<part:assessment>) discusses related work (@ch:related) and concludes
609	concludes (@ch:conclusion). @sec:isabelle explains the Isabelle syntax of the	584	(@ch:conclusion). @sec:isabelle explains the Isabelle listings, @app:notation
610	listings, @app:notation lists the notation, and @sec:voblint-repo describes the repository, its	585	collects the notation, and @sec:voblint-repo describes the repository and tooling.
611	checks, and the source tooling built for this work.		

thesis/content/02-background.typ

77e07947 → 0edeac81

759 old lines, 748 new lines

94% similar lines **+38** **-49** **18 changed blocks**

Old	77e07947	New	0edeac81
1	#import "../lib/math.typ": *	1	#import "../lib/math.typ": *
2	#import "../lib/code.typ": (	2	#import "../lib/code.typ": (
3	isacmd, isaconst, isai, isalocale, isasession, isathm, isatype, listing, playground-link,	3	isacmd, isaconst, isai, isalocale, isasession, isathm, isatype, listing, playground-link,
	128 unchanged lines hidden		
132		132	
133	@sec:state-of-art ended with a plan. Each step from source executions to the	133	@sec:state-of-art ended with a plan. Each step from source executions to the
134	verdicts should relate two sets of concrete stores, so that a proof can check	134	verdicts should relate two sets of concrete stores, so that a proof can check
135	that the step loses nothing. This chapter introduces the ideas needed to state	135	that the step loses nothing. Stating these steps requires the
136	these steps. It first describes what a program can actually do, then how an	136	stores a program can actually reach, the coarser descriptions an analysis
137	analysis describes the same behavior more coarsely, and finally how a solver	137	computes for them, and the solver that computes these descriptions. A loop that
138	computes this description. A loop that counts from 0 to 5 is the running	138	counts from 0 to 5 is the running example for all three.
139	example for all three.		
140		139	
141	== Control-flow graphs and the collecting semantics <sec:collecting>	140	== Control-flow graphs and the collecting semantics <sec:collecting>
142		141	
143	An analysis describes the states a program can be in at each of its points, so	142	An analysis describes the states a program can be in at each of its points, so
144	the points are made explicit first. A _control-flow graph_ (CFG) has the program	143	the points are made explicit first. A _control-flow graph_ (CFG) has the program
145	points as nodes and an edge for every way control can pass from one point to	144	points as nodes and an edge for every way control can pass from one point to
146	the next, labeled with the assignment or condition executed on the way.	145	the next, labeled with the assignment or condition executed on the way.
147	@fig:counting-loop shows the graph of a loop that counts `i` from 0 to 5,	146	@fig:counting-loop shows the graph of the running example, which counts `i`
148	the running example. The loop becomes a cycle through h, b, and t, and	147	from 0 to 5. The loop becomes a cycle through h, b, and t, and
149	its _loop head_ h is the node through which control enters and exits the cycle.	148	its _loop head_ h is the node through which control enters and exits the cycle.
150		149	
151	#figure(	150	#figure(
	29 unchanged lines hidden		
181	programs with procedures, @ch:traces defines Voblint's version over activation	180	programs with procedures, @ch:traces defines Voblint's version over activation
182	traces (#isaconst("node_collect")).	181	traces (#isaconst("node_collect")).
183		182	
184	The collecting semantics is the exact answer an analysis approximates. An	183	The collecting semantics is the exact answer an analysis approximates, and it
185	analysis is _sound_ if, at every program point, its result describes every store	184	makes soundness (@sec:intro-static) precise. An analysis is sound if, at every
186	that the collecting semantics gives there. It may describe more stores,	185	program point v, its result describes every store in $cal(C)(v)$. It may
187	which costs precision, but never fewer.	186	describe more stores, which costs precision.

188		187	
189	== Lattices and fixpoints <sec:lattices>	188	== Lattices and fixpoints <sec:lattices>
190		189	
	22 unchanged lines hidden		
213	(#isalocale("complete_lattice")) has a join $lJoin X$ and a meet $lMeet X$ for	212	(#isalocale("complete_lattice")) has a join $lJoin X$ and a meet $lMeet X$ for
214	every subset X @nipkow14[Def. 13.1] @mine17[Def. 2.5].	213	every subset X @nipkow14[Def. 13.1] @mine17[Def. 2.5].
215		214	
216	Voblint does not need a complete lattice. Its solver requires only a join	215	Voblint's solver requires less than a complete lattice, namely a join
217	semilattice with a least element (#isalocale("bounded_semilattice_sup_bot"))	216	semilattice with a least element (#isalocale("bounded_semilattice_sup_bot"))
218	that carries the widening and narrowing operators of @sec:widening. The solver	217	that carries the widening and narrowing operators of @sec:widening. The solver
219	joins two values at a time, whenever a new value reaches an unknown, so	218	joins two values at a time, whenever a new value reaches an unknown, so
220	joins need to exist only for pairs. Joins of infinite sets would not help an	219	joins need to exist only for pairs. Joins of infinite sets would not help an
221	executable analyzer anyway, since it computes only with finite representations.	220	executable analyzer anyway, since it computes only with finite representations.
222		221	
223	A numeric domain adds two operations to this structure. Its top element	222	A numeric domain adds a top element and an intersection to this structure. The
224	describes a variable about which nothing is known, for instance, a value read	223	top element describes a variable about which nothing is known, for instance, a
225	from input. Refinement at a branch adds an intersection, which does not have to	224	value read from input. Refinement at a branch uses the intersection, which does not have to
226	be the exact meet. It must keep every value that both operands admit, and its	225	be the exact meet. It must keep every value that both operands admit, and its
227	result must lie below both operands (#isalocale("sound_intersection"),	226	result must lie below both operands (#isalocale("sound_intersection"),
228	@sec:branches). An intersection that admits more values than both operands	227	@sec:branches). An intersection that admits more values than both operands
	44 unchanged lines hidden		
273	when $l' <= l$ and $u <= u'$. The join	272	when $l' <= l$ and $u <= u'$. The join
274	$[l, u] ljoin [l', u'] = [min(l, l'), max(u, u')]$ is the smallest interval	273	$[l, u] ljoin [l', u'] = [min(l, l'), max(u, u')]$ is the smallest interval
275	containing both. An interval cannot have holes, so the join may contain values	274	containing both. An interval cannot have holes, so the join may contain values
276	that neither operand contains: $absval(ivl(0, 0)) ljoin absval(ivl(5, 5)) = absval(ivl(0, 5))$. The meet	275	that neither operand contains: $absval(ivl(0, 0)) ljoin absval(ivl(5, 5)) = absval(ivl(0, 5))$. The meet intersects the bounds. These operations form the textbook interval domain. Voblint's representation
277	intersects the bounds. This is the textbook domain. Voblint's representation		
278	#isatype("ivl") admits every pair of bounds, so $lbot$ is one of several empty	276	#isatype("ivl") admits every pair of bounds, so $lbot$ is one of several empty
279	pairs (@sec:domain-carrier-laws).	277	pairs (@sec:domain-carrier-laws).
280		278	
	260 unchanged lines hidden		
541	(#isathm("warrowing_properties", thy: "Warrowing")), so an unknown whose value	539	(#isathm("warrowing_properties", thy: "Warrowing")), so an unknown whose value
542	no longer changes satisfies its inequality. The solver's correctness theorem	540	no longer changes satisfies its inequality. The solver's correctness theorem
543	uses only this and holds whether the right-hand sides are monotone	541	uses only this and holds whether the right-hand sides are monotone
544	(@sec:td). Monotonicity is needed for the classical theory of least fixpoints,	542	(@sec:td). Termination needs more. A classical widening is chosen so that repeated widening stabilizes
545	but not to certify that a returned valuation is a post-solution. Termination		
546	needs more. A classical widening is chosen so that repeated widening stabilizes		
547	after finitely many steps @cousot77[§9.1.3] @cousot92plilp[§4], and a classical	543	after finitely many steps @cousot77[§9.1.3] @cousot92plilp[§4], and a classical
548	narrowing must also make repeated narrowing stabilize. The solver's classes	544	narrowing must also make repeated narrowing stabilize. The solver's classes
549	omit these stabilization laws, since termination of the side-effecting TD that	545	omit these stabilization laws, because termination of the side-effecting TD that
550	Voblint uses is not proved,	546	Voblint uses is not proved (@sec:td). Within these laws, the
551	so the laws they state do not make the solver terminate. Within these laws, the		

552	choice of widening and narrowing affects only precision and termination.	547	choice of widening and narrowing affects only precision and termination.
553		548	
554	== Context-sensitive analysis of procedures <sec:interproc>	549	== Context-sensitive analysis of procedures <sec:interproc>
555		550	
556	The counting loop has no procedures. With procedures, the analysis must decide	551	The counting loop has no procedures. With procedures, the analysis must decide
557	which abstract state a callee's body starts from. Take the calls `bump(5)` and	552	which abstract state a callee's body starts from. Take the calls `bump(5)` and
558	`bump(4)` of @ch:intro, where `bump` returns its argument plus one. With one	553	`bump(4)` of @ch:intro, where `bump` returns its argument plus one. With one
559	unknown per program point, the entry of `bump` must cover both arguments, so,	554	unknown per program point, the entry of `bump` must cover both arguments, so,
560	over intervals, its result is at best $absval(ivl(5, 6))$, and the analysis cannot show	555	over intervals, its result is at best $absval(ivl(5, 6))$, and the analysis cannot show
561	that the first call returns 6. A context-sensitive analysis instead	556	that the first call returns 6. A context-sensitive analysis instead
562	separates the _activations_ of a procedure, each execution from its call to	557	separates the activations of a procedure (@sec:state-of-art) by their calling
563	its return, by a _calling context_, an abstraction of how the activation was	558	contexts (@sec:intro-static) @aho06[§7.2.1]. Its unknowns become pairs $[v, c]$ of a program point
564	reached @aho06[§7.2.1]. Its unknowns become pairs $[v, c]$ of a program point		
565	v and a context c @apinis12[§3], so every context of a procedure gets its	559	v and a context c @apinis12[§3], so every context of a procedure gets its
566	own copy of the procedure's unknowns.	560	own copy of the procedure's unknowns.
567		561	
568	Sharir and Pnueli describe two classical choices of what a context records	562	Sharir and Pnueli describe two classical choices of what a context records
569	@sharir81. The call-string approach uses the	563	@sharir81. The call-string approach records calling histories; bounded variants keep
570	most recent call sites as the context. Since `bump(5)` and `bump(4)` are	564	the most recent sites. Since `bump(5)` and `bump(4)` are
571	called from different sites, each gets its own copy. The functional approach	565	called from different sites, each gets its own copy. The functional approach
572	instead summarizes a procedure by its effect on each entry state, computed	566	summarizes a procedure's effect on entry states. Demand tabulation computes
573	only for the entry states that occur. With the entry state as context, `bump`	567	only entries encountered during analysis. With the entry state as context, `bump`
574	gets one copy for $absval(ivl(5, 5))$ and one for $absval(ivl(4, 4))$, with the	568	gets one copy for $absval(ivl(5, 5))$ and one for $absval(ivl(4, 4))$, with the
575	results 6 and 5. Neither choice subsumes the other. Only entry states	569	results 6 and 5. Neither choice subsumes the other. Only entry states
576	separate two calls from one site with different arguments, and only call	570	separate two calls from one site with different arguments, and only call
577	strings separate calls from two sites with the same argument. Apinis et al.	571	strings separate calls from two sites with the same argument. Apinis et al.
578	also distinguish calls by only part of the entry state @apinis12[§4], for	572	also distinguish calls by only part of the entry state @apinis12[§4], for
579	instance by the argument of `bump` alone, ignoring global variables it never	573	instance by the argument of `bump` alone, ignoring global variables it never
580	reads. The	574	reads. The
581	context is therefore a choice of abstraction, and the trivial choice, a single	575	context is therefore a choice of abstraction, and the trivial choice, a single
582	context, gives back the analysis with one unknown per program point.	576	context, gives back the analysis with one unknown per program point.
583		577	
584	A context-sensitive analysis handles a call with two abstract operations @apinis12[§3]. The first,	578	A context-sensitive analysis handles a call with two abstract operations @apinis12[§3]. The first,
585	$"enter"^sharp$, maps the caller's state before the call to the callee's entry	579	$"enter"^sharp$, maps the caller's state before the call to the callee's entry
586	state. The callee's context is chosen at the same point, for example as the	580	state. The callee's context is chosen at the same point, for example as the
587	extended call string or as this entry state itself. The second, $"comb"^sharp$,	581	extended call string or as this entry state itself. The second, $"comb"^sharp$,
588	computes the caller's state after the call. It needs the callee's exit state,	582	computes the caller's state after the call. It needs the callee's exit state for the return value and the caller's state before the call for the caller's local variables, which the callee does not see. Goblint's
589	which holds the return value, and the caller's state before the call, which		
590	holds the caller's local variables that the callee does not see. Goblint's		
591	#link(	583	#link(

592 "https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/framework/constraints.ml#L242-L252",	584 "https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/framework/constraints.ml#L242-L252",
593)[constraint generator]	585)[constraint generator]
594 implements them as `enter` and `combine` (@sec:calls).	586 implements them as `enter` and `combine` (@sec:calls).
595	587
596 == Side-effecting constraint systems <sec:side-effects>	588 == Side-effecting constraint systems <sec:side-effects>
597	589
598 The analyses so far are _flow-sensitive_: they respect the order in which	590 The analyses so far are _flow-sensitive_. They respect the order in which
599 statements execute and therefore compute a separate abstract state for every	591 statements execute and therefore compute a separate abstract state for every
600 program point. For some facts, one value for the whole program is enough. A	592 program point. For some facts, one value for the whole program is enough. A
601 _flow-insensitive_ analysis ignores execution order and keeps one abstract	593 _flow-insensitive_ analysis ignores execution order and keeps one abstract
602 value for an aspect of the state across the program, for example, one range for	594 value for an aspect of the state across the program, for example, one range for
603 a global variable, and mixed analyses choose between the two per aspect	595 a global variable, and mixed analyses choose between the two per aspect
604 @seidl26.	596 @seidl26.
605	597
606 A flow-insensitive fact needs an unknown of its own that belongs to no program	598 A flow-insensitive fact needs an unknown of its own, attached to no program point, and every point that changes the fact has to feed its new value into this unknown. An ordinary right-hand side for the unknown would have to list
607 point, and every point that changes the fact has to feed its new value into	
608 that unknown. An ordinary right-hand side for the unknown would have to list	
609 all these points in advance, which is not always possible. In a	599 all these points in advance, which is not always possible. In a
610 context-sensitive analysis, for example, an unknown that writes a global is a	600 context-sensitive analysis, for example, an unknown that writes a global is a
611 pair of a program point and a calling context, and the contexts are discovered	601 pair of a program point and a calling context, and the contexts are discovered
22 unchanged lines hidden	
634 instance, the entry state of a callee in one context is a global unknown, and	624 instance, the entry state of a callee in one context is a global unknown, and
635 every call that enters this context publishes its entry state to it	625 every call that enters this context publishes its entry state to it
636 (@sec:eq-call). An analysis may also keep facts of its own in global unknowns	626 (@sec:eq-call). An analysis may also keep facts of its own in global unknowns
637 that stand for no program variable at all (@ch:analysis-interface). A	627 that stand for no program variable at all (@ch:analysis-interface).
638 right-hand side then returns a pair @saan26phd[§2.4.5],	628
	629 Formally, a right-hand side returns a pair @saan26phd[§2.4.5],
639 $rhs(x) : (Unk union Unk_G -> A) -> (Unk_G harpoon.rt A) times A$, of a	630 $rhs(x) : (Unk union Unk_G -> A) -> (Unk_G harpoon.rt A) times A$, of a
640 partial map rho of the contributions it publishes, joined per global	631 partial map rho of the contributions it publishes, joined per global
641 unknown, and its own value d. If $rhs(x)(sol) = (rho, d)$, a post-solution	632 unknown, and its own value d. If $rhs(x)(sol) = (rho, d)$, a post-solution
642 must bound both, $d lle sol(x)$ and $rho(g) lle sol(g)$ for every	633 must bound both, $d lle sol(x)$ and $rho(g) lle sol(g)$ for every
643 $g in op("dom")(rho)$. If the assignments	634 $g in op("dom")(rho)$. If the assignments
644 `g = 5` and `g = 4` publish $absval(ivl(5, 5))$ and $absval(ivl(4, 4))$ to the unknown of a	635 `g = 5` and `g = 4` publish $absval(ivl(5, 5))$ and $absval(ivl(4, 4))$ to the unknown of a
645 flow-insensitive global `g`, that unknown must bound their join $absval(ivl(4, 5))$. This	636 flow-insensitive global `g`, that unknown must bound their join $absval(ivl(4, 5))$. This
646 describes what a right-hand side computes. The verified solver represents its	637 describes what a right-hand side computes. The verified solver represents its
647 evaluation by a strategy tree (@sec:td), so that it discovers reads and side	638 evaluation by a strategy tree (@sec:td), so that it discovers reads and side
648 effects while it solves.	639 effects while it solves.
649	640
650 With contexts as indices, a system can have infinitely many unknowns, of which	641 With contexts as indices, a system can have infinitely many unknowns. Only those that influence the _query_, the unknown whose value is asked for, matter. A _local_ solver therefore explores the system from the query and

651	only those that influence the _query_, the unknown whose value is asked for,		
652	matter. A _local_ solver therefore explores the system from the query and		
653	solves only the unknowns that it meets @seidl21. Its result comes with a	642	solves only the unknowns that it meets @seidl21. Its result comes with a
654	_certificate_, the property #isaconst("part_post_solution", thy: "Basics_side"),	643	_certificate_, the property #isaconst("part_post_solution", thy: "Basics_side"),
655	which requires the bounds above only on a set of unknowns that contains the	644	which requires the bounds above only on a set of unknowns that contains the
	12 unchanged lines hidden		
668	reuses the verified local top-down solver (TD) of @stade24 @tilscher26.	657	reuses the verified local top-down solver (TD) of @stade24 @tilscher26.
669		658	
670	TD starts from the query and evaluates right-hand sides on demand. When the	659	TD starts from the query and evaluates right-hand sides on demand. When the
671	right-hand side of x reads an unknown y, TD first solves y and records	660	right-hand side of x reads a local unknown y, TD first solves y and records
672	that x depends on y. An unknown is _stable_ if it has been evaluated and no	661	that x depends on y. TD provisionally marks an unknown _stable_ before
673	unknown that it read has changed since then. When the value of y changes	662	evaluating its right-hand side. A changed dependency removes that mark
674	later, every unknown that read it is _destabilized_ and evaluated again. The	663	(_destabilizes_ the unknown) and triggers reevaluation. The
675	unknowns at which a read closes a cycle are the solver's _widening points_	664	unknowns at which a read closes a cycle are the solver's _widening points_
676	@seidl21. TD detects them while it solves @apinis16[§4], so they can	665	@seidl21. TD detects them while it solves @apinis16[§4], so they can
677	differ from the loop heads of the control-flow graph. At a widening point, it	666	differ from the loop heads of the control-flow graph. At a widening point, it
678	updates the old value with the new one by warrowing. The	667	updates the old value with the new one by warrowing. The
679	#link(playground-link(_counting-loop, trace: "verbose"))[playground] replays this run on the counting loop step by step. Contributions published to a	668	#link(playground-link(_counting-loop, trace: "verbose"))[playground] replays this run on the counting loop step by step. Contributions published to a
680	global unknown are merged into its value by an _update rule_	669	global unknown are merged into its value by an _update rule_
681	(#isalocale("update_rule")), which keeps one record per _origin_, the unknown whose right-hand side	670	(#isalocale("update_rule")). Some rules retain separate contributions per
682	published the contribution (@sec:update-rules). The run ends	671	_origin_, the local unknown publishing them (@sec:update-rules). The run ends
683	once every unknown the query transitively reads is stable, and it returns this	672	once every local unknown the query transitively reads is stable, and it returns this
684	_stable set_ V together with the valuation sol.	673	_stable set_ V together with the valuation sol.
685		674	
686	To solve on demand, TD has to discover which unknowns a right-hand side reads.	675	To solve on demand, TD has to discover which unknowns a right-hand side reads.
	70 unchanged lines hidden		
757	and V is called the _solved set_. Unlike some TD variants without side effects	746	and V is called the _solved set_. Unlike some TD variants without side effects
758	@tilscher26jar, this solver has no termination proof (@sec:isabelle explains	747	@tilscher26jar, this solver has no termination proof (@sec:isabelle explains
759	how Isabelle defines such a function).	748	how Isabelle defines such a function).

thesis/content/03-program-model.typ

77e07947 → 0edeac81

894 old lines, 892 new lines

92% similar lines **+73** **-75** **35 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node	1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node
2	#import "@preview/curryst:0.6.0": prooftree, rule	2	#import "@preview/curryst:0.6.0": prooftree, rule
3	#import "@preview/cetz:0.5.2"	3	#import "@preview/cetz:0.5.2"
4	#import "../lib/math.typ": *	4	#import "../lib/math.typ": *
5	#import "../lib/theme.typ": vb	5	#import "../lib/theme.typ": vb
6	#import "../lib/figures.typ": *	6	#import "../lib/figures.typ": *
7	#import "../lib/code.typ": *	7	#import "../lib/code.typ": *

Old	77e07947	New	0edeac81
8	#import "../lib/sources.typ": proved, stated, thy	8	#import "../lib/sources.typ": proved, stated, thy
9	#import "../lib/theorems.typ": corollary, definition, example, lemma, theorem	9	#import "../lib/theorems.typ": corollary, definition, example, lemma, theorem
10	#import "../lib/cfg-graphs.typ": dot-edge-line, dot-label, dot-node-line, sum-graph	10	#import "../lib/cfg-graphs.typ": dot-edge-line, dot-label, dot-node-line, sum-graph
11		11	
12	= VIMP and Its Control-Flow Graphs <ch:program-model>	12	= VIMP and Its Control-Flow Graphs <ch:program-model>
13		13	
14	@ch:background analyzed a single loop through its control-flow graph. The	14	Voblint's soundness theorem speaks about executions of a VIMP program, while
15	soundness theorem, however, should talk about the programs a user writes, and	15	its analysis computes on a control-flow graph. This chapter connects the two.
16	these have procedures and recursion. Such a program has a meaning of its own,	16	Its result is the first inclusion of the chain of @fig:intro-nest. Every
17	given by a source semantics that executes the program text directly	17	source execution of a well-formed program (@sec:vimp) is matched by a graph execution that
18	(@sec:pstep). To generate its equations, however, an analysis needs a fixed,	18	holds the same store (@sec:csim).
		19	
		20	@ch:background analyzed a single loop through its control-flow graph. VIMP
		21	programs also have procedures and recursion, and their meaning is given by a
		22	source semantics that executes the program text directly (@sec:pstep). To
		23	generate its equations, however, an analysis needs a fixed,
19	finite set of program points, because its constraint system attaches one	24	finite set of program points, because its constraint system attaches one
20	abstract state to each of them. The source semantics provides no such set. It	25	abstract state to each of them. The source semantics provides no such set. It
21	locates an execution only by the command that still has to run, and under	26	locates an execution only by the command that still has to run, and under
22	recursion these commands nest arbitrarily deep (@sec:why-graph). Voblint therefore compiles a	27	recursion these commands nest arbitrarily deep (@sec:why-graph). Voblint therefore compiles a
23	program to a procedure-aware control-flow graph, whose nodes serve as the	28	program to a procedure-aware control-flow graph, whose nodes serve as the
24	program points, and generates its equations from the graph	29	program points, and generates its equations from the graph
25	(@fig:program-to-equations). The compilation step must then be part of the	30	(@fig:program-to-equations). The compilation step must then be part of the
26	proof. If soundness held only for the graph, the compiler would stay outside	31	proof. If soundness held only for the graph, the compiler would stay outside
27	the theorem, and a verdict would say nothing about the source program. This	32	the theorem, and a verdict would say nothing about the source program. The running example is the program of
28	chapter closes the gap with the first link of the soundness chain. Every		
29	source execution of a well-formed program is matched by a graph execution that holds the		
30	same store (@sec:csim). The running example is the program of		
31	@fig:program-to-equations, in which `main` calls `bump(5)` and `bump(4)`.	33	@fig:program-to-equations, in which `main` calls `bump(5)` and `bump(4)`.
32		34	
33	// Node names and edge labels come from registered `--dot` output	35	// Node names and edge labels come from registered `--dot` output
	114 unchanged lines hidden		
148		150	
149	== VIMP syntax and well-formed programs <sec:vimp>	151	== VIMP syntax and well-formed programs <sec:vimp>
150		152	
151	VIMP (@fig:vimp-island) is a small imperative language with C-like syntax	153	VIMP (@fig:vimp-island) is kept small because every construct needs a
152	that the formalization analyzes. It is kept small because every construct needs a		
153	semantics, compiler clauses, and a transfer function proved sound in every	154	semantics, compiler clauses, and a transfer function proved sound in every
154	abstract domain. It has unbounded integer variables, locals and globals,	155	abstract domain. It has unbounded integer variables, locals and globals,
155	procedures with value parameters, direct calls with an optional result,	156	procedures with value parameters, direct calls with an optional result,
156	recursion, assignment, sequencing, conditionals, `while` loops,	157	recursion, assignment, sequencing, conditionals, `while` loops,

157	nondeterministic input, and checks. It omits machine integers, pointers and the	158	nondeterministic input, and checks. It omits machine integers, pointers and the
158	heap, arrays and structs, indirect calls, threads, floating point, `goto`,	159	heap, arrays and structs, indirect calls, threads, floating point, `goto`,
159	`break`, `continue`, and exceptions. Each of these would need its own	160	`break`, `continue`, and exceptions (@ch:conclusion discusses the heap).
160	preservation argument at every layer (@ch:conclusion discusses the heap).		
161	@sec:vimp-vs-c compares the fragment with C and with Goblint's input.	161	@sec:vimp-vs-c compares the fragment with C and with Goblint's input.
162		162	
163	#figure(	163	#figure(
164	image("/shared/generated/svg/island.svg", width: 75%),	164	image("/shared/generated/svg/island.svg", width: 75%),
165	kind: image,	165	kind: image,
166	placement: auto,	166	placement: auto,
167	caption: [Constructs represented in VIMP relative to Goblint's C input,	167	caption: [Constructs represented in VIMP relative to Goblint's C input,
168	taken from the explainer page. Dashed regions are constructs that VIMP	168	taken from the interactive explainer page of the project website. Dashed regions are constructs that VIMP
169	omits. Sizes and positions are schematic. The figure compares only syntax.	169	omits. Sizes and positions are schematic. The figure compares only syntax.
170	@sec:vimp-vs-c lists where shared constructs have different meanings.],	170	@sec:vimp-vs-c lists where shared constructs have different meanings.],
171	) <fig:vimp-island>	171	) <fig:vimp-island>

19 unchanged lines hidden

191	]	191	]
192		192	
193	Each operator has its own constructor, because an abstract domain chooses its	193	Each operator has its own constructor, because an abstract domain chooses its
194	transformer by the operator, and the executable analyzer has to make this	194	abstract operation by the operator, and the executable analyzer has to make
195	choice by inspecting the syntax. The two AFP languages with procedures that were	195	this choice by inspecting the syntax. We considered two languages with
196	considered, IMP2 and Simpl, do not allow this. IMP2 @lammich19imp2 represents a binary operator by a HOL	196	procedures from Isabelle's Archive of Formal Proofs (AFP), IMP2 and Simpl.
		197	Both represent operators semantically instead. IMP2 @lammich19imp2 represents a binary operator by a HOL
197	function of type #isai("int \<Rightarrow> int \<Rightarrow> int"). Concrete	198	function of type #isai("int \<Rightarrow> int \<Rightarrow> int"). Concrete
198	evaluation can apply such a function, but executable code cannot compare it	199	evaluation can apply such a function, but executable code cannot compare it
199	with another one, so it cannot tell `+` from `*`. IMP2 also matches Voblint's	200	with another one, so it cannot tell `+` from `*`. IMP2's semantics is also deterministic, so it
200	programs poorly in other respects. Its semantics is deterministic, so it has	201	has no input, and its call command takes no arguments and returns no value.
201	no input, and its call command takes no arguments and returns no value, so	202	Parameters and results pass through variables set by generated code
202	parameters and results pass through variables set by generated code		
203	(@ch:related). Schirmer's Simpl @schirmer08simpl is more expressive, with	203	(@ch:related). Schirmer's Simpl @schirmer08simpl is more expressive, with
204	nondeterministic specifications and a derived call command that passes	204	nondeterministic specifications and a derived call command that passes
205	parameters, but its basic commands and conditions are likewise HOL functions	205	parameters, but its basic commands and conditions are likewise HOL functions
206	and sets of states. Both languages target program verification, where semantic objects are	206	and sets of states. Both languages target program verification, where such
207	convenient, whereas an analyzer needs syntax that it can inspect. VIMP is therefore defined from scratch, with such a syntax, which code	207	semantic objects are convenient. VIMP is therefore defined from scratch, with
208	generation can also export.	208	a syntax that the analyzer can inspect and that code generation can export.
209		209	
210	An expression is evaluated in a _store_ s of type #isatype("store"), a total	210	An expression is evaluated in a _store_ s of type #isatype("store"), a total
211	function #isai("vname \<Rightarrow> int") from variable names to integers.	211	function #isai("vname \<Rightarrow> int") from variable names to integers.

9 unchanged lines hidden

221	remainder by zero (#isaconst("c_div"), #isaconst("c_mod"), @sec:vimp-vs-c). It also has no	221	remainder by zero (#isaconst("c_div"), #isaconst("c_mod"), @sec:vimp-vs-c). It also has no
222	effects, so a transfer function may evaluate a guard again, as the backward	222	effects, so a transfer function may evaluate a guard again, as the backward
223	filtering of @ch:domains does. Input enters only through the statement	223	filtering of @ch:domains does. Input enters only through the statement
224	`x = __voblint_nondet_int();` and never inside an expression. This keeps	224	`x = __voblint_nondet_int();` and never inside an expression. Restricting input to this statement keeps #isaconst("aval") a function.
225	#isaconst("aval") a function.		
226		225	
227	Commands are built from expressions. Besides the structured statements, there	226	Commands are built from expressions. Besides the structured statements, there
228	are calls, which may store a result, and two markers that appear only during	227	are calls, which may store a result, and two markers that appear only during
	15 unchanged lines hidden		
244		243	
245	In concrete syntax, an assignment is written `x = e;` and a check	244	In concrete syntax, an assignment is written `x = e;` and a check
246	`__voblint_check(e);`. The parser lowers `true`, `false`, and unary minus to the	245	`__voblint_check(e);`. The parser lowers `true`, `false`, and unary minus to the
247	expression grammar above. A check states a condition for the analyzer to	246	expression grammar above. As in @sec:intro-static, a check states a condition
248	decide (@sec:verdicts) and does not influence the execution. It behaves like	247	for the analyzer to decide (@sec:verdicts). It does not influence the
		248	execution. It behaves like
249	#skipC whatever value e has, so it neither stops the execution nor refines the	249	#skipC whatever value e has, so it neither stops the execution nor refines the
250	store. Its constructor also carries the source position, which only the report	250	store. Its constructor also carries the source position, which only the report
251	uses, to name the check.	251	uses, to name the check.
252		252	
253	Calls come in two kinds. A call of a declared procedure starts an	253	Calls come in two kinds. A call of a declared procedure starts an activation
254	_activation_, one execution of the procedure body from its call to its return.	254	of that procedure (@sec:interproc).
255	VIMP also has three built-in _library calls_, `__voblint_nondet_int`, `min`, and `max`, which a program calls but does not declare. The fixed table	255	VIMP also has three built-in _library calls_, `__voblint_nondet_int`, `min`, and `max`, which a program calls but does not declare. The fixed table
256	#isaconst("special_table") recognizes their names, and	256	#isaconst("special_table") recognizes their names, and
257	#isaconst("special_result") gives the values they may return: any integer for	257	#isaconst("special_result") gives the values they may return: any integer for
	72 unchanged lines hidden		
330	) <fig:vimp-ast>	330	) <fig:vimp-ast>
331		331	
332	Not every parsed program can run meaningfully. The theorems therefore	332	Not every parsed program can run meaningfully. The theorems therefore
333	assume the _source contract_ #isaconst("wf_source_program"), which excludes	333	assume the _source contract_ #isaconst("wf_source_program"), which excludes programs in which a call cannot step or a result is undefined. It requires
334	programs whose calls cannot step or whose results are undefined. It requires		
335	that	334	that
336		335	
337	- every call names either a declared procedure with matching arity or a	336	- every call names either a declared procedure with matching arity or a
	65 unchanged lines hidden		
403	configurations $kappa$, $kappa'$, and a procedure table #isai("\<Pi>"). Its	402	configurations $kappa$, $kappa'$, and a procedure table #isai("\<Pi>"). Its
404	closure is #isai("\<rightarrow>\<^sub>p\<^sup>*"), and its rules are	403	closure is #isai("\<rightarrow>\<^sub>p\<^sup>*"), and its rules are
405	listed below. No rule inspects an annotation. The rules only copy annotations, from	404	listed below. No rule inspects an annotation. The rules only copy annotations, from
406	the rewritten command or from the procedure table. The	405	the rewritten command or from the procedure table. The interactive
407	#link(site("index.html#semantics"))[explainer page]	406	#link(site("index.html#semantics"))[explainer page] of the project website
408	animates these steps.	407	animates these steps.
409		408	

410	A call evaluates the actuals in the caller's store, resets the locals, binds	409	A call evaluates the actuals in the caller's store, resets the locals, binds
	25 unchanged lines hidden		
436		435	
437	== The procedure-aware control-flow graph <sec:graph>	436	== The procedure-aware control-flow graph <sec:graph>
438		437	
439	The source semantics executes a program by rewriting the command that remains	438	The analysis attaches an abstract state to each program point and a transfer
440	to run. The analysis instead works on a procedure-aware control-flow graph,	439	function to each transition between points. Voblint takes both from a
441	whose nodes are the program points that carry abstract states and whose local	440	procedure-aware control-flow graph, whose nodes are the program points and
442	edges and call edges carry the transfers.	441	whose local edges and call edges carry the transfers. The source semantics
		442	cannot play this role (@sec:why-graph).
443		443	
444	=== Why the analysis needs program points <sec:why-graph>	444	=== Program points for the analysis <sec:why-graph>
445		445	
446	An analysis attaches abstract states to program points and transfer functions	446	The source semantics #isaconst("pstep") offers neither program points nor
447	to transitions. #isaconst("pstep") offers neither in a usable form. Its only	447	transitions in a usable form. Its only
448	notion of location is the command that remains to run. Without recursion, only finitely many distinct residual commands arise,	448	notion of location is the command that remains to run. Without recursion, only finitely many distinct residual commands arise,
449	because a loop may run arbitrarily often, but its unfolding revisits the same	449	because a loop may run arbitrarily often, but its unfolding revisits the same
450	command forms. Recursion is different, because a recursive call can nest	450	command forms. Recursion is different, because a recursive call can nest
451	another #ctor("Restore") marker at every call depth. Residual commands	451	another #ctor("Restore") marker at every call depth. Residual commands
452	therefore do not form the finite, fixed set of locations that Voblint needs. An assignment is also not a transition between fixed program points. Inside a sequence, it steps through the rule	452	therefore do not form the finite, fixed set of locations that Voblint needs. An assignment is also not a transition between fixed program points. Inside a sequence, it steps through the rule
453	#isathm("pstep.Seq2"), which rebuilds the surrounding residual command. So the same assignment is executed by a different derivation inside every surrounding command.	453	#isathm("pstep.Seq2"), which rebuilds the surrounding residual command. So the same assignment is executed by a different derivation inside every surrounding command.
454	Finally, nothing names the entry of a procedure, its result, or the point where	454	Finally, nothing names the entry of a procedure, its result, or the point where
455	a caller resumes. The schematic control-flow graph of @fig:counting-loop solves	455	a caller resumes.
		456	
		457	The schematic control-flow graph of @fig:counting-loop solves
456	the first two problems. It has finitely many nodes and one transfer per edge.	458	the first two problems. It has finitely many nodes and one transfer per edge.
457	The procedure-aware graph then solves the third. It adds an entry node and a result	459	The procedure-aware graph then solves the third. It adds an entry node and a result
458	node per procedure, and call edges that record their continuations. Each gives	460	node per procedure, and call edges that record their continuations. Each gives
	10 unchanged lines hidden		
469	inequalities over unknowns indexed by the graph's nodes. A generic solver	471	inequalities over unknowns indexed by the graph's nodes. A generic solver
470	that knows nothing about programs then solves this system. The solver	472	that knows nothing about programs then solves this system. The solver
471	needs no knowledge of calls, because a call contributes right-hand sides and	473	needs no knowledge of calls, because a call contributes right-hand sides and
472	side effects to the same system (@ch:equations). The price is two extra	474	side effects to the same system (@ch:equations). The approach requires two extra proof layers. This chapter must prove by a simulation that the compiler
473	proof layers. This chapter must prove by a simulation that the compiler		
474	preserves the program's executions, and @ch:traces must say what the inequalities	475	preserves the program's executions, and @ch:traces must say what the inequalities
475	mean for those executions. In return, the argument about programs	476	mean for those executions. In return, the argument about programs
476	ends at a solver-independent certificate, a post-solution on the solved set	477	ends at a solver-independent certificate, a post-solution on the solved set
	37 unchanged lines hidden		
514		515	

515	Returns, in contrast, are ordinary local edges. A #keyw("return") in p leads	516	Returns, in contrast, are ordinary local edges. A #keyw("return") in p leads
516	to the result node $ctor("FunctionResult") thin p$, which all callers of p	517	to the result node $ctor("FunctionResult") thin p$, which all callers of p
517	share, so recursion needs no copies of nodes. A shared result node cannot know	518	share, so recursion needs no copies of nodes. A shared result node does not determine where to continue, and in a compiled graph, no edge leaves it. The call edge
518	where to continue, and in a compiled graph, no edge leaves it. The call edge		
519	records that place instead, as its continuation k. After a return, execution	519	records that place instead, as its continuation k. After a return, execution
520	resumes at the continuation saved in the frame, and an analysis connects the	520	resumes at the continuation saved in the frame, and an analysis connects the
521	callee's exit state to the correct caller through the call edge. The matching	521	callee's exit state to the correct caller through the call edge. The matching
522	caller is explicit in the graph, so no separate return-to-call matching is	522	caller is explicit in the graph, so no separate return-to-call matching is
523	needed. A completed program ends at	523	needed. A completed program ends at
524	$ctor("FunctionResult") thin #raw("main")$ (#isaconst("cfg_exit")), so there	524	$ctor("FunctionResult") thin #raw("main")$ (#isaconst("cfg_exit")), so there
525	is no separate global exit node.	525	is no separate global exit node.
526		526	
527	== Compiling VIMP to a graph <sec:compilation>	527	== Compilation to a control-flow graph <sec:compilation>
528		528	
529	The compiler turns a VIMP program into a procedure-aware graph. Its design	529	The compiler turns a VIMP program into a procedure-aware graph. Its design
530	decides which program points exist, and the simulation of @sec:csim relies on	530	decides which program points exist, and the simulation of @sec:csim relies on
531	how it lays out the code of each command.	531	how it lays out the code of each command.
532		532	
533	=== Compiling commands with continuations #thy-badge("Voblint_Compile", "VIMP_Proc_to_CFG") <sec:compile>	533	=== Compilation with continuations #thy-badge("Voblint_Compile", "VIMP_Proc_to_CFG") <sec:compile>
534		534	
535	To emit the edges of a command, a compiler has to know where control goes	535	To emit the edges of a command, a compiler has to know where control goes
536	after it. Compilers commonly pass this target down, so that each statement	536	after it. Compilers commonly pass this target down, so that each statement
	114 unchanged lines hidden		
651	#isaconst("wf_source_program") that $italic("ps")$ lists every declared	651	#isaconst("wf_source_program") that $italic("ps")$ lists every declared
652	procedure except `main` exactly once.	652	procedure except `main` exactly once.
653		653	
654	=== Executing a graph with a frame stack #thy-badge("Voblint_CFG", "CFG_Exec") <sec:cstep>	654	=== Graph execution with a frame stack #thy-badge("Voblint_CFG", "CFG_Exec") <sec:cstep>
655		655	
656	The compiler produces a static object. To say that it preserves the behavior	656	The compiler produces a static object. To say that it preserves the behavior
657	of a program, the graph needs an execution of its own, which @sec:csim then	657	of a program, the graph needs an execution of its own, which @sec:csim then
	34 unchanged lines hidden		
692	@ch:traces, #isaconst("cstep") is defined for an arbitrary graph, not only for	692	@ch:traces, #isaconst("cstep") is defined for an arbitrary graph, not only for
693	compiled ones.	693	compiled ones.
694		694	
695	== Compilation preserves source executions <sec:csim>	695	== Simulation of source executions <sec:csim>
696		696	
697	So far, #isaconst("pstep") and #isaconst("cstep") are two separate	697	So far, #isaconst("pstep") and #isaconst("cstep") are two separate
698	definitions. One describes source programs, the other arbitrary graphs, and	698	definitions. One describes source programs, the other arbitrary graphs, and
699	nothing yet says that a compiled graph behaves like its program. The soundness	699	nothing yet says that a compiled graph behaves like its program. The soundness
700	theorem is stated over #isaconst("pstep"), but the analysis solves equations	700	theorem is stated over #isaconst("pstep"), but the analysis solves equations
701	generated from the graph, so every source execution must be matched by an execution of the	701	generated from the graph, so every source execution must be matched by an execution of the

702	compiled graph that holds the same store at corresponding points. This is proved with a _forward simulation_, a	702	compiled graph that holds the same store at corresponding points. The matching is proved with a _forward simulation_, a
703	relation between source and graph configurations that every source step	703	relation between source and graph configurations that every source step
704	preserves.	704	preserves.
705		705	
	69 unchanged lines hidden		
775	The initial source configuration is related to the node after the	775	The initial source configuration is related to the node after the
776	#isaconst("EA_Body") edge of `main` (#isathm("compile_prog_main_base")).	776	#isaconst("EA_Body") edge of `main` (#isathm("compile_prog_main_base")).
777	Every finite source execution of a well-formed program is therefore matched by a	777	Every finite source execution of a well-formed program is therefore matched by a
778	graph execution from the entry that reaches corresponding nodes with the same store. This gives the first inclusion of the soundness chain of	778	graph execution from the entry that reaches corresponding nodes with the same store. This matching gives the first inclusion of the soundness chain of
779	@fig:intro-nest.	779	@fig:intro-nest.
780		780	
781	=== Locating source configurations at graph nodes #thy-badge("Voblint_Compile", "Elaborated_Compilation") <sec:at-node>	781	=== Source configurations at graph nodes #thy-badge("Voblint_Compile", "Elaborated_Compilation") <sec:at-node>
782		782	
783	The simulation names no node for a source configuration taken on its own.	783	The simulation names no node for a source configuration taken on its own.
784	@ch:results needs one, because the analyzer's verdicts are stated at graph	784	@ch:results needs one, because the analyzer's verdicts are stated at graph
785	nodes, while a source configuration carries only its residual command. The	785	nodes, while a source configuration carries only its residual command. The
786	residual does not determine the node either, because equal commands can occur	786	residual does not determine the node either, because equal commands can occur
787	at different places. In `if (b) { x = 1; } else { x = 1; }`, the residual	787	at different places. In `if (b) { x = 1; } else { x = 1; }`, the residual
788	`x = 1` is related to both branch nodes. The node must come from	788	`x = 1` is related to both branch nodes. The node must come from
789	the run.	789	the run.
790		790	
791	Every command therefore carries an annotation (@sec:vimp). A parsed program	791	Every command therefore carries an annotation (@sec:vimp). A parsed program
792	leaves it empty, and the proof fills it in. The program's _elaboration_	792	leaves it empty, and the proof fills it in. The program's _annotated copy_
793	(#isaconst("elab_pt")) annotates each statement with the nodes the compiler	793	(#isaconst("elab_pt")) annotates each statement with the nodes the compiler
794	emits for it. The elaboration compiles to the same graph	794	emits for it. The annotated copy compiles to the same graph
795	(#isathm("compile_prog_elab_pt")), and on it,	795	(#isathm("compile_prog_elab_pt")), and on it,
796	#isaconst("point", thy: "Elaborated_Compilation") reads the related node off	796	#isaconst("point", thy: "Elaborated_Compilation") takes the related node from
797	the residual's annotation (#isathm("csim_point_compile_prog")). Every source execution of a	797	the residual's annotation (#isathm("csim_point_compile_prog")). Every source execution of a
798	well-formed program is the _erasure_ (#isaconst("erase")) of a run of its	798	well-formed program is the _erasure_ (#isaconst("erase")) of a run of its
799	elaboration (#isathm("psteps_lift")), so the node can be read off the elaborated	799	annotated copy (#isathm("psteps_lift")), so the annotated run determines the node.
800	run.		
801		800	
802	#definition(name: [Observed at a node], isa: "at_node", cmd: "fun")[	801	#definition(name: [Observed at a node], isa: "at_node", cmd: "fun")[
803	A configuration $(r, s, "frs")$ is _observed_ at v, written	802	A configuration $(r, s, "frs")$ is _observed_ at v, written
804	$p, s_0 tack.r (r, s, "frs") gt.closed v$, if a run of the elaboration of	803	$p, s_0 tack.r (r, s, "frs") gt.closed v$, if a run of the annotated copy of
805	p from s_0 reaches an annotated configuration that erases to it, names	804	p from s_0 reaches an annotated configuration that erases to it, names
806	v, and is related to v by the simulation. A configuration reachable	805	v, and is related to v by the simulation. A configuration reachable
807	along two paths can be observed at two nodes.	806	along two paths can be observed at two nodes.
808	]	807	]
809		808	
810	== Adequacy of VIMP relative to C <sec:vimp-vs-c>	809	== Comparison of VIMP with C <sec:vimp-vs-c>

811		810	
812	Every source-level guarantee in this thesis is interpreted against the	811	Every source-level guarantee in this thesis is stated over the executions of
813	executions of #isaconst("pstep"), the source semantics of VIMP, so no	812	#isaconst("pstep"). Whether #isaconst("pstep") describes the intended language,
814	guarantee can show that #isaconst("pstep") describes the intended language.	813	its adequacy, therefore lies outside the proof (@ch:intro). This section
815	This can only be argued, and this section collects the argument.	814	argues adequacy by comparing VIMP with C11 @iso-c11 (committee draft N1570).
816	@tab:vimp-vs-c compares VIMP with C11 @iso-c11 (committee draft N1570)	815	@tab:vimp-vs-c lists the constructs that both languages have but model
817	wherever both languages have a construct but model it differently. The comparison	816	differently. The comparison
818	is not complete, so C may differ from VIMP in further places.	817	is not complete, so C may differ from VIMP in further places.
819		818	
820	#figure(	819	#figure(
	16 unchanged lines hidden		
837	[`&&`, `||`], [both operands evaluated],	836	[`&&`, `||`], [both operands evaluated],
838	[second operand only if needed (#c11("6.5.13p4"), #c11("6.5.14p4"))],	837	[second operand only if needed (#c11("6.5.13p4"), #c11("6.5.14p4"))],
839	[initial values], [globals 0, `main`'s locals arbitrary],	838	[initial values], [globals 0, `main`'s locals arbitrary],
840	[static 0, automatic indeterminate (#c11("6.7.9p10"))],	839	[uninitialized: static 0, automatic indeterminate (#c11("6.7.9p10"))],
841	[callee locals], [0 on every entry], [indeterminate (#c11("6.7.9p10"))],	840	[callee locals],
		841	[0 on every entry],
		842	[uninitialized automatic: indeterminate (#c11("6.7.9p10"))],
842	[checks], [never stop the execution, never refine],	843	[checks], [never stop the execution, never refine],
843	[`assert` aborts on 0 unless `NDEBUG` (#c11("7.2p1"), #c11("7.2.1.1"))],	844	[`assert` aborts on 0 unless `NDEBUG` (#c11("7.2p1"), #c11("7.2.1.1"))],
844	[`min`, `max`], [library calls (#isaconst("special_table"))], [no counterpart],	845	[`min`, `max`], [library calls (#isaconst("special_table"))], [no counterpart],
	40 unchanged lines hidden		
885	@leroy09. Relating VIMP to them would need a translation with its own adequacy	886	@leroy09. Relating VIMP to them would need a translation with its own adequacy
886	argument.	887	argument.
887		888	
888	These differences limit what a verdict says about a C program, but the	889	VIMP supplies the features needed to study this analyzer architecture:
889	analysis of VIMP still answers the question this thesis asks. The subject of	890	procedures, recursion, locals, globals, and calls with results. A C front end
890	the formalization is the analyzer architecture, and VIMP has the features the	891	could reuse the framework through a verified translation, with a preservation
891	architecture has to handle: procedures, recursion, locals and globals, and	892	argument for each additional feature.
892	calls with results. Since VIMP also has the shape of Goblint's normalized		
893	input, a C front end with a verified translation could reuse the framework,		
894	although each omitted feature would need its own preservation argument.		

thesis/content/04-traces.typ

77e07947 → 0edeac81

613 old lines, 594 new lines

84% similar lines **+85** **-104** **28 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node	1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node
2	#import "../lib/math.typ": *	2	#import "../lib/math.typ": *
3	#import "../lib/theme.typ": vb	3	#import "../lib/theme.typ": vb
	9 unchanged lines hidden		
13	For an analysis that computes one abstract state per node, this suffices. A	13	For an analysis that computes one abstract state per node, this suffices. A
14	context-sensitive analysis, however, computes a separate state for each node and	14	context-sensitive analysis, however, computes a separate state for each node and

```
15  calling context, so justifying its result requires knowing which executions each
16  context stands for. This chapter gives that distinction a concrete meaning. It
17  groups executions into _activation traces_, reads calling contexts off them,
18  and states the _coverage contract_, the local conditions under which an
19  analysis result contains every store that an execution brings to a node in a
20  context.
21
22  == Why calling contexts need activation traces <sec:why-traces>
23
24  A context-sensitive analysis splits the unknown $[v]$ of a node into unknowns
25  $[v, c]$, one per calling context $c$ (@sec:interproc). Seidl et al. state that the invariant
    for
                        19 unchanged lines hidden
45  node-store pairs $(v, s)$ that a run passes through (@fig:flat-nested, top)
46  interleaves the steps of all activations. In the program of @fig:cfgmap,
47  `sum(2)` calls `dec` and then itself, so one run visits the nodes of `sum` in
48  three activations, and the list does not say which call created each of them
49  or which return resumes which caller.
50
51  Grouping the same run by activation, in contrast, makes these relationships
52  explicit (@fig:flat-nested, bottom). Each call of a procedure starts a new
53  _activation_, which lasts until the call returns @aho06[§7.2.1], and each
54  activation has its own activation path. A call creates a new activation attached to its
    caller, and a return
55  composes the finished callee back into that caller.
56
57  #let _acts = (vb.neutral, vb.accent, vb.sign, vb.par, vb.cong, vb.trusted)
58  // Colour alone does not survive greyscale printing: every box names its
                       106 unchanged lines hidden
165     written out by hand, and stores are omitted.],
166 ) <fig:flat-nested>
167
168 == From graph executions to activation traces <sec:traces>
169
170 Activation traces adapt the _local traces_ of threads @schwarz21 to procedure activations.
171
172 === Grouping a run by activation #thy-badge("Voblint_CFG", "Activation_Trace_Def")
    <sec:activation-trace>
173
174 The grouping of @fig:flat-nested becomes a datatype with one constructor per
```

```
15  calling context, so justifying its result requires knowing which executions each
16  context stands for. This chapter defines this correspondence. It
17  groups executions into _activation traces_ and assigns calling contexts to
18  them. It then states the _coverage contract_, the local conditions under which
19  an analysis result contains every store that an execution brings to a node in a
20  context.
21
22  == Calling contexts and the collecting semantics <sec:why-traces>
23
24  A context-sensitive analysis splits the unknown $[v]$ of a node into unknowns
25  $[v, c]$, one per calling context $c$ (@sec:interproc). Seidl et al. state that the invariant
    for
                        19 unchanged lines hidden
45  node-store pairs $(v, s)$ that a run passes through (@fig:flat-nested, top)
46  interleaves the steps of all activations. In the program of @fig:cfgmap,
47  `sum(2)` calls `dec` and then itself, so one run visits the nodes of `sum` in
48  three activations, and the list records neither the call that created each activation nor the
    caller that each return resumes.
49
50
51  Grouping the same run by activation, in contrast, makes these relationships
52  explicit (@fig:flat-nested, bottom). Each box holds one activation
53  (@sec:vimp) and lists the node-store pairs that this activation passes
    through, its _activation path_. A call creates a new activation attached to
54  its caller, and a return composes the finished callee back into that caller.
55
56  #let _acts = (vb.neutral, vb.accent, vb.sign, vb.par, vb.cong, vb.trusted)
57  // Colour alone does not survive greyscale printing: every box names its
                       106 unchanged lines hidden
164     written out by hand, and stores are omitted.],
165 ) <fig:flat-nested>
166
167 == Activation traces of graph executions <sec:traces>
168
169 This section turns the grouping of @fig:flat-nested into a set of traces and
170 shows that every source execution yields one. The construction adapts _local
171 traces_, which describe the view of one thread on a concurrent execution
172 @schwarz21, from threads to procedure activations (@sec:activation-trace).
173
174 === Activations of a run #thy-badge("Voblint_CFG", "Activation_Trace_Def") <sec:activation-
    trace>
175
176 The grouping of @fig:flat-nested becomes a datatype with one constructor per
```

175	way a box arises. The outermost box, the activation of `main`, is a root. A box	177	way a box arises. The outermost box, the activation of `main`, is a root. A box
	31 unchanged lines hidden		
207		209	
208	Each trace represents the history of one activation up to a program point, so	210	Each trace represents the history of one activation up to a program point, so
209	on well-formed compiled programs, its activation path stays inside the nodes of	211	on well-formed compiled programs, its activation path stays inside the nodes of
210	one procedure (#isathm("valid_activation_trace_frag_callers")). A callee is a	212	one procedure (#isathm("valid_activation_trace_frag_callers")). Calls and
211	new activation and gets a trace of its own, which holds its caller as a field.	213	returns start new traces instead of extending the current one, so a whole run
212	When the callee finishes, a $ctor("Resume")$ creates a new trace for the	214	is a family of traces linked by their caller and callee fields.
213	resumed caller, which holds both the frozen caller and the finished callee.		
214	A whole run is therefore a family of traces linked by these fields.		
215		215	
216	Because a $ctor("Call", thy: "Activation_Trace_Def")$ stores its caller	216	Because a $ctor("Call", thy: "Activation_Trace_Def")$ stores its caller
217	exactly, frozen at the call site, a finished callee composes back into the	217	exactly, frozen at the call site, a finished callee composes back into the
218	activation that created it, and no search for a compatible stack frame is	218	activation that created it, and no search for a compatible stack frame is
219	needed. This is what makes context-specific return flow sound. A return	219	needed. Composing a callee back into its creator makes context-specific return flow sound. A return
220	continues exactly the caller that created the callee, in each context that	220	continues exactly the caller that created the callee, in each context of this caller, so the exit state of a callee entered in context c' flows back only to
221	caller carries, so the exit state of a callee entered in context c' flows back only to		
222	the callers whose calls admitted c'. The abstract state for a context therefore needs to	221	the callers whose calls admitted c'. The abstract state for a context therefore needs to
223	describe only the executions in that context.	222	describe only the executions in that context.
224		223	
225	This design follows the local traces of Schwarz et al. @schwarz21[§3]. A	224	This design follows the local traces of Schwarz et al. @schwarz21[§3]. A
226	local trace is the view of one thread on a concurrent execution and ends at a	225	local trace is the view of one thread on a concurrent execution and ends at a
227	program point with that thread's local state. The set of local traces is	226	program point with that thread's local state. The set of local traces is
228	defined inductively, from initial traces, a rule that starts a created thread	227	defined inductively, from initial traces, a rule that starts a created thread
229	with the local state of its creator, and one rule per edge that extends a	228	with the local state of its creator, and one rule per edge that extends a
230	trace by a step. For a lock, this rule also incorporates a second trace, one	229	trace by a step. A lock also incorporates an initial trace or one ending
231	that ends in an unlock of the same mutex. Activation traces take over this	230	in an unlock of the same mutex. Activation traces take over this
232	scheme with an activation in place of a thread. $ctor("Root")$ corresponds to	231	scheme with an activation in place of a thread. $ctor("Root")$ corresponds to
233	an initial trace, $ctor("Call", thy: "Activation_Trace_Def")$ to thread	232	an initial trace, $ctor("Call", thy: "Activation_Trace_Def")$ to thread
234	creation, a local step to a unary edge rule, and $ctor("Resume")$ to the	233	creation, a local step to a unary edge rule, and $ctor("Resume")$ to the
235	binary lock rule, which here incorporates the finished callee. The traces	234	binary lock rule, which here incorporates the finished callee. The traces
236	themselves differ. A local trace is an acyclic graph of events from several	235	themselves differ. A local trace is an acyclic graph of events from several
237	threads, ordered by program order and by the creation and locking orders, and	236	threads, ordered by program order and by the creation and locking orders, and
238	a lock may incorporate a trace of whichever thread last released the mutex. An	237	a lock may incorporate a trace of whichever thread last released the mutex. An
239	activation trace holds the path of a single activation and refers to at most	238	activation trace holds the path of a single activation and refers to at most
240	two other traces, and a $ctor("Resume")$ accepts only the callee that this	239	two other traces, and a $ctor("Resume")$ accepts only the callee that this
241	activation itself created. Activations do not run concurrently, so nothing	240	activation itself created. Activations do not run concurrently, so nothing
242	corresponds to reading globals written by another thread.	241	corresponds to reading globals written by another thread.
243		242	
244	=== Valid traces follow graph steps #thy-badge("Voblint_CFG", "Activation_Trace_Def") <sec:valid>	243	=== Valid traces #thy-badge("Voblint_CFG", "Activation_Trace_Def") <sec:valid>
245		244	

```
246 The datatype also admits terms that no execution produces. A path may contain
247 a step that follows no edge of the graph, for instance, a jump from the entry of
                                31 unchanged lines hidden
279 continuation $italic("cont")$, where the caller continues with the store that
280 #isaconst("combine_collect") builds from both final stores.
281
282 === Every source execution yields a valid trace #thy-badge("Voblint_Compile",
    "Source_To_Trace") <sec:source-bridge>
283
284 The rules of valid traces mirror the rules of #isaconst("cstep"), but nothing
285 so far states that a graph execution actually yields a valid trace. Graph configurations are
    therefore related to traces.
286 Besides the node and the store, the relation matches the trace's callers with
287 the frame stack, so that it still holds after a return.
288
289 #block(breakable: false)[
290   #definition(name: [Trace representation], isa: "activation_trace_repr", cmd: "definition")[
                                10 unchanged lines hidden
301
302 For a well-formed compiled program (#isaconst("wf_compile_input")), the
303 correspondence is lock-step. A local graph step extends the current trace, a
304 call creates a $ctor("Call", thy: "Activation_Trace_Def")$ trace, and a return creates a
    $ctor("Resume")$
305 trace (#isathm("cstep_preserves_activation_trace_repr")). The initial configuration has a
    representing trace, a $ctor("Root")$ trace
306 (#isathm("located_activation_trace_entry")). Hence,
307 every configuration that a graph execution of such a program reaches has a
308 representing valid trace (#isathm("csteps_preserve_located_activation_trace")). The converse
    is neither proved nor needed.
309
310 #block(breakable: false)[
311   #theorem(name: [Source executions are traces], isa: "source_run_has_activation_trace")[
                                10 unchanged lines hidden
322 ]
323
324 The proof composes the simulation of @sec:csim (#isathm("csim_star")) with
325 the lock-step correspondence above. The source configuration, the graph
326 configuration, and the trace hold the same store, so every store a source
327 execution reaches is the final store of a valid trace.
328
329 === Collecting trace stores per node #thy-badge("Voblint_CFG", "Activation_Trace_Collect")
    <sec:collect>
```

```
245 The datatype also admits terms that no execution produces. A path may contain
246 a step that follows no edge of the graph, for instance, a jump from the entry of
                                31 unchanged lines hidden
278 continuation $italic("cont")$, where the caller continues with the store that
279 #isaconst("combine_collect") builds from both final stores.
280
281 === Valid traces of source executions #thy-badge("Voblint_Compile", "Source_To_Trace")
    <sec:source-bridge>
282
283 The rules of valid traces mirror the rules of #isaconst("cstep"), but nothing
284 so far states that a graph execution yields a valid trace. The following
285 relation closes this gap. It also matches the trace's callers with the frame
286 stack, so that it still holds after a return.
287
288 #block(breakable: false)[
289   #definition(name: [Trace representation], isa: "activation_trace_repr", cmd: "definition")[
                                10 unchanged lines hidden
300
301 For a well-formed compiled program (#isaconst("wf_compile_input")), the
302 correspondence is lock-step. A local graph step extends the current trace, a
303 call creates a $ctor("Call", thy: "Activation_Trace_Def")$ trace, and a return
304 a $ctor("Resume")$ trace (#isathm("cstep_preserves_activation_trace_repr")).
305 A $ctor("Root")$ trace represents the initial configuration
306 (#isathm("located_activation_trace_entry")), so every reached configuration
307 has a representing valid trace
308 (#isathm("csteps_preserve_located_activation_trace")). The converse is neither
309 proved nor needed.
310
311 #block(breakable: false)[
312   #theorem(name: [Source executions are traces], isa: "source_run_has_activation_trace")[
                                10 unchanged lines hidden
323 ]
324
325 The proof composes the simulation of @sec:csim (#isathm("csim_star")) with
326 this lock-step correspondence, which both preserve the store.
327
328 === Trace stores per node #thy-badge("Voblint_CFG", "Activation_Trace_Collect") <sec:collect>
```

```
Collecting the final stores of the valid traces per node gives the _node
collecting semantics_. It plays the role of the collecting semantics of @ch:background for programs
```

19 unchanged lines hidden

```
structure. The same set of traces thus supports two views: collected per node,
and split by context (@sec:contexts).

== Assigning calling contexts to traces #thy-badge("Voblint_CFG", "Activation_Trace_Context") <sec:contexts>

A context-sensitive analysis of the running example claims the result $6$ for
`bump` in the context of the call `bump(5)` and $5$ in that of `bump(4)`. Checking
such claims requires knowing which executions each context stands for. This
section assigns contexts to activation traces and uses them to split the
stores collected at each node.
```

20 unchanged lines hidden

```
]

A policy returns a set of contexts rather than one, because an analysis may
split a call into cases. Goblint's entry operation may return several
alternatives for one call, each a pair of a resume state and an entry
state, and each alternative gets its own context. Suppose a caller passes an
argument that is either negative or positive, and its state keeps the two
cases apart, as two path-sensitive states. With one alternative
per case, the callee is analyzed in one context for negative arguments and in
another for positive ones, and a sign analysis keeps the sign in both, whereas a
single entry state would join the cases to $ltop$. When the cases overlap, one
concrete call belongs to several contexts at once
(#isathm("ov_two_contexts_admitted")).

The set may also be empty, for instance, at a call that the analysis
considers unreachable. An entry-state policy admits no context there, because
the analysis's state at the call site describes no caller store
(@sec:eq-routing). Stores that the analysis result excludes need no context. At a call that the program actually makes, however, an empty set
would lose the callee, which the coverage contract rules out (@sec:contract).

A policy speaks about a single call. The contexts of a whole trace follow by
applying it along the calls that created the trace. The root activation
carries the initial context #isai("c\<^sub>0"). A callee carries every
```

```
]

A policy returns a set of contexts rather than one, because an analysis may
split a call into cases. Goblint's $"enter"^sharp$ (@sec:interproc) may
return several alternatives for one call, each a pair of a caller state for the
return and a callee entry state, and each alternative gets its own context.
Suppose a caller passes an argument that is either negative or positive, and
its state keeps the two cases apart. A path-sensitive analysis, for instance,
keeps one state per path. With one alternative per case, the callee is analyzed in one context for negative arguments and in
another for positive ones, and a sign analysis keeps the sign in both, whereas a
single entry state would join the cases to $ltop$. When the cases overlap, one
concrete call belongs to several contexts at once
(#isathm("ov_two_contexts_admitted")).

The set may also be empty, for instance, at a call that the analysis
considers unreachable. An entry-state policy admits no context there, because
the analysis's state at the call site describes no caller store
(@sec:eq-routing). Stores that the analysis result excludes need no context. At a call that the program actually makes, however, an empty set
would lose the callee, which the coverage contract rules out (@sec:contract).

A policy speaks about a single call. The contexts of a whole trace follow by
applying it along the calls that created the trace. The root activation
carries the initial context #isai("c\<^sub>0"). A callee carries every
```

406	context c' that the policy admits for its call from some context c of its	404	context c' that the policy admits for its call from some context c of its caller (#isaconst("admits_call_context")). The policy receives the caller's store and the entered store that #isaconst("call_enter") computes from it. A
407	caller (#isaconst("admits_call_context"), which passes the policy the caller's		
408	store and the entered store that #isaconst("call_enter") computes from it). A		
409	local step keeps the contexts of the trace	405	local step keeps the contexts of the trace
410	(#isathm("activation_context_rel_extend")), and a resumed caller carries	406	(#isathm("activation_context_rel_extend")), and a resumed caller carries
411	the contexts it had before the call	407	the contexts it had before the call
	20 unchanged lines hidden		
432	partial tabulation, which tabulates procedure summaries only for the calling	428	partial tabulation, which tabulates procedure summaries only for the calling
433	contexts the analysis meets @saan26phd[§3.3] @schwarz25phd[§5, p. 110]. Unlike	429	contexts the analysis meets @saan26phd[§3.3] @schwarz25phd[§5, p. 110]. Unlike
434	bounded call strings, they can form an infinite set, so the analysis may not	430	bounded call strings, they can form an infinite set, so the analysis may not
435	terminate (@sec:certificate). The traces themselves store no context, since	431	terminate (@sec:certificate). The traces store no context, so the concrete
436	the concrete semantics should not change with the analysis. The policy is	432	semantics stays independent of the analysis, and the theorems take the policy
437	instead a parameter of the theorems, which therefore hold for every policy. It	433	as a parameter and hold for every policy. A policy may even be defined from
438	may even be defined from the analysis result, as the entry-state policy is	434	the analysis result, as the entry-state policy is (@sec:eq-routing). This is
439	(@sec:eq-routing). This is not circular, because the final guarantee concerns	435	not circular, because no policy changes the stores collected per node. The
440	the stores collected per node, which no policy changes. The policy only	436	policy only splits them among contexts.
441	decides how these stores are split among contexts.		
442		437	
443	The node collecting semantics can now be split by context. A store belongs to	438	The node collecting semantics can now be split by context. A store belongs to
444	context c at node v if some valid trace ending at v with that store	439	context c at node v if some valid trace ending at v with that store
	11 unchanged lines hidden		
456		451	
457	The set #isai("\<A>\<^bsub>\<G>,adm,c_0,g,S\<^esub> v c") is called the _activation collecting semantics_ at v and c.	452	The set #isai("\<A>\<^bsub>\<G>,adm,c_0,g,S\<^esub> v c") is called the _activation collecting semantics_ at v and c.
458	It contains the final stores of the valid traces that end at v and carry	453	It contains the final stores of the valid traces that end at v and carry
459	c, and the abstract state computed for $[v, c]$ must describe it. In the running example under interval entry-state contexts, `bump(5)` gets the	454	c, and the abstract state computed for $[v, c]$ must describe it. In the
460	context $[[5, 5]]$ and `bump(4)` the context $[[4, 4]]$	455	running example under interval entry-state contexts (@sec:eq-routing), the
461	(@sec:eq-routing). The final store of `bump(5)`, with result 6, lies in	456	final store of `bump(5)`, with result 6, lies in the set of context
462	the activation collecting semantics of context $[[5, 5]]$ at `bump`'s result node, and the final store of `bump(4)`, with result 5, in that of context	457	$[[5, 5]]$ at `bump`'s result node, and that of `bump(4)`, with result 5, in
463	$[[4, 4]]$. The node collecting semantics there contains both stores.	458	the set of context $[[4, 4]]$. Each context's state thus describes one result,
464		459	whereas a single state for the node bounds it at best by $absval(ivl(5, 6))$
465	Splitting by context makes the result more precise. A single abstract state	460	(@sec:interproc). No store is lost as long as every valid trace carries a
466	for the result node must describe both stores, so an interval analysis can at best bound the result by $absval(ivl(5, 6))$. A state per context only has to describe the stores of	461	context. The next section reduces the requirement to contain these sets to
467	its own context, 6 in $[[5, 5]]$ and 5 in $[[4, 4]]$. No store is lost as long as every valid trace carries a context. An analysis	462	local checks.
468	therefore needs one set of stores per node and context that contains the		
469	activation collecting semantics, and the next section reduces this requirement		
470	to local checks.		
471		463	

472	== The coverage contract and its soundness #thy-badge("Voblint_CFG", "Activation_Trace_Abstract") <sec:contract>	464	== The coverage contract and its soundness #thy-badge("Voblint_CFG", "Activation_Trace_Abstract") <sec:contract>
473		465	
474	An analysis result is sound if, at every node v and in every context c,	466	An analysis result is sound if, at every node v and in every context c,
475	it accounts for the stores of the activation collecting semantics	467	it accounts for the stores of the activation collecting semantics
476	#isai("\<A>\<^bsub>\<G>,adm,c_0,g,S\<^esub> v c"). This is stated over a _cover_, a function #isai("cover") that assigns a set of	468	#isai("\<A>\<^bsub>\<G>,adm,c_0,g,S\<^esub> v c"). Soundness is stated over a _cover_, a function #isai("cover") that assigns a set of
477	stores to each node and context. Since valid	469	stores to each node and context. Since valid
478	traces grow by only four rules, it suffices to check the cover against each	470	traces grow by only four rules, it suffices to check the cover against each
479	rule once. The _coverage contract_ does this with one local obligation per rule,	471	rule once. The _coverage contract_ does this with one local obligation per rule,
	20 unchanged lines hidden		
500	In #oblig("CALL"), "c' admitted from s" means that the policy admits c'	492	In #oblig("CALL"), "c' admitted from s" means that the policy admits c'
501	for this call edge from s in context c. #oblig("RETURN") must hold for	493	for this call edge from s in context c. #oblig("RETURN") must hold for
502	every admitted c', since one call may admit several contexts and the	494	every admitted c', since one call may admit several contexts and the
503	returning activation may carry any of them. Because c' ranges only over	495	returning activation may carry any of them. In "c' admitted at u from s for
504	admitted contexts, the return of `bump(5)` combines only callee results of		
505	contexts entered from this call. In "c' admitted at u from s for		
506	(p', e)", the admitting call edge may be any edge leaving u, into p' with	496	(p', e)", the admitting call edge may be any edge leaving u, into p' with
507	entered store e (#isaconst("admits_call_context")). A compiled graph has one call edge per	497	entered store e (#isaconst("admits_call_context")). A compiled graph has one call edge per
508	call site (#isaconst("calls_source_unique"),	498	call site (#isaconst("calls_source_unique"),
509	#isathm("compile_prog_calls_source_unique")), so on such a graph the admitting	499	#isathm("compile_prog_calls_source_unique")), so on such a graph the admitting
510	edge is always the call edge of the return. In the running	500	edge is always the call edge of the return. In the running
511	example under entry-state contexts, #oblig("CALL") requires the cover at the	501	example under entry-state contexts, #oblig("CALL") requires the cover at the
512	entry of `bump` in context $[[5, 5]]$ to contain the entered store with	502	entry of `bump` in context $[[5, 5]]$ to contain the entered store with
513	$n = 5$, and #oblig("RETURN") requires the cover at the continuation in `main`	503	$n = 5$, and #oblig("RETURN") requires the cover at the continuation in `main`
514	to contain the store in which `a` holds 6. @fig:contract shows the	504	to contain the store in which `a` holds 6. @fig:contract shows these four
515	obligations at one call, together with a fifth, #oblig("TOTAL").	505	obligations at one call. A fifth obligation, #oblig("TOTAL"), concerns calls
		506	that get no context.
516		507	
517	#let _key(pos, name, body, col: vb.neutral) = node(	508	#let _key(pos, name, body, col: vb.neutral) = node(
518	pos,	509	pos,
	35 unchanged lines hidden		
554	#oblig("RETURN") reads.],	545	#oblig("RETURN") reads.],
555	) <fig:contract>	546	) <fig:contract>
556		547	
557	#oblig("CALL") and #oblig("RETURN") constrain only the contexts that the policy	548	#oblig("CALL") and #oblig("RETURN") constrain only admitted contexts. If a
558	admits. If a call from a store in the cover gets no context at all, both	549	call from a store in the cover gets no context, both hold vacuously, yet the
559	hold vacuously, and no obligation requires the cover at the call's	550	resumed caller still carries its contexts at the continuation
560	continuation to contain the store the call returns with. The resumed caller, however, still carries its contexts there	551	(#isathm("resume_keeps_context")), so the cover can miss a reached store
561	(#isathm("resume_keeps_context")), so the cover can miss a store that the	552	(#isathm("total_dropped_unsound")). #oblig("TOTAL") therefore requires the
562	program reaches (#isathm("total_dropped_unsound")). #oblig("TOTAL") therefore requires the	553	policy to admit a callee context for every call from a store
563	policy to admit at least one callee context for every call from a store	554	$s in #isai("cover u c")$ (#isaconst("call_context_total_on")). Call strings

564	$s in #isai("cover u c")$ (#isaconst("call_context_total_on")). It asks only	555	and the unit context policy meet it for every cover
565	about stores in the cover, because entry-state policies admit no context for	556	(#isathm("call_context_total_on_of_fun")), and entry-state contexts through
566	the stores that the analysis excludes (@sec:contexts). Call strings and the	557	the coverage of the analysis's entry (@sec:eq-routing). The locale
567	unit context policy meet #oblig("TOTAL") for every cover (#isathm("call_context_total_on_of_fun")). For	558	#isalocale("activation_coverage") assumes the five obligations, and they imply
568	entry-state contexts, it follows from the coverage of the analysis's entry	559	the goal by induction over the four rules of valid traces.
569	(@sec:eq-routing).		
570			
571	Isabelle states the five obligations as the assumptions of the locale		
572	#isalocale("activation_coverage").		
573		560	
574	#{	561	#{
575	show raw.where(block: true): set text(size: 6pt)	562	show raw.where(block: true): set text(size: 6pt)
576	thy("activation_coverage")	563	thy("activation_coverage")
577	}	564	}
578		565	
579	The contract implies the goal by induction over the four rules of valid		
580	traces, one obligation per rule.		
581			
582	#block(breakable: false)[	566	#block(breakable: false)[
583	#theorem(name: [Context-indexed soundness], isa: "activation_collect_sound")[	567	#theorem(name: [Context-indexed soundness], isa: "activation_collect_sound")[
584	If the cover meets the five obligations of	568	If the cover meets the five obligations of
	17 unchanged lines hidden		
602	#proved("node_collect_covered")	586	#proved("node_collect_covered")
603	]	587	]
604		588	
605	This completes the concrete half of the soundness argument. Every store that a	589	With @ch:program-model and @sec:source-bridge, this completes the concrete
606	finite source execution reaches appears in a graph execution (@ch:program-model),	590	half of the soundness argument. Every store that a finite source execution
607	as the final store of a valid trace (@sec:source-bridge), in the activation	591	reaches lies in every cover that meets the coverage contract. The following
608	collecting semantics of the trace's contexts (@sec:contexts), and hence in	592	chapters build such covers from abstract domains (@ch:domains), transfer
609	every cover that meets the coverage contract. An analysis therefore never	593	functions (@ch:analysis-interface), and a solved equation system
610	reasons about whole runs, only about sets of stores that each kind of step	594	(@ch:equations, @ch:solving).
611	preserves. The following chapters build such analyses from abstract domains		
612	(@ch:domains), transfer functions (@ch:analysis-interface), and a solved		
613	equation system (@ch:equations, @ch:solving).		

thesis/content/05-domains.typ

77e07947 → 0edeac81

1003 old lines, 998 new lines

91% similar lines **+89** **-94** **20 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node	1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node
2	#import "../lib/math.typ": *	2	#import "../lib/math.typ": *
3	#import "../lib/code.typ": (	3	#import "../lib/code.typ": (

9 unchanged lines hidden

```
= Abstract Domains <ch:domains>

@ch:traces reduced soundness to five obligations on a cover, which assigns
each node and context a set of stores. The obligations are local conditions,
and together they guarantee that the cover contains every store that an execution in that
context reaches at the node (#isathm("activation_collect_sound")). These sets
are arbitrary and usually infinite. An analyzer, however, computes with finite
abstract values, and its solver (@sec:td) compares, joins, widens, and narrows them without
knowing what they mean. Recall from @sec:abs-int that an abstract value $a$
denotes a set $conc(a)$ of concrete values. A value of a numeric domain such
as Sign or Interval denotes a set of integers, $conc(a) subset.eq ZZ$. An
abstract state $d$, in contrast, denotes a set of stores,
$conc(d) subset.eq (#isatype("vname") -> ZZ)$ (@sec:domain-states). An
arbitrary lattice of abstract values is not enough, for two reasons. First,
the order must agree with the concretization. The solver only proves inequalities
$a lle b$ in the abstract order, while the obligations of @ch:traces are
inclusions between sets. The law
$ a lle b ==> conc(a) subset.eq conc(b) $
turns each such inequality into an inclusion (#isathm("gamma_mono"),
@fig:sign-conc). Second, a value can denote no integer without being the
lattice's bottom element. The interval pair $absval(ivl(5, 3))$, whose lower bound
exceeds its upper bound, is such a value. An analyzer that recognizes only the
bottom element as empty misses it. This is still sound, since the analyzer
then assumes more stores than exist, but it costs precision. After the next
join, the information that a program point is unreachable is gone
(@sec:nonrel-state).

This chapter derives what a domain must provide. The solver needs operations
it can compute with, the soundness proof needs laws that turn the solver's
inequalities into inclusions between sets of stores, and the analyzer needs to
recognize every value that denotes no integer, so that an unreachable program point
stays unreachable.
```

```
= Abstract Domains <ch:domains>

This chapter determines what an abstract domain must provide so that the
abstract values a solver computes over-approximate the stores that executions
reach. @ch:traces reduced soundness to five
obligations on a cover, which assigns each node and context a set of stores.
The obligations are local conditions, and together they guarantee that the
cover contains, for each node and context, every store that an execution in
this context reaches at the node (#isathm("activation_collect_sound")). These
sets are arbitrary and usually infinite. An analyzer, however, computes with
finite abstract values, and its solver (@sec:td) compares, joins, widens, and
narrows them without knowing what they mean. A domain therefore supplies
operations the solver can compute with and laws that let the soundness proof
turn the solver's inequalities into inclusions between sets of stores.

Recall from @sec:abs-int that an abstract value $a$
denotes a set $conc(a)$ of concrete values. A value of a numeric domain such
as Sign or Interval denotes a set of integers, $conc(a) subset.eq ZZ$. An
abstract state $d$, in contrast, denotes a set of stores,
$conc(d) subset.eq (#isatype("vname") -> ZZ)$ (@sec:domain-states). An
arbitrary lattice of abstract values is not enough. First,
the order must agree with the concretization. The solver only proves inequalities
$a lle b$ in the abstract order, while the obligations of @ch:traces are
inclusions between sets. The law
$ a lle b ==> conc(a) subset.eq conc(b) $
turns each such inequality into an inclusion (#isathm("gamma_mono"),
@fig:sign-conc). Second, a value can denote no integer without being the
lattice's bottom element. The interval pair $absval(ivl(5, 3))$, whose lower bound
exceeds its upper bound, is such a value. An analyzer that recognizes only the
bottom element as empty misses it. Missing it is still sound, since the analyzer then assumes
more stores than exist, but it loses precision. After the next
join, the information that a program point is unreachable is gone
(@sec:nonrel-state).
```

```
#let _snode(pos, name, body) = node(pos, text(size: 8pt, body), name: name, inset: 3pt)
#let _order = 0.7pt + vb.neutral
#let _hit = 1.1pt + vb.accent
```

450 unchanged lines hidden

```
The meet intersects the bounds and does not normalize, since $lbot$ would not
be the greatest lower bound of, say, $absval(ivl(0, 3))$ and $absval(ivl(5, 9))$ in the order
on pairs (#isathm("meet_ivl_normalized_breaks_greatest")). An empty pair is
therefore not necessarily $lbot$, and the emptiness test inspects the bounds
(#isaconst("is_bottom_ivl")): a pair is empty if $l > u$, if its lower bound is
$+infinity$, or if its upper bound is $-infinity$. The widening is the
standard one of @sec:widening, except that widening with $lbot$ returns the
other operand (#isaconst("widen_ivl_core")), and the narrowing refines only
infinite bounds (#isaconst("narrow_ivl_td")).

== Abstract states as sets of stores <sec:domain-states>

A numeric domain describes the possible values of one variable. The
obligations of @ch:traces, however, are stated over sets of stores. A store
maps every variable name to an integer,
$#isatype("store") = #isatype("vname") -> ZZ$, so at a program point, the
analysis must describe whole stores. This thesis uses the two forms
introduced in @sec:constraints, non-relational and relational states. A non-relational
description gives each
variable its own abstract value, and a relational one records how variables
relate to each other. The two differ in what they can state. Take the stores with
$0 <= x <= y <= 5$ (@fig:rel-vs-nonrel). The best non-relational description
over intervals is ${x |-> absval(ivl(0, 5)), y |-> absval(ivl(0, 5))}$. It shows $x >= 0$ and
$y <= 5$, but not $x <= y$, since it also admits $x = 5, y = 0$. A relational
```

40 unchanged lines hidden

```
each state $d$ to the set $conc(d)$ of stores it describes. Each kind of state
defines its own concretization, and Isabelle overloads one constant,
#isaconst("gamma_S"), written $conc(d)$, for all of them. The numeric
analyses use non-relational states. An analysis that records relations
$x <= y$ between variables uses a relational state, whose concretization maps
a whole state to a set of stores instead of each variable to a set of
integers. The interface of this chapter admits such a state, because the
solver and the framework ask only for order, join, and a monotone
concretization. Neither requires the state to map variables to abstract values. The
relational state of the order analysis instantiates the solver's classes and
#isalocale("executable_domain") without being a numeric domain. The
```

```
#let _snode(pos, name, body) = node(pos, text(size: 8pt, body), name: name, inset: 3pt)
#let _order = 0.7pt + vb.neutral
#let _hit = 1.1pt + vb.accent
```

450 unchanged lines hidden

```
The meet intersects the bounds and does not normalize, since $lbot$ would not
be the greatest lower bound of, say, $absval(ivl(0, 3))$ and $absval(ivl(5, 9))$ in the order
on pairs (#isathm("meet_ivl_normalized_breaks_greatest")). A meet can
therefore return an empty pair other than $lbot$. The emptiness test
(#isaconst("is_bottom_ivl")) therefore inspects the bounds. It reports a pair
as empty if $l > u$, if its lower bound is
$+infinity$, or if its upper bound is $-infinity$. The widening is the
standard one of @sec:widening, except that widening with $lbot$ returns the
other operand (#isaconst("widen_ivl_core")), and the narrowing refines only
infinite bounds (#isaconst("narrow_ivl_td")).

== Abstract states as sets of stores <sec:domain-states>

A numeric domain describes the possible values of one variable. The
obligations of @ch:traces, however, are stated over sets of stores. A store
maps every variable name to an integer,
$#isatype("store") = #isatype("vname") -> ZZ$, so at a program point, the
analysis must describe whole stores. A non-relational description
(@sec:constraints) gives each variable its own abstract value. A relational
description instead records how variables relate to each other. The two differ in what they
can state. Take the stores with
$0 <= x <= y <= 5$ (@fig:rel-vs-nonrel). The best non-relational description
over intervals is ${x |-> absval(ivl(0, 5)), y |-> absval(ivl(0, 5))}$. It shows $x >= 0$ and
$y <= 5$, but not $x <= y$, since it also admits $x = 5, y = 0$. A relational
```

40 unchanged lines hidden

```
each state $d$ to the set $conc(d)$ of stores it describes. Each kind of state
defines its own concretization, and Isabelle overloads one constant,
#isaconst("gamma_S"), written $conc(d)$, for all of them. The numeric
analyses use non-relational states. The concretization of a relational state
maps the whole state to a set of stores instead of each variable to a set of
integers. The interface of this chapter admits such a state, because the
solver and the framework ask only for order, join, and a monotone
concretization. The relational state that records relations $x <= y$
(@sec:rel-state) instantiates the solver's classes and
#isalocale("executable_domain") without being a numeric domain. The remaining
```

```
574 remaining sections of this chapter develop the non-relational case, and
575 @sec:rel-state presents the relational one.
576
577 == Pointwise states and the unreachable state #thy-badge("Voblint_Domain",
    "Nonrelational_State") <sec:nonrel-state>
578
```

65 unchanged lines hidden

```
644 empties a single variable, as in ${x |-> signval(bot), y |-> signval(top)}$. Treating
645 every empty state as unreachable is correct at the moment of the test, but the
646 information does not survive the next step. Joining two differently empty
647 states can give a nonempty one (@fig:domain-reachability). This is sound,
648 since it only adds stores, but it loses the information that the point is unreachable
649 and is thus imprecise.
650
651 #let _r = claim-row("dom-disjunct-sign", "13:5")
652 #assert(_r.verdict == "DEAD", message: "dom-disjunct-sign: the caption says unreachable")
```

27 unchanged lines hidden

```
680     #claim-ref("dom-disjunct-sign")).],
681 ) <fig:domain-reachability>
682
683 Voblint therefore makes unreachability a value of its own, as Nipkow and
684 Klein do with an option type @nipkow14[§13.7] and as Goblint does. Goblint's
685 framework lifts the local state by an outer bottom element for dead code
686 (#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/
    src/framework/analyses.ml#L120-L135")[`Analyses.Dom`]), and a transfer function that finds
    its path unreachable
687 raises the `Deadcode` exception instead of returning a state. In Voblint, a
688 lifted state (#isatype("lifted")) is either #ctor("Bot"), meaning unreachable,
689 or $ctor("Lifted")(d)$ for an ordinary state $d$, with #ctor("Bot") below every
690 other value and $conc(ctor("Bot")) = emptyset$.
691
692 After every transfer, the analysis tests whether the result is empty and, if
693 so, replaces it with #ctor("Bot") (#isaconst("normalize_lift")). Comparing with
694 the bottom state cannot implement this test, for the reason above. Instead, the test relies
    on the domain's emptiness test, which #isathm("is_empty_correct")
695 makes exact for every value. A pointwise state is empty exactly when some
696 variable's value is (#isaconst("is_empty_state"),
697 #isathm("is_empty_state_iff_gamma_state_empty")). The variable names form an
```

```
574 sections of this chapter develop the non-relational case.
575
576 == Pointwise states and the unreachable state #thy-badge("Voblint_Domain",
    "Nonrelational_State") <sec:nonrel-state>
577
643 empties a single variable, as in ${x |-> signval(bot), y |-> signval(top)}$. Treating
644 every empty state as unreachable is correct at the moment of the test, but the
645 information does not survive the next step. Joining two differently empty
646 states can give a nonempty one. The join is sound, since it only adds stores,
647 but it loses the information that the point is unreachable.
648 @fig:domain-reachability shows a program where this join occurs. Its analyzer
649 output, like every analyzer output quoted in this thesis, comes from a
650 _claim_, a recorded analyzer run that each build of the thesis repeats and
651 compares with the quoted output.
652
653 #let _r = claim-row("dom-disjunct-sign", "13:5")
654 #assert(_r.verdict == "DEAD", message: "dom-disjunct-sign: the caption says unreachable")
682     #claim-ref("dom-disjunct-sign")).],
683 ) <fig:domain-reachability>
684
685 Voblint therefore makes unreachability a value of its own. A lifted state
686 (#isatype("lifted")) is either #ctor("Bot"), meaning unreachable, or
687 $ctor("Lifted")(d)$ for an ordinary state $d$, with #ctor("Bot") below every
688 other value and $conc(ctor("Bot")) = emptyset$. Nipkow and Klein use an option
689 type for this purpose @nipkow14[§13.7]. Goblint's framework lifts the local
690 state by an outer bottom element for dead code
691 (#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/
    src/framework/analyses.ml#L120-L135")[`Analyses.Dom`]).
692 A Goblint transfer function that finds its path unreachable raises the
693 `Deadcode` exception instead of returning a state.
694
695 After every transfer, the analysis tests whether the result is empty and, if
696 so, replaces it with #ctor("Bot") (#isaconst("normalize_lift")). The test
697 relies on the domain's emptiness test, which #isathm("is_empty_correct") makes
698 exact for every value. A pointwise state is empty exactly when some
699 variable's value is (#isaconst("is_empty_state"),
700 #isathm("is_empty_state_iff_gamma_state_empty")). The variable names form an
```

698	infinite set, so the executable analyzer decides this on a finite	701	infinite set, so the executable analyzer decides this on a finite
	9 unchanged lines hidden		
708		711	
709	== Forward evaluation and truth tests #thy-badge("Voblint_Domain", "Forward_Domain") <sec:domain-forward>	712	== Forward evaluation and truth tests #thy-badge("Voblint_Domain", "Forward_Domain") <sec:domain-forward>
710		713	
711	The carrier operations serve the solver. The analysis additionally needs	714	The operations of @sec:domain-carrier-laws serve the solver. The analysis
712	operations that mirror the program's own arithmetic, comparisons, and branch conditions. They are called the domain's _primitives_. This section	715	additionally needs operations that mirror the program's own arithmetic,
713	and the next two introduce them, one use at a time.	716	comparisons, and branch conditions. They are called the domain's
		717	_primitives_, and each of them is introduced where the analysis first uses it.
714		718	
715	An assignment `x = e` needs the abstract value of e. Forward evaluation	719	An assignment `x = e` needs the abstract value of e. Forward evaluation
716	computes it from an abstract state d, as in the generic abstract interpreter	720	computes it from an abstract state d, as in the generic abstract interpreter
	16 unchanged lines hidden		
733	$"Some"("false")$ if $l = u = 0$. Otherwise, it returns $"None"$. So	737	$"Some"("false")$ if $l = u = 0$. Otherwise, it returns $"None"$. So
734	$absval(ivl(1, 5))$ is true, $absval(ivl(0, 0))$ is false, and $absval(ivl(0, 5))$ is undecided.	738	$absval(ivl(1, 5))$ is true, $absval(ivl(0, 0))$ is false, and $absval(ivl(0, 5))$ is undecided.
735		739	
736	== Answering checks with three-valued queries <sec:queries>	740	== Three-valued queries for checks <sec:queries>
737		741	
738	A check `__voblint_check(c)` asks whether c holds in every store that	742	A check `__voblint_check(c)` asks whether c holds in every store that
739	reaches its program point. The analysis must answer this from the abstract	743	reaches its program point. The analysis must answer this from the abstract
	39 unchanged lines hidden		
779		783	
780	Negation keeps _unknown_. The connectives `!`, `&&`, and `||` combine the answers	784	Negation keeps _unknown_. The connectives `!`, `&&`, and `||` combine the answers
781	of their operands in three-valued logic. A definite _false_ decides `&&`, and a	785	of their operands in three-valued logic. A definite _false_ decides `&&`, and a
782	definite _true_ decides `||`, regardless of the other operand's answer. Both	786	definite _true_ decides `||`, regardless of the other operand's answer. Any
783	operands are evaluated in the same state, so for x in $absval(ivl(1, 2))$,	787	other expression e is read as $e != 0$, its truth value in VIMP. A new domain
784	`x == 1 || x == 2` stays unknown, although every store satisfies it (claim		
785	#claim-ref("dom-disjunction-one-state")). Goblint is more precise here. CIL		
786	compiles `&&` and `||` into branches		
787	(#link("https://github.com/goblint/cil/blob/003821f1d4a95c758bb255080e9b91a22ea5df31/src/frontc/cabs2cil.ml#L5264-L5299")[`cabs2cil.ml`]),		
788	and where `x == 1` fails, Goblint's interval refinement narrows $absval(ivl(1, 2))$ to		
789	$absval(ivl(2, 2))$		
790	(#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/analyses/baseInvariant.ml#L395-L410")[`baseInvariant.ml`],		
791	@saan26phd[§4.5.1, Rem. 8]). Voblint's interval inverse of a failed equality		
792	changes nothing (#isaconst("inv_eq_ivl")). Any other		
793	expression e is read as $e != 0$, its truth value in VIMP. A new domain		
794	therefore gets checks on arbitrary conditions by providing these two	788	therefore gets checks on arbitrary conditions by providing these two
795	comparisons.	789	comparisons.
796	One proof covers every domain. The answer _true_ certifies that c holds in		

797	every store the state describes, _false_ that it holds in none, and _unknown_		
798	makes no claim (#isathm("check_query_sound")). #isalocale("sound_check_query")		
799	combines the queries with forward evaluation. An analysis passes these answers		
800	to the analyses it runs with and to the verdicts as answers to the query		
801	#isaconst("EvalInt") of @sec:coop-queries (@sec:coop-mcp, @sec:verdicts).		
802		790	
803	For example, the condition `x <= y && y != 0` is answered from $"less"(hat(y), hat(x))$ and	791	For example, the condition `x <= y && y != 0` is answered from $"less"(hat(y), hat(x))$ and
804	$"eq"(hat(y), hat(0))$ alone. In Sign, with x non-positive and y positive,	792	$"eq"(hat(y), hat(0))$ alone. In Sign, with x non-positive and y positive,
	35 unchanged lines hidden		
840	#_q.verdict.],	828	#_q.verdict.],
841	) <fig:query-tree>	829	) <fig:query-tree>
842		830	
843	== Refining states at branch guards #thy-badge("Voblint_Domain", "Backward_Domain") <sec:branches>	831	One proof covers every domain. The answer _true_ certifies that c holds in
		832	every store the state describes, _false_ that it holds in none, and _unknown_
		833	makes no claim (#isathm("check_query_sound")). #isalocale("sound_check_query")
		834	combines the queries with forward evaluation. An analysis passes these answers
		835	to the analyses it runs with and to the verdicts as answers to the query
		836	#isaconst("EvalInt") of @sec:coop-queries (@sec:coop-mcp, @sec:verdicts).
		837	
		838	The queries evaluate both operands of a connective in the same state, so for x in $absval(ivl(1, 2))$,
		839	`x == 1 || x == 2` stays unknown, although every store satisfies it (claim
		840	#claim-ref("dom-disjunction-one-state")). Goblint is more precise here. CIL
		841	compiles `&&` and `||` into branches
		842	(#link("https://github.com/goblint/cil/blob/003821f1d4a95c758bb255080e9b91a22ea5df31/src/frontc/cabs2cil.ml#L5264-L5299")[`cabs2cil.ml`]),
		843	and where `x == 1` fails, Goblint's interval refinement narrows $absval(ivl(1, 2))$ to
		844	$absval(ivl(2, 2))$
		845	(#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/analyses/baseInvariant.ml#L395-L410")[`baseInvariant.ml`],
		846	@saan26phd[§4.5.1, Rem. 8]). Voblint's interval inverse of a failed equality
		847	changes nothing (#isaconst("inv_eq_ivl")).
		848	
		849	== State refinement at branch guards #thy-badge("Voblint_Domain", "Backward_Domain") <sec:branches>
844		850	
845	When the analysis enters a branch, it knows whether the condition held. On the	851	When the analysis enters a branch, it knows whether the condition held. On the
846	true arm of `if (0 < x)`, for example, x is positive. Ignoring this knowledge is sound, since the	852	true arm of `if (0 < x)`, for example, x is positive. Ignoring this knowledge is sound, since the
	18 unchanged lines hidden		
865	)	871	)
866	}))	872	}))
867		873	

868	In this program, y ends up as $\|x\|$, so the check always holds. Without	874	In this program, y ends up as $\|x\|$, so the check always holds. If the
869	guard filtering, both arms keep $x \|-> ltop$, y becomes $ltop$, and the state	875	analysis ignores the guard, both arms keep $x \|-> ltop$, y becomes $ltop$, and the state
870	cannot show that the check holds. The guard filter runs the condition in	876	cannot show that the check holds. The guard filter runs the condition in
871	reverse, as in the backward abstract interpretation of tests by Cousot	877	reverse, as in the backward abstract interpretation of tests by Cousot
872	@cousot99[§§8.5, 10.1] and the backward analysis of Nipkow and Klein	878	@cousot99[§§8.5, 10.1] and the backward analysis of Nipkow and Klein
	60 unchanged lines hidden		
933	interpreting #isalocale("sound_nonrelational_ops"). For Interval, the record	939	interpreting #isalocale("sound_nonrelational_ops"). For Interval, the record
934	is #isaconst("ivl_ops").	940	is #isaconst("ivl_ops").
935		941	
936	A domain supplies nothing else. In particular, it writes no transfer	942	These two groups are all a domain supplies. The primitives determine the
937	functions, because the primitives determine every edge	943	transfer function of every edge (#isaconst("tf_abs"), #isaconst("enter_for")),
938	(#isaconst("tf_abs"), #isaconst("enter_for")):	944	so a domain writes none itself:
939	#block(inset: (left: 1.5em, y: 0.3em))[	945	#block(inset: (left: 1.5em, y: 0.3em))[
940	#grid(	946	#grid(
941	columns: (auto, 1fr),	947	columns: (auto, 1fr),
	35 unchanged lines hidden		
977		983	
978	== Carriers without least upper bounds <sec:excluded-carriers>	984	== Carriers without least upper bounds <sec:excluded-carriers>
979		985	
980	The interface asks more of a carrier than an order with a join. The verified TD	986	The verified TD solver requires a #isalocale("bounded_semilattice_sup_bot")
981	solver requires a #isalocale("bounded_semilattice_sup_bot")		
982	(#isalocale("TD_side_upd_rule")), whose join is the _least_ upper bound, and	987	(#isalocale("TD_side_upd_rule")), whose join is the _least_ upper bound, and
983	the framework's proofs use this leastness as well	988	the framework's proofs use this leastness as well
984	(#isathm("sound_refinement.bfilter_reductive")). Not every domain in	989	(#isathm("sound_refinement.bfilter_reductive")). Goblint's exclusion-set
985	practical use meets this.	990	domain #link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/cdomain/value/cdomains/int/defExcDomain.ml")[`DefExc`]
986		991	loses this property once its bit ranges are dropped. It describes an integer by a
987	Goblint's exclusion-set domain #link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/cdomain/value/cdomains/int/defExcDomain.ml")[`DefExc`] describes an integer either by a	992	definite value, `Definite n`, or by `Excluded S`, every integer outside a
988	definite value, `Definite n`, or by a finite set of excluded values,	993	finite set S, and `Excluded S` lies below `Excluded S'` when $S' subset.eq S$.
989	`Excluded S`, which stands for every integer outside S. A smaller exclusion	994	Over mathematical integers, every upper bound of `Definite 1` and `Definite 2`
990	set is less precise, so `Excluded S` lies below `Excluded S'` when S' is a	995	is some `Excluded S` with $1, 2 in.not S$, and any $p in.not S union {1, 2}$
991	subset of S. Goblint also attaches a bit range to every value, which makes	996	gives the strictly smaller upper bound `Excluded` $(S union {p})$. Admitting
992	the domain finite for each integer type. Read over mathematical integers,	997	such a carrier, by bounded integers as in Goblint or by a weaker join, is left
993	without this range, the domain has no least upper bound of `Definite 1` and	998	for future work.
994	`Definite 2`. An upper bound must describe both integers, so it is some		
995	`Excluded S` with $1, 2 in.not S$. Adding a further excluded value		
996	$p in.not S union {1, 2}$ gives `Excluded` $(S union {p})$, which is again an		
997	upper bound and lies strictly below. Since S is finite and the integers are		
998	not, every upper bound has a smaller one, and no least upper bound exists.		
999			
1000	Voblint could admit such a carrier only by bounding its integers, as Goblint		

1001	does, or by weakening the join to an arbitrary upper bound throughout the		
1002	solver and the framework and reproving the results that rely on leastness.		
1003	Both are left for future work.		

thesis/content/06-analysis-interface.typ

77e07947 → 0edeac81

264 old lines, 268 new lines

83% similar lines **+48** **-44** **13 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8": diagram, node	1	#import "@preview/fletcher:0.5.8": diagram, node
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": isaconst, isai, isalocale, isathm, isatype, listing, oblig, thy-badge	3	#import "../lib/code.typ": isaconst, isai, isalocale, isathm, isatype, listing, oblig, thy-badge
4	#import "../lib/math.typ": *	4	#import "../lib/math.typ": *
5	#import "../lib/sources.typ": thy	5	#import "../lib/sources.typ": thy
6	#import "../lib/figures.typ": call-edge, entry-node, intra-edge, ppoint, snapshot-var	6	#import "../lib/figures.typ": call-edge, entry-node, intra-edge, ppoint, snapshot-var
7	#import "../lib/claims.typ": claim-ref	7	#import "../lib/claims.typ": claim-ref
8		8	
9	= The Analysis Interface <ch:analysis-interface>	9	= The Analysis Interface <ch:analysis-interface>
10		10	
		11	The obligations of @ch:traces speak about the edges, calls, and contexts of a
		12	program. Much of what discharges them is the same for every analysis, namely,
		13	the equations built from the control-flow graph, the calling contexts, and the
		14	solver. Voblint therefore separates an analysis from the framework that runs
		15	it, as Goblint does @seidl26[§6] (@sec:state-of-art), and makes the split in
		16	its proof as well as in its program. This chapter states what an analysis must
		17	supply, and what it must prove about it, so that the framework can discharge
		18	the obligations of @ch:traces once for every analysis. An analysis supplies a
		19	record of abstract operations, and the analysis contract states what they
		20	preserve.
		21	
11	@ch:domains ended with laws about one abstract state. Its order agrees with	22	@ch:domains ended with laws about one abstract state. Its order agrees with
12	its meaning, and its operations over-approximate the concrete ones. The	23	its meaning, and its operations over-approximate the concrete ones. For the
13	obligations of @ch:traces, however, speak about edges, calls, and contexts, so	24	non-relational domains of @ch:domains, these primitives determine every edge
14	something must say what an analysis does at each step of a program. The	25	transfer (@sec:domain-contract). Other analyses need more. A relational
15	non-relational domains of @ch:domains settle this through their primitives,	26	analysis does not store one value per variable (@sec:rel-state), an analysis
16	which determine every edge (@sec:domain-contract). Other analyses need more. A	27	may keep facts that belong to no program point, and analyses may need each
17	relational analysis does not store one value per variable (@sec:rel-state), an	28	other's facts to be precise (@ch:cooperation). Calls, moreover, need more than
18	analysis may keep facts that belong to no program point, and analyses may need	29	one transfer, because the caller's locals survive the call, while the callee's
19	each other's facts to be precise (@ch:cooperation). Calls, moreover, need more	30	globals and result flow back.
20	than one transfer, because the caller's locals survive the call, while the		
21	callee's globals and result flow back.		
22			
23	Everything else between an analysis and the obligations is the same for every		

Old	77e07947	New	0edeac81
24	analysis, namely, the equations built from the control-flow graph, the calling		
25	contexts, and the solver. Goblint therefore separates an analysis from the framework that runs it		
26	@seidl26[§6], and Voblint makes the same split in its program and in its proof.		
27	This chapter describes what an analysis must supply, and what it must prove		
28	about it, so that the framework can discharge the obligations of @ch:traces		
29	once for every analysis. An analysis supplies a record of abstract operations,		
30	and the analysis contract states what they preserve.		
31		31	
32	== Edge transfers and the enter/combine protocol <sec:calls>	32	== Edge transfers and the enter/combine protocol <sec:calls>
33		33	
34	An ordinary edge of the control-flow graph carries an action a, such as an	34	An ordinary edge of the control-flow graph carries an action a, such as an
35	assignment or a guard, whose concrete meaning #isaconst("edge_step") is fixed	35	assignment or a guard, whose concrete meaning #isaconst("edge_step") is fixed
36	by @ch:program-model. For a store s, #isai("edge_step a s") is the set of	36	by @ch:program-model. For a store s, #isai("edge_step a s") is the set of
37	stores the edge can produce. An analysis describes the stores at a program	37	stores the edge can produce. An analysis describes the stores at a program
38	point by an abstract state d of its _local domain_ D_L and	38	point by an abstract state d of its _local domain_ D_L and
39	supplies, for each action a, a transfer function $asem(a)$. The transfer is	39	supplies, for each action a, a transfer function $asem(a)$. The transfer is
40	sound if it loses no successor:	40	sound if it loses no successor:
41	$ s in conc(d) and s' in #isai("edge_step a s") ==> s' in conc(asem(a) thin d). $	41	$ s in conc(d) and s' in #isai("edge_step a s") ==> s' in conc(asem(a) thin d). $
42	This is the abstract counterpart of the obligation #oblig("INTRA") of	42	This condition is the abstract counterpart of the obligation #oblig("INTRA") of
43	@ch:traces for one edge.	43	@ch:traces for one edge.
44		44	
45	A call cannot be handled by one such transfer. After it returns, the caller's	45	A call cannot be handled by one such transfer. After it returns, the caller's
46	own locals are the ones from before the call, while the globals and the result	46	own locals are the ones from before the call, while the globals and the result
47	come from the callee (#isaconst("combine_collect"), #oblig("RETURN")). An analysis	47	come from the callee (#isaconst("combine_collect"), #oblig("RETURN")). An analysis
48	therefore needs two states at a call, one for the callee to start from and one	48	therefore needs two states at a call, one for the callee to start from and one
49	that saves what the caller needs when execution returns. As in Goblint, it	49	that saves what the caller needs when execution returns. As in Goblint,
50	supplies two operations. _Enter_ (#isaconst("dgs_enter")) produces both	50	_enter_ (#isaconst("dgs_enter")) produces both
51	states, and _combine_ (#isaconst("dgs_combine_env"),	51	states, and _combine_ (#isaconst("dgs_combine_env"),
52	#isaconst("dgs_combine_assign")) builds the caller's state after the call from	52	#isaconst("dgs_combine_assign")) builds the caller's state after the call from
53	the resume state and the callee's exit state. Take one call:	53	the resume state and the callee's exit state. Take one call:
	56 unchanged lines hidden		
110	#isaconst("combine_collect") does.],	110	#isaconst("combine_collect") does.],
111	) <fig:return-stores>	111	) <fig:return-stores>
112		112	
113	Like Goblint's `enter`, enter may return several pairs, one per path in	113	Enter may return several pairs, and each pair can get its own context
114	Goblint's #link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/lifters/specLifters.ml#L722")[path-sensitivity lifter] @apinis14[§5.7]. The analyses	114	(@sec:contexts). Goblint returns one pair per path in its
		115	#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/lifters/specLifters.ml#L722")[path-sensitivity lifter] @apinis14[§5.7]. The analyses
115	shipped with Voblint return one, so path-sensitive analyses remain future work.	116	shipped with Voblint return one, so path-sensitive analyses remain future work.
116		117	

```
== Local soundness laws of an analysis <sec:sound-core>

The operations of @sec:calls say how an analysis propagates abstract states.
They do not by themselves guarantee that these states cover the concrete
executions, and the framework cannot check this without knowing what the
analysis tracks. It therefore asks every analysis for local guarantees, which
@ch:equations composes into the five obligations of @ch:traces for every
program and context policy. Edge transfers account for #oblig("INTRA") and
combine for #oblig("RETURN"). The obligations #oblig("CALL") and #oblig("TOTAL") depend
on the context policy, so the framework derives them from the entry coverage of
@sec:calls together with the policy (@sec:eq-routing), and #oblig("INIT") requires the
initial state $d_0$ to cover the initial stores, which the analysis discharges
when it is registered (#isathm("dg_analysis.init_sound")).

The framework takes the operations of an analysis as one parameter, the record
#isatype("dg_spec") with one field per operation: the edge transfers, enter,
the two combine stages, and a query handler that @ch:cooperation uses to let
analyses exchange facts.

#{
  show raw.where(block: true): set text(size: 6.5pt)
  thy("dg_spec")
}

Every field except the query handler carries, marked $sharp$, the name of the
corresponding method of Goblint's `Spec`. A #isatype("call_info") names the
callee, its formals, the arguments, and the destination of a call.

The record holds the operations, and the _analysis contract_
```

```
== Local soundness laws of an analysis <sec:sound-core>

The operations of @sec:calls say how an analysis propagates abstract states.
They do not by themselves guarantee that these states cover the concrete
executions, and the framework cannot check this without knowing what the
analysis tracks. It therefore asks every analysis for local guarantees, which
@ch:equations composes into the five obligations of @ch:traces for every
program and context policy. Edge transfers account for #oblig("INTRA") and
combine for #oblig("RETURN"). The obligations #oblig("CALL") and #oblig("TOTAL") depend
on the context policy, so the framework derives them from the entry coverage of
@sec:calls together with the policy (@sec:eq-routing). Finally, #oblig("INIT")
requires the initial state $d_0$ to cover the initial stores, which the
analysis discharges when it is registered (#isathm("dg_analysis.init_sound")).

The framework takes the operations of an analysis as one parameter, the record
#isatype("dg_spec") with one field per operation: the edge transfers, enter,
the two combine stages, and a query handler that @ch:cooperation uses to let
analyses exchange facts.

#{
  show raw.where(block: true): set text(size: 6.5pt)
  thy("dg_spec")
}

Every field except the query handler carries, marked $sharp$, the name of the
corresponding method of
#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/framework/analyses.ml#L168")[`Spec`],
the module signature of a Goblint analysis. A #isatype("call_info") names the
callee, its formals, the arguments, and the destination of a call.

The record holds the operations, and the _analysis contract_
```

21 unchanged lines hidden

```
== Facts shared across program points <sec:shared-facts>

Not every fact belongs to one program point. An analysis may, for example,
keep one range for a global variable `g` that holds throughout the execution.
```

```
== Facts shared across program points <sec:shared-facts>

Not every fact belongs to one program point. A flow-insensitive fact, such as
one range for a global variable `g` across the whole execution, lives in an
unknown attached to no program point, and the points that change it contribute
to that unknown by side effects (@sec:side-effects). The transfers of
@sec:calls only map local states to local states, so the interface has to give
them access to such unknowns.
```

```
173 === Reading and publishing analysis globals #thy-badge("Voblint_Framework", "DG_Spec_Sound")
    <sec:analysis-globals>
180 === Analysis globals #thy-badge("Voblint_Framework", "DG_Spec_Sound") <sec:analysis-globals>

174
181

175 Seidl et al. describe a Goblint analysis with two domains @seidl26[§6]. The
182 Seidl et al. describe a Goblint analysis with two domains @seidl26[§6]. The

176 local domain $D_L$ holds facts per program point and context, and the _global domain_ $D_G$
    holds
183 local domain $D_L$ holds facts per program point and context, and the _global domain_ $D_G$
    holds

177 the values of _analysis globals_, which are unknowns without a program point. An assignment
    to `g`
184 the values of _analysis globals_, which are unknowns without a program point. An assignment
    to `g`

178 publishes its value to the analysis global of `g` as a side effect, and an
185 publishes its value to the analysis global of `g` as a side effect, and an

179 edge that reads `g` depends on that unknown. Goblint's
186 edge that reads `g` depends on that unknown. Goblint's `Spec` likewise

180 #link(
187 declares a local lattice `D` and a global lattice `G`.

181   "https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/
    framework/analyses.ml#L168",

182 )[`Spec`]

183 accordingly declares a local lattice `D` and a global lattice `G`.

184
188

185 The word _global_ thus has two meanings. A _program global_ is a variable of the VIMP
    program. An analysis global is a fact the
189 The word _global_ thus has two meanings. A _program global_ is a variable of the VIMP
    program. An analysis global is a fact the

186 analysis keeps flow-insensitively. The two are independent, and by default, the numeric
    analyses keep program
190 analysis keeps flow-insensitively. The two are independent, and by default, the numeric
    analyses keep program

    34 unchanged lines hidden

221 #isaconst("combine_pub") are the same for combine. An analysis without
225 #isaconst("combine_pub") are the same for combine. An analysis without

222 analysis globals ignores $eta$ and meets the three laws of @sec:sound-core.
226 analysis globals ignores $eta$ and meets the three laws of @sec:sound-core.

223
227

224 === Where program globals are stored #thy-badge("Voblint_Framework", "DG_Keyed_Split_Spec")
    <sec:mixed-flow>
228 === Placement of program globals #thy-badge("Voblint_Framework", "DG_Keyed_Split_Spec")
    <sec:mixed-flow>

225
229

226 The contract leaves open where program globals are kept. By default, program globals are part
    of the local state,
230 The contract leaves open where program globals are kept. The default placement

227 as in Goblint's default base analysis for single-threaded programs
231 of @sec:analysis-globals keeps them in the local state, as in Goblint's default

232 base analysis for single-threaded programs

228 (#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/
    src/analyses/base.ml#L539")[`base.ml`],
233 (#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/
    src/analyses/base.ml#L539")[`base.ml`],

229 @saan26phd[§5.6.1]).
234 @saan26phd[§5.6.1]).

230 The other placement keeps each program global at an analysis global of its
235 The other placement keeps each program global at an analysis global of its

    22 unchanged lines hidden

253 #claim-ref("mixed-flow-sign")). In sequential VIMP, the flow-insensitive
258 #claim-ref("mixed-flow-sign")). In sequential VIMP, the flow-insensitive

254 placement can thus cost precision.
259 placement can thus cost precision.

255
260

256 The flow-insensitive placement matters beyond this example. Another thread may
261 The flow-insensitive placement matters for concurrent programs. Another thread

257 write a shared global at any point, so the simplest thread-modular analysis
262 may write a shared global at any point, so the simplest thread-modular

258 keeps shared globals flow-insensitively @seidl26[§1, Ex. 4] @saan26phd[§5.3.1]. More
    generally, mixed flow-sensitivity
263 analysis, which analyzes each thread on its own, keeps shared globals

259 lets an analysis choose per fact whether precision at each program point is
264 flow-insensitively @seidl26[§1, Ex. 4] @saan26phd[§5.3.1].
```

260	worth a value per point and context.		
261	The placement is proved to meet the analysis contract for the analyzer's	265	The placement is proved to meet the analysis contract for the analyzer's
262	product state (#isathm("keyed_split_contract")), so the end-to-end soundness theorem	266	product state (#isathm("keyed_split_contract")), so the end-to-end soundness theorem
263	covers both placements, and a future extension with threads can build on a	267	covers both placements, and a future extension with threads can build on a
264	verified flow-insensitive core.	268	verified flow-insensitive core.

thesis/content/07-cooperation.typ

77e07947 → 0edeac81

374 old lines, 370 new lines

90% similar lines **+34** **-38** **18 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": isaconst, isai, isalocale, isathm, isatype, listing, oblig, thy-badge	3	#import "../lib/code.typ": isaconst, isai, isalocale, isathm, isatype, listing, oblig, thy-badge
4	#import "../lib/math.typ": *	4	#import "../lib/math.typ": *
5	#import "../lib/sources.typ": proved, thy	5	#import "../lib/sources.typ": proved, thy
6	#import "../lib/theorems.typ": definition	6	#import "../lib/theorems.typ": definition
7	#import "../lib/figures.typ": verdict as verdict-chip	7	#import "../lib/figures.typ": verdict as verdict-chip
8	#import "../lib/claims.typ": claim-check, claim-ref, claim-row	8	#import "../lib/claims.typ": claim-check, claim-ref, claim-row
9		9	
10	#let verdict(name, loc) = verdict-chip(claim-row(name, loc).verdict)	10	#let verdict(name, loc) = verdict-chip(claim-row(name, loc).verdict)
11	#let state(name, loc) = absval(raw(claim-row(name, loc).state))	11	#let state(name, loc) = absval(raw(claim-row(name, loc).state))
12		12	
13	= Cooperating Analyses <ch:cooperation>	13	= Cooperating Analyses <ch:cooperation>
14		14	
15	An analysis of @ch:analysis-interface sees only its own state. For some	15	An analysis of @ch:analysis-interface sees only its own state. Some checks,
16	properties a user checks, however, a single analysis is not enough. They can	16	however, can only be proved by combining what several analyses know while
17	only be proved by combining what several analyses know while they run	17	they run together. The program below contains such a check:
18	together. Consider the		
19	example below:		
20		18	
21	#listing(lang: "c", claim: "coop-eq-both", ```	19	#listing(lang: "c", claim: "coop-eq-both", ```
22	fun main() {	20	fun main() {
	15 unchanged lines hidden		
38	#verdict("coop-eq-interval", "16:7") with #state("coop-eq-interval", "16:7").	36	#verdict("coop-eq-interval", "16:7") with #state("coop-eq-interval", "16:7").
39	The order analysis (@sec:relational) is relational. Its state is a set of pairs (x, y), each	37	The order analysis (@sec:relational) is relational. Its state is a set of pairs (x, y), each
40	meaning that the value of x is at most the value of y. After the guards, it	38	meaning that the value of x is at most the value of y. After the guards, it
41	holds (x, y) and (y, x), so it knows that $x = y$. It tracks no values of variables,	39	holds (x, y) and (y, x), so it knows that $x = y$. Since it tracks no values of variables, it
42	however, so it learns nothing about `z` and reports #verdict("coop-eq-order", "16:7") too. Together, the two can	40	learns nothing about `z` and reports #verdict("coop-eq-order", "16:7") too. Together, the two can
43	decide it, if Interval's assignment can learn what	41	decide it, if Interval's assignment can learn what
44	the order analysis knows at that point (@fig:mcp-step).	42	the order analysis knows at that point (@fig:mcp-step).

```
45                                                                  43
                                39 unchanged lines hidden
85    kind: image,                                                  83    kind: image,
86    placement: none,                                              84    placement: none,
87    caption: [One step of Interval and the order analysis at `z = (x == y)` in   85    caption: [One step of Interval and the order analysis at `z = (x == y)` in
88      the program above (schematic). Interval's assignment asks #isaconst("EvalInt") $(x ==   86      the program above (schematic). Interval's assignment asks which values
      y)$. The channel
89      asks the handler of every analysis and gives Interval the meet of their   87      $x == y$ may take (#isaconst("EvalInt") $(x == y)$, @sec:coop-queries). The channel
90      answers. Each analysis updates only its own field         88      asks the query handler of every analysis and gives Interval the meet of their
91      of the product state. The final states match the analyzer run   89      answers. Each analysis updates only its own part of the combined state. The final states
                                                                        match the analyzer run
92      #verdict("coop-eq-both", "16:7") with #state("coop-eq-both", "16:7").],   90      #verdict("coop-eq-both", "16:7") with #state("coop-eq-both", "16:7").],
93    ) <fig:mcp-step>                                              91    ) <fig:mcp-step>
94                                                                  92
95    Goblint solves this with its                                  93    Goblint solves this with its
96    #link(                                                        94    #link(
97      "https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/   95      "https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/
      analyses/mCP.ml",                                             analyses/mCP.ml",
98    )[_master control program_]                                   96    )[_master control program_]
99    (MCP). The selected analyses run side by side, each on its own part of one   97    (MCP). The selected analyses run side by side, each on its own part of one
100   product state, and a transfer may ask all selected analyses a question   98    product state, and a transfer may ask all selected analyses a question
101   through the manager's `ask` function. Goblint has no machine-checked argument that these   99    through the MCP's `ask` function. Goblint has no machine-checked argument
      exchanges are sound. A
102   proof, however, has a modularity problem. If Interval's assignment is proved   100   that these exchanges are sound, and a proof of them has a modularity problem. If Interval's
                                                                        assignment is proved
103   sound using the order analysis's invariant, the two proofs are coupled, every   101   sound using the order analysis's invariant, the two proofs are coupled, every
104   new pair of analyses needs a new proof, and adding an analysis reopens old   102   new pair of analyses needs a new proof, and adding an analysis reopens old
105   ones. This chapter describes how each analysis instead gets one proof   103   ones. Voblint instead gives each analysis one proof obligation that does not
106   obligation that does not mention the analyses it runs with, and why analyses   104   mention the analyses it runs with. Analyses that meet it combine into one
107   meeting it combine into one analysis that meets the analysis contract of   105   analysis that meets the analysis contract of @sec:sound-core.
108   @sec:sound-core.                                              106
109                                                                 107
110   == Queries, answers, and channels #thy-badge("Voblint_Framework", "Analysis_Query")   107   == Queries, answers, and channels #thy-badge("Voblint_Framework", "Analysis_Query")
      <sec:coop-queries>                                            <sec:coop-queries>
111                                                                 108
112   The analyses must agree on what a question means and on what an answer   109   The analyses must agree on what a question means and on what an answer
113   claims. A _query_ asks for a fact about the stores a state describes. Voblint   110   claims. A _query_ asks for a fact about the stores a state describes. Voblint
114   has one kind, named after Goblint's. The query #isaconst("EvalInt") $e$ asks   111   has one kind, named after Goblint's. The query #isaconst("EvalInt") $e$ asks
115   which integers the expression $e$ may evaluate to. Its answer is a value of the reduced   112   which integers the expression $e$ may evaluate to. Its answer is a value of Int
      product Int (#isatype("int_dom"),                             (#isatype("int_dom")), the reduced product of
116   @sec:reduced-product), lifted by a bottom and by a top element that claims nothing   113   the four numeric domains Sign, Interval, Parity, and Congruence
                                                                  114   (@sec:reduced-product), lifted by a bottom and by a top element that claims nothing
117   (#isatype("query_lift")). A                                   115   (#isatype("query_lift")). A
118   comparison evaluates to $0$ or $1$, so the exact answer $1$ to   116   comparison evaluates to $0$ or $1$, so the exact answer $1$ to
```

```
#isaconst("EvalInt") $(x == y)$ says that $x = y$.
#isaconst("EvalInt") $(x == y)$ says that $x = y$.
```

35 unchanged lines hidden

```
and the pointwise meet of two channels that hold at $s$ holds at $s$
(#isathm("query_algebra.channel_holds_inf")).
and the pointwise meet of two channels that hold at $s$ holds at $s$
(#isathm("query_algebra.channel_holds_inf")).

== Proving each analysis against true answers #thy-badge("Voblint_Framework", "MCP_Spec") <sec:coop-channels>
== Soundness against true answers #thy-badge("Voblint_Framework", "MCP_Spec") <sec:coop-channels>

A transfer that uses an answer is sound only if the answer is true.
Interval's assignment in @fig:mcp-step uses the wrapper
#isaconst("ask_assign"), which any analysis can put around its assignment
$x := e$. The wrapper asks #isaconst("EvalInt") $e$. When the answer is one
integer $n$, it runs the analysis's own assignment of the literal $n$, and
otherwise, the assignment of $e$. At $z := (x == y)$, Interval receives $1$ and
sets $z$ to $absval(ivl(1, 1))$, which is right only if the answer is true. Proving
that from the order analysis's invariant would couple the two proofs.
Interval's assignment in @fig:mcp-step uses the wrapper
#isaconst("ask_assign"), which any analysis can put around its assignment
$x := e$. The wrapper asks #isaconst("EvalInt") $e$. When the answer is one
integer $n$, it runs the analysis's own assignment of the literal $n$, and
otherwise, the assignment of $e$. At $z := (x == y)$, Interval receives $1$ and
sets $z$ to $absval(ivl(1, 1))$. Proving the answer true from the order analysis's
invariant would couple the two proofs.

Voblint avoids this coupling by separating two facts, that a transfer uses its
answers correctly and that the answers are true. The transfer
$#isaconst("ls_step") thin c thin italic("ch") thin a thin x$ of an analysis $c$
receives the channel $italic("ch")$ as an extra argument, besides the action
$a$ and the state $x$ before the edge. Its soundness is proved under the
assumption that the channel holds. The transfer must lose no successor, as in
@sec:calls, but only of stores at which every answer is true
(#isathm("ls_step_sound_iff")). Whether the answers are true is not the
transfer's concern, and @sec:coop-mcp shows that the framework always supplies
a channel that holds.
Voblint avoids this coupling by separating two facts, that a transfer uses its
answers correctly and that the answers are true. The transfer
$#isaconst("ls_step") thin c thin italic("ch") thin a thin x$ of an analysis $c$
receives the channel $italic("ch")$ as an extra argument, besides the action
$a$ and the state $x$ before the edge. Its soundness is proved under the
assumption that the channel holds. The transfer must lose no successor, as in
@sec:calls, but only of stores at which every answer is true
(#isathm("ls_step_sound_iff")). @sec:coop-mcp proves the answers true once
for all analyses.

For #isaconst("ask_assign"), this assumption is exactly what the proof needs.
In the example, only the stores with $x = y$ remain, because the answer $1$ is
false at all others. At these stores, $e$ evaluates to $1$, so the analysis's
own assignment of the literal $1$ is sound for $e$ as well
(#isathm("ask_assign_sound")). The proof never mentions the order analysis
and works with any analyses whose answers hold.
For #isaconst("ask_assign"), this assumption is exactly what the proof needs.
In the example, only the stores with $x = y$ remain, because the answer $1$ is
false at all others. At these stores, $e$ evaluates to $1$, so the analysis's
own assignment of the literal $1$ is sound for $e$ as well
(#isathm("ask_assign_sound")). The proof never mentions the order analysis.

An analysis without analysis globals is written as a _local specification_
(#isatype("local_spec")). It bundles the operations of @sec:calls, each taking
a channel, with a handler #isaconst("ls_query") that answers queries about a
state, so that the analysis can answer queries from the analyses it runs with.
An analysis without analysis globals is written as a _local specification_
(#isatype("local_spec")). It bundles the operations of @sec:calls, each taking
a channel, with a handler #isaconst("ls_query") that answers queries about a
state, so that the analysis can answer the queries of the others.

#{
  show raw.where(block: true): set text(size: 6.5pt)
#{
  show raw.where(block: true): set text(size: 6.5pt)
```

14 unchanged lines hidden

207	$conc(x)$ at which its own channel holds (#isaconst("sound_query")), and the	204	$conc(x)$ at which its own channel holds (#isaconst("sound_query")), and the
208	analyses rely on one another only through this law. Verasco proves each	205	analyses rely on one another only through this law. Verasco proves each
209	transfer function under the hypothesis that the channels it receives are	206	transfer function under the hypothesis that the channels it receives are
210	correct @jourdan15[§7], following Astrée @cousot07astree[§§5.3, 6], and Voblint	207	correct @jourdan15[§7], following the static analyzer Astrée @cousot07astree[§§5.3, 6], and Voblint
211	applies the same form of proof to Goblint's queries.	208	applies the same form of proof to Goblint's queries.
212		209	
213	== Combining analyses into one sound analysis #thy-badge("Voblint_Framework", "MCP_Spec") <sec:coop-mcp>	210	== Combination into one sound analysis #thy-badge("Voblint_Framework", "MCP_Spec") <sec:coop-mcp>
214		211	
215	The analyses cannot be solved one after another, because each may need the	212	The analyses cannot be solved one after another, because each may need the
216	other's facts at the same program point and context. In the program below, the	213	other's facts at the same program point and context. In the program below, the
217	exchange runs in both directions, here between the order analysis and Int, the	214	exchange runs in both directions, here between the order analysis and Int.
218	reduced product (@sec:reduced-product).		
219		215	
220	#let _z(name) = claim-check(name, "z == 1")	216	#let _z(name) = claim-check(name, "z == 1")
221	#listing(lang: "c", claim: "coop-mutual-both", ```	217	#listing(lang: "c", claim: "coop-mutual-both", ```
	123 unchanged lines hidden		
345	a new value back, as Goblint's MCP hands each analysis its own part of the	341	a new value back, as Goblint's MCP hands each analysis its own part of the
346	product state. A component therefore leaves the concretizations of all other	342	product state. A component therefore leaves the concretizations of all other
347	fields unchanged (#isathm("field_frame")), which makes the components	343	fields unchanged (#isathm("field_frame")), which makes the components
348	_independent_ (#isaconst("mcp_independent")). #isaconst("mcp_combine") runs the	344	_independent_ (#isaconst("mcp_independent")).
		345	
		346	The combination #isaconst("mcp_combine") runs the
349	components in turn, gives each of them the same channel, and meets the answers of	347	components in turn, gives each of them the same channel, and meets the answers of
350	their handlers. If the list of components is nonempty, the components are	348	their handlers. If the list of components is nonempty, the components are
351	independent, and each is a sound local specification, then the combination is	349	independent, and each is a sound local specification, then the combination is
352	a sound local specification for the intersection of their concretizations	350	a sound local specification for the intersection of their concretizations
353	(#isathm("mcp_combine_sound")). The components cannot invalidate one	351	(#isathm("mcp_combine_sound")). Instantiated with the analyses Voblint ships, the theorem gives the
354	another's guarantees, since each guarantee is about a field no other component		
355	writes. Instantiated with the analyses Voblint ships, the theorem gives the		
356	result of this chapter. Any nonempty list #isai("as") of distinct analyses	352	result of this chapter. Any nonempty list #isai("as") of distinct analyses
357	combines into a sound local specification:	353	combines into a sound local specification:
358		354	
359	#proved("mcp_comp_sound")	355	#proved("mcp_comp_sound")
360		356	
361	Running the combined handler on the current state x gives a channel for the	357	Running the combined handler on the current state x gives a channel for the
362	premise of the theorem (#isaconst("ls_channel")). A query that comes back while	358	premise of the theorem (#isaconst("ls_channel")). A query that is asked again
363	it is being answered gets $ltop$, as in Goblint's `MCP.query`. Every answer of	359	while its answer is being computed gets $ltop$, as in Goblint's `MCP.query`. Every answer of
364	this channel holds at every store of $conc(x)$ (#isathm("ls_channel_sound")),	360	this channel holds at every store of $conc(x)$ (#isathm("ls_channel_sound")),
365	so #isaconst("dg_spec_of") can pass it to the operations and turn the	361	so #isaconst("dg_spec_of") can pass it to the operations and turn the
366	combination into a record that meets the analysis contract	362	combination into a record that meets the analysis contract

367	(#isathm("dg_spec_of_contract")).	363	(#isathm("dg_spec_of_contract")).
368		364	
369	Each analysis is thus proved sound once, against any channel that holds, and	365	Each analysis is thus proved sound once, against any channel that holds, and
370	its proof never mentions the analyses it runs with. Their combination meets the	366	its proof never mentions the analyses it runs with. Their combination meets the
371	analysis contract and decides the check of the opening example. Adding an	367	analysis contract and decides the check of the opening example. Adding an
372	analysis requires only its own proof, and the proofs of the others stay	368	analysis requires only its own proof, and the proofs of the others stay
373	unchanged. @ch:equations discharges the obligations of @ch:traces for any	369	unchanged. @ch:equations discharges the obligations of @ch:traces for any
374	analysis that meets the analysis contract, the combination included.	370	analysis that meets the analysis contract, the combination included.

thesis/content/08-equations.typ

77e07947 → 0edeac81

519 old lines, 519 new lines

95% similar lines **+28** **-28** **11 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": (	3	#import "../lib/code.typ": (
	10 unchanged lines hidden		
14		14	
15	= Context-Indexed Equations and Routing <ch:equations>	15	= Context-Indexed Equations and Routing <ch:equations>
16		16	
17	@ch:traces reduced soundness to a cover, which assigns to every node v	17	@ch:traces reduced soundness to a cover, a set of stores #isai("cover v c") for every node v and context c, and stated five local
18	and context c a set of stores #isai("cover v c"), and stated five local		
19	obligations under which the cover contains every store that an execution	18	obligations under which the cover contains every store that an execution
20	brings to v in context c. @ch:analysis-interface and @ch:cooperation stated what an analysis supplies	19	brings to v in context c. @ch:analysis-interface and @ch:cooperation stated what an analysis supplies
21	and proves. What is still missing is a way to compute such a cover.	20	and proves. A way to compute such a cover is still missing.
22	This chapter turns the procedure-aware graph of @ch:program-model into a	21	This chapter turns the procedure-aware graph of @ch:program-model into a
23	system of equations and shows that every post-solution induces a sound cover.	22	system of equations and shows that every post-solution induces a sound cover.
24		23	
25	Precisely, every post-solution sol of the equations should satisfy	24	Precisely, every post-solution sol of the equations should satisfy
26	$	25	$
27	#isai("\<A>\<^bsub>\<G>,adm,c_0,g,S\<^esub> v c") subset.eq conc_(M)(sol(v, c))	26	#isai("\<A>\<^bsub>\<G>,adm,c_0,g,S\<^esub> v c") subset.eq conc_(M)(sol(v, c))
28	$	27	$
29	at every node v and context c. The left side is the activation collecting	28	at every node v and context c. The left side is the activation collecting
30	semantics of @sec:contexts, and $conc_(M)(sol(v, c))$ is the set of stores	29	semantics of @sec:contexts, and $conc_(M)(sol(v, c))$ is the set of stores
31	that the solved abstract state at (v, c) describes under the solved values	30	that the solved abstract state at (v, c) describes under the solved values
32	of the analysis globals (@sec:sound-core). For the edges inside a procedure,	31	of the analysis globals (@sec:analysis-globals). For the edges inside a procedure,
33	the equations follow the graph directly. A call is harder, since the equations	32	the equations follow the graph directly. A call is harder, since the equations
34	must also determine which context of the callee it enters. Under entry-state	33	must also determine which context of the callee it enters. Under entry-state
35	policies, this context is computed from the abstract state at the call, which	34	policies, this context is computed from the abstract state at the call, which
36	is known only while the equations are being solved.	35	is known only while the equations are being solved.
37		36	

38	== Context-indexed unknowns and call equations #thy-badge("Voblint_Framework", "DG_Indexed_Generator") <sec:eq-call>	37	== Context-indexed unknowns and call equations #thy-badge("Voblint_Framework", "DG_Indexed_Generator") <sec:eq-call>
39		38	
40	Separate contexts give the activations of `bump(5)` and `bump(4)` separate	39	The equations must keep the activations of `bump(5)` and `bump(4)` apart, so
41	copies of `bump`, so the first result stays exactly 6 (@sec:interproc).	40	that the first call's result stays exactly 6 (@sec:interproc). Voblint
42		41	therefore indexes the unknowns of the equations by contexts. For every
43	Voblint therefore indexes the unknowns of the equations by contexts. For every		
44	node v and context c, there is a _local_ unknown (v, c), whose value	42	node v and context c, there is a _local_ unknown (v, c), whose value
45	$sol(v, c)$ is the abstract state for the activations of context c at v.	43	$sol(v, c)$ is the abstract state for the activations of context c at v.
46	These are the indices of the cover. Goblint's local unknowns have the	44	The pairs (v, c) are the indices of the cover. Goblint's local unknowns have the
47	#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/framework/analyses.ml#L26-L28")[same shape] @apinis14[§5.2].	45	#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/framework/analyses.ml#L26-L28")[same shape] @apinis14[§5.2].
48	Besides the local unknowns, the equations have _global_ unknowns without a	46	Besides the local unknowns, the equations have _global_ unknowns without a
49	program point (@sec:td). They hold the analysis globals of @sec:shared-facts	47	program point (@sec:side-effects). They hold the analysis globals of @sec:shared-facts
50	and the entry states that calls publish to their callees.	48	and the entry states that calls publish to their callees.
51		49	
52	The equation of (v, c) has to collect every abstract state that can reach	50	The equation of (v, c) has to collect every abstract state that can reach
	28 unchanged lines hidden		
81		79	
82	At a call, the analyzer chooses the callee's context with a _routing_ function	80	At a call, the analyzer chooses the callee's context with a _routing_ function
83	$ctxh(u, c, e)$ of the call site u, the caller context c, and the entry	81	$ctxh(u, c, e)$ of the call site u, the caller context c, and the entry
84	state e. Under entry-state contexts, the choice depends on e, which is	82	state e. Under entry-state contexts, the choice depends on e, so the
85	known only while the equations are solved. The set of calls that contribute to	83	calls that contribute to a callee context become known only during solving,
86	a callee context is therefore not known in advance, and the	84	as the opening of this chapter noted. The equation of the callee's entry
87	callee's entry cannot list them in its equation. Each call instead publishes	85	therefore cannot list them. Each call instead publishes its entry state to the
88	its entry state to the callee as a side effect (@sec:side-effects).	86	callee as a side effect (@sec:side-effects).
89		87	
90	#let _loc(pos, body) = node(	88	#let _loc(pos, body) = node(
91	pos,	89	pos,
	107 unchanged lines hidden		
199	$c_2 = [[4, 4]]$. The two calls thus use separate copies of `bump`, and the	197	$c_2 = [[4, 4]]$. The two calls thus use separate copies of `bump`, and the
200	first result stays exactly 6.	198	first result stays exactly 6.
201		199	
202	In this protocol, the caller's equation publishes the entry state to the callee	200	The callee's entry $(ctor("FunctionEntry") thin p, c')$ is a local unknown,
203	and reads the callee's exit state. The verified TD solver, however, accepts side effects	201	and the verified TD solver accepts side effects only to global unknowns
204	only to global unknowns (@sec:eq-seed-global), so a call cannot publish to the	202	(@sec:side-effects, @sec:eq-seed-global). Each callee entry p and context
205	local unknown of the callee's entry directly. Each callee entry p and context		
206	c' therefore has a global unknown, the _seed_	203	c' therefore has a global unknown, the _seed_
207	$ctor("Activation_Seed") thin p space c'$ (#isatype("global_unknown")). Every	204	$ctor("Activation_Seed") thin p space c'$ (#isatype("global_unknown")). Every
208	call routed to (p, c') publishes to it, and the equation of	205	call routed to (p, c') publishes to it, and the equation of
	18 unchanged lines hidden		
227	classified as bottom is required to have an empty concretization	224	classified as bottom is required to have an empty concretization
228	(#isathm("routed_context.is_bot_sound", thy: "Routed_Context", display: "is_bot_sound")).	225	(#isathm("routed_context.is_bot_sound", thy: "Routed_Context", display: "is_bot_sound")).
229		226	

230	== Routing that agrees with the context policy #thy-badge("Voblint_Framework", "Routed_Context") <sec:eq-routing>	227	== Routing consistent with the context policy #thy-badge("Voblint_Framework", "Routed_Context") <sec:eq-routing>
231		228	
232	Every call is assigned a callee context twice. The equations choose one from	229	Every call is assigned a callee context twice. The equations choose one from
233	the abstract entry state, and the concrete semantics of @ch:traces chooses one	230	the abstract entry state, and the concrete semantics of @ch:traces chooses one
	242 unchanged lines hidden		
476	(@sec:calls) guarantees such an entry pair for every call from a store that the	473	(@sec:calls) guarantees such an entry pair for every call from a store that the
477	caller's solved state describes, which is totality.	474	caller's solved state describes, which is totality.
478		475	
479	== Post-solutions satisfy the coverage contract #thy-badge("Voblint_Framework", "Routed_Context") <sec:eq-discharge>	476	== Coverage by post-solutions #thy-badge("Voblint_Framework", "Routed_Context") <sec:eq-discharge>
480		477	
481	#figure(	478	#figure(
482	table(	479	table(
	15 unchanged lines hidden		
498	@ch:traces, for an arbitrary program.],	495	@ch:traces, for an arbitrary program.],
499	) <tab:eq-obligations>	496	) <tab:eq-obligations>
500		497	
501	Each obligation of @ch:traces is met by one part of the generated equations	498	With the equations and their routing fixed, it remains to show that every
502	(@tab:eq-obligations). The argument holds for every	499	post-solution induces a cover that meets the five obligations of @ch:traces.
503	domain and context policy. Its locale	500	Each obligation is met by one part of the generated equations
504	#isalocale("routed_context") assumes a post-solution sol on a solved set	501	(@tab:eq-obligations), for every domain and context policy. The locale
505	closed under local edges and call continuations, the	502	#isalocale("routed_context") of this argument assumes a post-solution sol on
506	#isalocale("analysis_contract") of @ch:analysis-interface, a sound emptiness test,	503	a solved set closed under local edges and call continuations, and the
507	a sound callee resolution, and adequate and total routing. The theorem adds that the program entry	504	#isalocale("analysis_contract") of @ch:analysis-interface. It further assumes
		505	that a state classified as bottom describes no store (@sec:eq-call) and that
		506	routing is adequate and total. The resolved callees of a call must include its
		507	callee from every store that the caller's solved state describes. The theorem adds that the program entry
508	is solved in the initial context and that the initial state d_0 describes the initial stores S_0.	508	is solved in the initial context and that the initial state d_0 describes the initial stores S_0.
509	#theorem(name: [Routed collecting soundness], isa: "activation_collect_dg_sound")[	509	#theorem(name: [Routed collecting soundness], isa: "activation_collect_dg_sound")[
510	#proved("activation_collect_dg_sound")	510	#proved("activation_collect_dg_sound")
511	]	511	]
512	The solver gives guarantees only for the unknowns in its solved set, so the	512	The solver gives guarantees only for the unknowns in its solved set, so the
513	cover #isai("cover v c") is $conc_(M)(sol(v, c))$ there and empty elsewhere. No	513	cover #isai("cover v c") is $conc_(M)(sol(v, c))$ there and empty elsewhere. No
514	execution reaches an unknown outside the solved set, since the set contains the	514	execution reaches an unknown outside the solved set, since the set contains the
515	program entry and is closed under local edges, call continuations, and routed	515	program entry and is closed under local edges, call continuations, and routed
516	calls. The proof checks each obligation for this cover, one row of	516	calls. The proof checks each obligation for this cover, one row of
517	@tab:eq-obligations each. Instantiated for the policies of @tab:eq-policies,	517	@tab:eq-obligations each. Instantiated for the policies of @tab:eq-policies,
518	the theorem covers the analyzer's equations, and @ch:solving computes such a	518	the theorem covers the analyzer's equations, and @ch:solving computes such a
519	post-solution.	519	post-solution.

thesis/content/09-solving.typ

77e07947 → 0edeac81

352 old lines, 349 new lines

85% similar lines **+51** **-54** **22 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "@preview/cetz:0.5.2"	2	#import "@preview/cetz:0.5.2"
3	#import "../lib/code.typ": fixture, isaconst, isai, isalocale, isathm, isatype, listing, thy-badge	3	#import "../lib/code.typ": fixture, isaconst, isai, isalocale, isathm, isatype, listing, thy-badge
4	#import "../lib/sources.typ": thy, update-rule-steps	4	#import "../lib/sources.typ": thy, update-rule-steps
5	#import "../lib/math.typ": *	5	#import "../lib/math.typ": *
6	#import "../lib/theme.typ": vb	6	#import "../lib/theme.typ": vb
7	#import "../lib/claims.typ": claim-check, claim-ref, claim-snapshot, claim-text	7	#import "../lib/claims.typ": claim-check, claim-ref, claim-snapshot, claim-text
8	#import "../lib/figures.typ": verdict	8	#import "../lib/figures.typ": verdict
9		9	
10	#let _b = $italic("bump")$	10	#let _b = $italic("bump")$
11		11	
12	= Solving the Equations <ch:solving>	12	= Post-Solutions from the Verified Solver <ch:solving>
13		13	
14	@ch:equations showed that a post-solution of the generated equations is	14	@ch:equations showed that a post-solution of the generated equations is
15	sound. At every node and context, its abstract state describes every store that	15	sound. At every node and context, its abstract state describes every store that
16	an execution reaches there in that context	16	an execution reaches there in that context
17	(#isathm("activation_collect_dg_sound")). The solver, however, makes the	17	(#isathm("activation_collect_dg_sound")). The solver, however, makes the
18	valuation a post-solution only on its solved set, so the theorem requires this	18	valuation a post-solution only on its solved set, so the theorem requires this
19	set to contain the program entry and every unknown that an execution can reach	19	set to contain the program entry and every unknown that an execution can reach
20	from it. This chapter shows that the analyzer obtains such a solved set and	20	from it. This chapter shows that every solve that returns provides such a
21	post-solution. Voblint solves the equations with the	21	solved set and post-solution. Voblint keeps a copy of the theories of the
22	verified TD solver of Tilscher et al. @tilscher26, which is proved	22	verified TD solver of @sec:td in its repository, called the _vendored_ solver
23	partially correct. To use it, Voblint first brings the equations into the form the	23	below. Voblint first
24	solver accepts and then derives the premises about the solved set from the	24	brings the equations into the form this solver accepts and then derives the
25	solver's correctness theorem.	25	premises about the solved set from the solver's partial-correctness theorem.
26		26	
27	== Encoding the equations for the solver <sec:eq-encoding>	27	== Solver encoding of the equations <sec:eq-encoding>
28		28	
29	The solver accepts a right-hand side only as a strategy tree, uses one value	29	The vendored solver accepts a right-hand side only as a strategy tree, uses one
30	type for all unknowns, and allows side effects only to global unknowns. A direct encoding of the equations, as in Apinis et al.	30	value type for all unknowns, and allows side effects only to global unknowns. A
31	and Goblint, needs four things this interface does not offer	31	direct encoding of the equations, as in Apinis et al. @apinis12 and Goblint,
		32	needs features this interface lacks. Voblint closes each gap with an
		33	adapter on the equations and leaves the solver and its proof as they are
32	(@tab:eq-adapters). @ch:equations already stated the equations in the adapted	34	(@tab:eq-adapters). @ch:equations already stated the equations in the adapted
33	form, and this section explains why each adaptation is needed. Buffering is the	35	form, and this section explains why each adaptation is needed. Buffering is the
34	only adapter applied to a right-hand side after it is built. It changes when	36	only adapter applied to a right-hand side after it is built. It changes only
35	publications are issued but not what the right-hand side computes, publishes,	37	the time at which publications are issued. The right-hand side computes, publishes,

Old	77e07947	New	0edeac81
36	or reads (@sec:update-rules). The adapters change the equations, not the solver.	38	and reads the same values as before (@sec:update-rules).
37		39	
38	#figure(	40	#figure(
39	table(	41	table(
	25 unchanged lines hidden		
65		67	
66	A local edge always reads the same unknown, its predecessor. A call, in	68	A local edge always reads the same unknown, its predecessor. A call, in
67	contrast, chooses what to read from values it has already read. In the running	69	contrast, chooses what to read from values it has already read. In the running
68	example, the continuation $(italic("pp3"), c_0)$ first reads the caller	70	example (@fig:eq-unknowns), the continuation $(italic("pp3"), c_0)$ first
69	$(italic("pp2"), c_0)$. From that value, it computes the entry state	71	reads the caller $(italic("pp2"), c_0)$. From that value, it computes the entry
70	$n = absval(ivl(5, 5))$, which determines the context c_1 and thus the result it reads	72	state, which binds n to $absval(ivl(5, 5))$. This state determines the context
71	next, $(ctor("FunctionResult") thin italic("bump"), c_1)$.	73	c_1 and thus the result it reads next,
		74	$(ctor("FunctionResult") thin italic("bump"), c_1)$.
72	The vendored TD solver accepts a right-hand side only as a strategy tree, which	75	The vendored TD solver accepts a right-hand side only as a strategy tree, which
73	issues one read at a time and passes each value read to a continuation	76	issues one read at a time and passes each value read to a continuation
74	(@sec:td). Every right-hand side of Voblint is such a tree. For a local edge,	77	(@sec:td). Every right-hand side of Voblint is such a tree. For a local edge,
75	the tree reads the predecessor and applies the transfer. For a call it chains reads whose	78	the tree reads the predecessor and applies the transfer. For a call it chains reads whose
76	continuations choose the next read from the value received. In the notation of	79	continuations choose the next read from the value received. In the notation of
77	@sec:td, the value of	80	@sec:td, the value of
78	a call at u in context c (#isaconst("rhs_call")) is the value of the tree	81	a call at u in context c (#isaconst("rhs_call")) is the value of the tree
79	$t_"call"$,	82	$t_"call"$,
80	$	83	$
81	t_"call" = #ctor("QueryL") (& (u, c), lambda d. \	84	t_"call" = #ctor("QueryL") (& (u, c), lambda d. \
82	& #ctor("Side") (ctor("Activation_Seed") thin p space c', e, \	85	& #ctor("Side") (ctor("Activation_Seed") thin p space c', e, \
83	& quad #ctor("QueryL") ((ctor("FunctionResult") thin p, c'), lambda r.	86	& quad #ctor("QueryL") ((ctor("FunctionResult") thin p, c'), lambda r.
84	#ctor("Answer", thy: "Basics_side") (sh("combine") (q, r))))),	87	#ctor("Answer", thy: "Basics_side") (sh("combine") (q, r))))).
85	$	88	$
86	The tree first reads the caller's state d at the call site and computes the	89	The tree first reads the caller's state d at the call site and computes the
87	entry pair $(q, e) = enterh(d)$ and the callee context $c' = ctxh(u, c, e)$.	90	entry pair $(q, e) = enterh(d)$ and the callee context $c' = ctxh(u, c, e)$.
88	It then publishes the entry state e for the callee's entry in c', reads	91	It then publishes the entry state e for the callee's entry in c', reads
89	the callee's exit state r, and answers $sh("combine")(q, r)$, the abstract state	92	the callee's exit state r, and answers $sh("combine")(q, r)$, the abstract state
90	of the caller once the call has returned. #isaconst("routed_call_program") implements	93	of the caller once the call has returned. #isaconst("routed_call_program") implements
91	this tree and additionally handles bottom entry states and several callees or	94	this tree and additionally handles bottom entry states and several callees or
92	entry pairs.	95	entry pairs.
93		96	
94	=== Publishing entry states to global seeds <sec:eq-seed-global>	97	=== Entry states as global seeds <sec:eq-seed-global>
95		98	
96	In $t_"call"$, the caller pushes the entry state to the callee with a	99	In $t_"call"$, the #ctor("Side") step publishes the entry state to the seed of
97	#ctor("Side") step. Otherwise, the equation of the callee's entry would have	100	the callee in c', the global unknown that the callee's entry reads
98	to read every call routed to it. Under entry-state contexts, however, these	101	(@sec:eq-call). Apinis et al. push the entry state directly into the callee's

99	calls depend on the entry states, which are known only while the equations are	102	entry $(ctor("FunctionEntry") thin p, c')$ @apinis12[§6], and Goblint does the
100	solved (@sec:eq-call).	103	same with its local side effect `sidel`
101	Apinis et al. push the entry state directly into the callee's entry		
102	$(ctor("FunctionEntry") thin p, c')$ @apinis12[§6], and Goblint does the same		
103	with its local side effect `sidel`		
104	(#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/framework/constraints.ml#L244")[`constraints.ml`]).	104	(#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/framework/constraints.ml#L244")[`constraints.ml`]).
105	That entry, however, is a local unknown, and the vendored TD solver's	105	That entry, however, is a local unknown, and the vendored TD solver's
106	#ctor("Side") accepts only global unknowns. Allowing local targets would mean	106	#ctor("Side") accepts only global unknowns. Allowing local targets would mean
107	changing the solver and redoing its correctness proof.	107	changing the solver and redoing its correctness proof, so Voblint adds the seed
		108	as an intermediate global unknown.
108		109	
109	Voblint therefore publishes the entry state to the callee's seed, a global	110	Compared with a side effect into the local entry, a seed can lose precision.
110	unknown that the entry reads (@sec:eq-call).	111	As global unknowns, seeds are merged by the update rule for globals
111			
112	Seeds have a known disadvantage compared with a side effect into the local		
113	entry. As global unknowns, they are merged by the update rule for globals		
114	(@sec:update-rules), which may widen an entry state even when no loop or	112	(@sec:update-rules), which may widen an entry state even when no loop or
115	recursion is involved. Without contexts, both calls of the running example	113	recursion is involved. Without contexts, both calls of the running example
116	publish to the one seed of `bump`, and warrowing widens it. The analyzer then	114	publish to the one seed of `bump`, and warrowing widens it. The analyzer then
117	reports #verdict(claim-check("pg-contexts-none", "a == 6").at(3)) for `a == 6` with	115	reports #verdict(claim-check("pg-contexts-none", "a == 6").at(3)) for `a == 6` with
118	#absval(raw(claim-check("pg-contexts-none", "a == 6").at(4))), whose lower bound $-infinity$ comes	116	#absval(raw(claim-check("pg-contexts-none", "a == 6").at(4))), whose lower bound $-infinity$ comes
119	from this widening.	117	from this widening.
120		118	
121	=== Local and global values in one product #thy-badge("Voblint_Framework", "DG_State") <sec:global-unknowns>	119	=== Local and global values in one product #thy-badge("Voblint_Framework", "DG_State") <sec:global-unknowns>
122		120	
123	A local unknown holds an abstract state of the analysis's local domain, and an	121	A local unknown holds an abstract state of the analysis's local domain, and an
124	analysis global holds a value of its global domain (@sec:shared-facts). The	122	analysis global holds a value of its global domain (@sec:shared-facts). The
125	vendored TD solver, however, has a single value type $'d$ for all unknowns, so Voblint	123	vendored TD solver, however, has a single value type $'d$ for all unknowns, so Voblint
126	stores both kinds in their product: every unknown holds a pair #isatype("dg_state") of	124	stores both kinds in their product. Every unknown holds a pair #isatype("dg_state") of
127	a local and a global half, #isaconst("dg_local") and #isaconst("dg_global"),	125	a local and a global half, #isaconst("dg_local") and #isaconst("dg_global"),
128	uses one half and leaves the other at $lbot$ (@tab:eq-carrier). Order and join	126	uses one half and leaves the other at $lbot$ (@tab:eq-carrier). Order and join
129	on pairs work componentwise, so the solver's requirements on the value type	127	on pairs work componentwise, so the solver's requirements on the value type
130	follow from those on the two halves. The half an unknown uses depends on what it stores, not on	128	follow from those on the two halves. The half an unknown uses depends on what
131	whether the solver treats it as global. A seed, for example, is a global	129	it stores. A seed, for example, is a global unknown, because calls publish to
132	unknown, because calls publish to it, but it stores an entry state, which is a	130	it, but it stores an entry state and therefore uses the local half.
133	local state.		
134		131	
135	#figure(	132	#figure(
136	table(	133	table(

13 unchanged lines hidden

```
    global unknowns are named by #isatype("global_unknown").],
) <tab:eq-carrier>

Transfers never see the pair or the seeds. Goblint instead combines the two kinds of value in a
#link(
  "https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/constraint/translators.ml#L30-L32",
)[lifted sum]
in which each unknown holds a value of exactly one of the two kinds.
Voblint uses the product because its componentwise lattice structure keeps
the Isabelle proofs simple, at the price of an unused #lbot half in every
unknown.

== Update rules for global unknowns #thy-badge("TD", "Update_rules") <sec:update-rules>

A global unknown, such as a seed, receives contributions from several places.
The _update rule_ merges each new contribution into the value the unknown
holds and decides both precision and termination. Tilscher et al. state the
solver's requirements on an update rule as the locale
#isalocale("update_rule") and prove that five rules satisfy it @tilscher26,
so Voblint can offer all five as a user setting
(#isathm("update_rule_update_global_of")). @fig:update-rules compares them.

The rules differ in what they keep per _origin_, the right-hand side that
published a contribution (@sec:td), and in whether they widen. Of the two
_joining rules_, _join_ joins every contribution into the value, and _join per
origin_ joins each origin's latest contribution. Of the three _warrowing rules_,
```

92 unchanged lines hidden

```
unknowns that contains the program entry and is closed under execution steps.
A solve that returns provides something different. It gives a valuation #sol
and a set $V$ of local unknowns on which #sol is a post-solution, and $V$ is
closed under the reads of right-hand sides, starting from the result of
`main`. The solver thus works backward from that result, whereas the theorem
follows executions forward from the entry. The two directions meet because
every node of a procedure reaches the procedure's result, even code after a
```

11 unchanged lines hidden

```
given as relations. The final theorem therefore needs no premise about which
unknowns the solve visited.

The solver is not proved to terminate. HOL admits only terminating recursion,
```

```
    global unknowns are named by #isatype("global_unknown").],
) <tab:eq-carrier>

The pair and the seeds belong to the encoding, and the transfers of an analysis
work on its local and global domains directly. Goblint instead combines the two kinds of value in a
#link(
  "https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/constraint/translators.ml#L30-L32",
)[lifted sum]
in which each unknown holds a value of exactly one of the two kinds.
Voblint uses the product because its componentwise lattice structure keeps the Isabelle proofs simple, although every unknown then carries an unused #lbot half.

== Update rules for global unknowns #thy-badge("TD", "Update_rules") <sec:update-rules>

A global unknown, such as a seed, receives contributions from several places.
Its update rule (@sec:td) merges each new contribution into the value the
unknown holds and thereby decides both precision and termination. Tilscher et
al. state the solver's requirements on an update rule as the locale
#isalocale("update_rule") and prove that five rules satisfy it @tilscher26,
so the analyzer offers all five as a setting
(#isathm("update_rule_update_global_of")). @fig:update-rules compares them.

The rules differ in what they keep per origin, the right-hand side that
published a contribution (@sec:td), and in whether they widen. Of the two
_joining rules_, _join_ joins every contribution into the value, and _join per
origin_ joins each origin's latest contribution. Of the three _warrowing rules_,
```

92 unchanged lines hidden

```
unknowns that contains the program entry and is closed under execution steps.
A solve that returns provides something different. It gives a valuation #sol
and a set $V$ of local unknowns on which #sol is a post-solution, and $V$ is
closed under local reads of right-hand sides, starting from the result of
`main`. The solver thus works backward from that result, whereas the theorem
follows executions forward from the entry. The two directions meet because
every node of a procedure reaches the procedure's result, even code after a
```

11 unchanged lines hidden

```
given as relations. The final theorem therefore needs no premise about which
unknowns the solve visited.

This argument applies to a solve that returns, and the vendored solver has no
```

		286	termination proof (@sec:td). HOL admits only terminating recursion,
290	so Isabelle defines the solver with a domain predicate	287	so Isabelle defines the solver with a domain predicate
291	(#isaconst("solve_dom", thy: "TD_side_upd_rule"), @sec:isabelle), the	288	(#isaconst("solve_dom", thy: "TD_side_upd_rule"), @sec:isabelle), the
292	arguments on which its recursion terminates, and proves its correctness on that	289	arguments on which its recursion terminates, and proves its correctness on that
	12 unchanged lines hidden		
305	not apply to Voblint (@sec:rel-solvers), so the end-to-end soundness theorem	302	not apply to Voblint (@sec:rel-solvers), so the end-to-end soundness theorem
306	(@sec:headline) is a partial-correctness result about every returned report.	303	(@sec:headline) is a partial-correctness result about every returned report.
307		304	
308	The rest of the soundness proof never looks inside the solver. It needs the	305	The rest of the soundness proof uses two properties of the solver. The
309	returned valuation to be a post-solution on the solved set, which is the	306	returned valuation is a post-solution on the solved set, which is the
310	certificate below, and the solved set to be finite, so that	307	certificate below, and the solved set is finite, so that
311	@ch:results can join the finitely many contexts solved at a node into one	308	@ch:results can join the finitely many contexts solved at a node into one
312	verdict. Voblint therefore keeps the solver as a parameter of the analysis.	309	verdict. Voblint therefore keeps the solver as a parameter of the analysis.
313	The locale #isalocale("certified_solver") states what the proof needs from a	310	The locale #isalocale("certified_solver") states what the proof needs from a
	10 unchanged lines hidden		
324		321	
325	T (#isatype("eqsT", thy: "Basics_side")) maps each unknown u to its	322	T (#isatype("eqsT", thy: "Basics_side")) maps each unknown u to its
326	strategy tree $T med u$. Evaluating a tree under the final valuation #sol	323	strategy tree $T med u$. Evaluating a tree under the final valuation #sol
327	determines three things: the answer it returns, the local unknowns it reads,	324	determines the answer it returns, the local unknowns it reads,
328	and the contributions it publishes. In Isabelle these are	325	and the contributions it publishes. In Isabelle these are
329	$#isaconst("eq", thy: "Basics_side") med T med u med sol$,	326	$#isaconst("eq", thy: "Basics_side") med T med u med sol$,
330	$#isaconst("dep\<^sub>L") med T med sol med u$, and	327	$#isaconst("dep\<^sub>L") med T med sol med u$, and
	19 unchanged lines hidden		
350	(#isathm("entry_state_activation_collect_sound_of_terminates")), and the five	347	(#isathm("entry_state_activation_collect_sound_of_terminates")), and the five
351	obligations of @ch:traces hold as @tab:eq-obligations shows. @ch:results turns	348	obligations of @ch:traces hold as @tab:eq-obligations shows. @ch:results turns
352	these states into verdicts.	349	these states into verdicts.

thesis/content/10-results.typ

77e07947 → 0edeac81

442 old lines, 436 new lines

91% similar lines **+35** **-41** **19 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/cetz:0.5.2"	1	#import "@preview/cetz:0.5.2"
2	#import "../lib/code.typ": (	2	#import "../lib/code.typ": (
3	claim-playground-link, isaconst, isai, isalocale, isathm, isatype, listing, oblig,	3	claim-playground-link, isaconst, isai, isalocale, isathm, isatype, listing, oblig,
	19 unchanged lines hidden		
23		23	
24	= From Solutions to Verdicts <ch:results>	24	= From Solutions to Verdicts <ch:results>
25		25	
26	This chapter derives the guarantees of the analyzer's report for the	26	This chapter answers what the analyzer's report guarantees about the
27	executions of the source program. Solving the equations of @ch:equations	27	executions of the source program. Solving the equations of @ch:equations
28	(@ch:solving) gives an abstract state for each CFG node and context the solver	28	(@ch:solving) gives an abstract state for each CFG node and context the solver
29	reached, as well as a value for each global unknown. By @sec:certificate, these states cover every	29	reached, as well as a value for each global unknown. By @sec:certificate, these states cover every

30	execution. A user, however, reads neither the states nor the contexts. The user reads	30	execution. A user of the analyzer, however, reads neither the states nor the contexts. The user reads
31	a report with one verdict for every `__voblint_check` and a diagnostic wherever	31	a report with one verdict for every `__voblint_check` and a diagnostic wherever
32	a division may fail. The chapter shows how the analyzer derives this report	32	a division may fail. The chapter shows how the analyzer derives this report
33	from the solution and why every verdict in it is sound.	33	from the solution. Its main result, a theorem about the analyzer function
		34	itself, states that every store a finite source execution reaches at a check
		35	satisfies the definite verdicts there.
34		36	
35	== The analyzer function and its report #thy-badge("Voblint_CLI", "Analysis_Run") <sec:report>	37	== The analyzer function and its report #thy-badge("Voblint_CLI", "Analysis_Run") <sec:report>
36		38	
	45 unchanged lines hidden		
82	of #isai("\<R>\<^bsub>res\<^esub> v"). @sec:chain shows that this set contains every	84	of #isai("\<R>\<^bsub>res\<^esub> v"). @sec:chain shows that this set contains every
83	store that an activation in context c reaches at v.	85	store that an activation in context c reaches at v.
84		86	
85	== Computing check verdicts from the report <sec:verdicts>	87	== Check verdicts from the report <sec:verdicts>
86		88	
87	The command-line tool prints the report through its checks: each check at v	89	The command-line tool prints the report through its checks: each check at v
88	receives one of four verdicts, #verdict("PROVED"), #verdict("REFUTED"),	90	receives one of four verdicts, #verdict("PROVED"), #verdict("REFUTED"),
89	#verdict("UNKNOWN"), or #verdict("DEAD"). The	91	#verdict("UNKNOWN"), or #verdict("DEAD"). The
90	#link(playground-base)[browser playground] shows the same verdicts inline in	92	#link(playground-base)[browser playground] shows the same verdicts inline in
91	the editor and beside the graph, together with the state of every node and context, and a replay of the solve (@sec:ocaml-boundary, @sec:tracing). Each	93	the editor and beside the graph (@sec:ocaml-boundary). Each
92	verdict is printed at the source position of its check, and the text report checks that these positions are unique (#isathm("run_voblint_labelled_check_sound")).	94	verdict is printed at the source position of its check, and the text report checks that these positions are unique (#isathm("run_voblint_labelled_check_sound")).
93		95	
94	The analyzer first computes one verdict per state. Each state d in	96	The analyzer first computes one verdict per state. Each state d in
	9 unchanged lines hidden		
104	holds in every store the state describes (#isathm("mcp_classify_proved"),	106	holds in every store the state describes (#isathm("mcp_classify_proved"),
105	#isathm("mcp_classify_refuted")).	107	#isathm("mcp_classify_refuted")).
106		108	
107	The verdict of the CFG node soundly joins the per-context verdicts of its states	109	The analyzer then joins the per-context verdicts of a node's states into one
108	(#isaconst("aggregate_verdicts")). If every state other than #ctor("Bot")	110	verdict for the node (#isaconst("aggregate_verdicts")). If every state other than #ctor("Bot")
109	gives #verdict("PROVED"), the node is #verdict("PROVED"). If every such state	111	gives #verdict("PROVED"), the node is #verdict("PROVED"). If every such state
110	gives #verdict("REFUTED"), it is #verdict("REFUTED"). If the states disagree,	112	gives #verdict("REFUTED"), it is #verdict("REFUTED"). If the states disagree,
111	or one of them gives #verdict("UNKNOWN"), the node is #verdict("UNKNOWN").	113	or one of them gives #verdict("UNKNOWN"), the node is #verdict("UNKNOWN").
112	Disagreement cannot be resolved otherwise. If one context proves `x > 0` and	114	For disagreeing states, #verdict("UNKNOWN") is the only sound choice. If one context proves `x > 0` and
113	another refutes it, the stores reaching v in the first satisfy the condition	115	another refutes it, the stores reaching v in the first satisfy the condition
114	and those in the second violate it, so no single definite verdict holds for	116	and those in the second violate it, so no single definite verdict holds for
115	every store at the node.	117	every store at the node.
116	States classified #verdict("DEAD"), the #ctor("Bot") states, are neutral in	118	States classified #verdict("DEAD"), the #ctor("Bot") states, are neutral in
117	this aggregation, so a node at which the report holds no state, or only	119	this aggregation, so a node at which the report holds no state, or only

118	#ctor("Bot") states, is #verdict("DEAD").	120	#ctor("Bot") states, is #verdict("DEAD").
119		121	
120	The chain of @sec:chain has to end in	122	A user reads the verdicts rather than the states. To state their correctness
121	something an end user reads, and an end user reads the verdicts rather than	123	as an inclusion of sets of stores, like that of the states (@sec:contract),
122	the states. To state that	124	the verdicts at v are turned into the stores consistent with them, the
123	the verdicts are correct as an inclusion of sets, the verdicts at v are turned	125	_verdict set_
124	into the set of stores consistent with them, the _verdict set_		
125	#isai("\<V>\<^bsub>res\<^esub> v"). If v carries the check `x > 0` with	126	#isai("\<V>\<^bsub>res\<^esub> v"). If v carries the check `x > 0` with
126	verdict #verdict("PROVED"), the verdict set is $setcomp(s, s(x) > 0)$: the verdict	127	verdict #verdict("PROVED"), the verdict set is $setcomp(s, s(x) > 0)$: the verdict
127	tells the user that x is positive there and nothing else.	128	tells the user that x is positive there and nothing else.
	18 unchanged lines hidden		
146		147	
147	The verdict set can be much larger than	148	The verdict set can be much larger than
148	#isai("\<R>\<^bsub>res\<^esub> v"), because a verdict keeps only	149	#isai("\<R>\<^bsub>res\<^esub> v"), because a verdict keeps only
149	the truth value of one condition (last row of @fig:chain). Soundness needs	150	the truth value of one condition (last row of @fig:chain). Soundness only
150	only that every store reaching v lies in it.	151	requires that every store reaching v lies in it.
151		152	
152	Since the verdicts at v are computed from the states at v, every store that	153	Since the verdicts at v are computed from the states at v, every store that
153	one of these states describes satisfies every definite verdict there. Thus	154	one of these states describes satisfies every definite verdict there. Thus
	44 unchanged lines hidden		
198	isai("\<V>\<^bsub>res\<^esub> v"),	199	isai("\<V>\<^bsub>res\<^esub> v"),
199	)))	200	)))
200	The first step places a store that a finite source execution reaches in the node	201	The first step places a store that a finite source execution reaches in the node
201	collecting semantics, at a node v that the simulation relates to the run. On the	202	collecting semantics, at a node v that the simulation relates to the run. On the annotated copy of the program, this is every node at which the run is observed
202	elaboration, this is every node at which the run is observed		
203	(#isathm("csim_point_compile_prog"), @sec:at-node). The second step splits	203	(#isathm("csim_point_compile_prog"), @sec:at-node). The second step splits
204	#isaconst("node_collect") into #isaconst("activation_collect") over the	204	#isaconst("node_collect") into #isaconst("activation_collect") over the
205	contexts #isai("adm") admits, with `main` in #isai("c\<^sub>0")	205	contexts #isai("adm") admits, with `main` in #isai("c\<^sub>0")
206	(@sec:contexts). It needs only that every valid trace carries some context.	206	(@sec:contexts). It only requires that every valid trace carries some context.
207	Functional policies meet this for every trace, and the entry-state policy	207	Functional policies meet this for every trace, and the entry-state policy
208	meets it through #oblig("TOTAL") on the solution (@sec:contract). The third	208	meets it through #oblig("TOTAL") on the solution (@sec:contract). The third
209	inclusion is the analyzer's soundness result. Its proof combines the second	209	inclusion is the analyzer's soundness result. Its proof combines the second
	164 unchanged lines hidden		
374	nonterminating programs are covered through their prefixes.	374	nonterminating programs are covered through their prefixes.
375	+ #isaconst("run_voblint") $"config"$ p $=$ #ctor("Analysed") $"res"$: the	375	+ #isaconst("run_voblint") $"config"$ p $=$ #ctor("Analysed") $"res"$: the
376	analyzer returned a report. Input validity and solver termination are	376	analyzer returned a report. Input validity and solver termination are
377	therefore not separate premises (@sec:report), and the theorem is a	377	therefore not separate premises (@sec:report). The theorem is a
378	partial-correctness result: termination is not proved for every program	378	partial-correctness result, since termination is not proved for every
379	(@sec:certificate).	379	program (@sec:certificate).
380	]	380	]
381		381	
382	The conclusions are stated at graph nodes, but a source configuration names	382	The conclusions are stated at graph nodes, but a source configuration names
383	none, since its residual command can occur at several places in the program.	383	none, since its residual command can occur at several places in the program.

```
The node comes from the run, through the relation
$p, s_0 tack.r ("residual", s, "frs") gt.closed v$ of @sec:at-node
(#isaconst("at_node")), which may observe one configuration at several nodes.
                                                                                  The theorem states that the configuration is observed at some node, and that
                                                                                  at every node $v$ where it is observed

The theorem has two halves. The first states that the configuration is
observed at some node. The second states that at every node $v$ where it is
observed

#[
  #set enum(numbering: n => "(S" + str(n) + ")")
  + #isai("s \<in> \<C>\<^bsub>\<G>,g,S\<^esub> v"): the store is collected at that node;
  + #isai("s \<in> \<R>\<^bsub>res\<^esub> v"): some state the report
    holds at $v$, under one of its contexts, describes $s$;
  + #isai("s \<in> \<V>\<^bsub>res\<^esub> v"): every #verdict("PROVED") check listed at $v$
    holds in $s$, and every #verdict("REFUTED") check is false in $s$.
]

== What verdicts and diagnostics guarantee #thy-badge("Voblint_CLI", "Analysis_Certified")
<sec:verdict-meaning>

By the theorem of @sec:headline, the four verdicts make the following
statements about _covered executions_, the finite source executions from an
initial store in #isaconst("cinit_stores"), stopped at any point:
- #verdict("PROVED"): whenever a covered execution reaches the check, the condition
  holds (#isathm("run_voblint_check_sound")).
- #verdict("REFUTED"): whenever a covered execution reaches the check, the condition
  fails (#isathm("run_voblint_check_sound")). It is not a verified
  counterexample: it establishes no reaching execution.
- #verdict("UNKNOWN"): the analysis decides neither.
- #verdict("DEAD"): no covered execution reaches the check: nothing is
  collected at its node (#isathm("run_voblint_dead_check_unreached")), and a
  check that an execution reaches is never #verdict("DEAD")
(#isathm("run_voblint_check_sound")).
#verdict("PROVED") and #verdict("REFUTED") hold vacuously when no execution
reaches the check, and only #verdict("DEAD") claims anything about
reachability. The corresponding theorem for #verdict("PROVED") and #verdict("REFUTED") is:

#[
  #show raw.where(block: true): set text(size: 6.5pt)
  #proved("run_voblint_check_sound")
]
```

```
The node comes from the run, through the relation
$p, s_0 tack.r ("residual", s, "frs") gt.closed v$ of @sec:at-node
(#isaconst("at_node")), which may observe one configuration at several nodes.
The theorem states that the configuration is observed at some node, and that
at every node $v$ where it is observed

#[
  #set enum(numbering: n => "(S" + str(n) + ")")
  + #isai("s \<in> \<C>\<^bsub>\<G>,g,S\<^esub> v"): the store is collected at that node;
  + #isai("s \<in> \<R>\<^bsub>res\<^esub> v"): some state the report
    holds at $v$, under one of its contexts, describes $s$;
  + #isai("s \<in> \<V>\<^bsub>res\<^esub> v"): every #verdict("PROVED") check listed at $v$
    holds in $s$, and every #verdict("REFUTED") check is false in $s$.
]

== Meaning of verdicts and diagnostics #thy-badge("Voblint_CLI", "Analysis_Certified")
<sec:verdict-meaning>

By the theorem of @sec:headline, the four verdicts make the following
statements about _covered executions_, the finite source executions from an
initial store in #isaconst("cinit_stores"), stopped at any point:
- #verdict("PROVED"): whenever a covered execution reaches the check, the condition
  holds (#isathm("run_voblint_check_sound")).
- #verdict("REFUTED"): whenever a covered execution reaches the check, the condition
  fails (#isathm("run_voblint_check_sound")). It is not a verified
  counterexample, since it establishes no reaching execution.
- #verdict("UNKNOWN"): the analysis decides neither.
- #verdict("DEAD"): no covered execution reaches the check, because nothing is
  collected at its node (#isathm("run_voblint_dead_check_unreached")).
  Conversely, a check that an execution reaches is never #verdict("DEAD")
(#isathm("run_voblint_check_sound")).
#verdict("PROVED") and #verdict("REFUTED") hold vacuously when no execution
reaches the check, and only #verdict("DEAD") claims anything about
reachability. The corresponding theorem for #verdict("PROVED") and #verdict("REFUTED") is:

#[
  #show raw.where(block: true): set text(size: 6.5pt)
  #proved("run_voblint_check_sound")
]
```

424	The report also contains arithmetic diagnostics. At every division and	422	At every division and remainder, the analyzer asks the queries of
425	remainder, the analyzer asks, through the queries of	423	@sec:coop-queries whether the divisor can be zero. It reports an error if the
426	@sec:coop-queries, whether the divisor can be zero. It reports an error if the analyses	424	divisor is zero whenever a covered execution reaches its node
427	show that it is zero, a warning if they cannot exclude zero, and nothing	425	(#isathm("run_voblint_arithmetic_refuted")), a warning if the analyses cannot
428	otherwise. An error means that the divisor is zero whenever a covered execution reaches	426	exclude zero, and nothing otherwise. Without a diagnostic at v, every divisor
429	its node (#isathm("run_voblint_arithmetic_refuted")). At a node v without a diagnostic,	427	on an intraprocedural edge leaving v is nonzero in every store collected at
430	every divisor in an expression on an intraprocedural edge leaving v is	428	v (#isathm("run_voblint_arithmetic_safe") adds call arguments):
431	nonzero in every store collected at v		
432	(#isathm("run_voblint_arithmetic_safe") adds call arguments):		
433		429	
434	#[	430	#[
435	#show raw.where(block: true): set text(size: 6.5pt)	431	#show raw.where(block: true): set text(size: 6.5pt)
436	#proved("run_voblint_arithmetic_intra_safe")	432	#proved("run_voblint_arithmetic_intra_safe")
437	]	433	]
438		434	
439	These theorems complete the chain of @fig:intro-nest. Each verdict holds for	435	These theorems complete the chain of @fig:intro-nest. Each verdict holds for
440	every covered execution that reaches its check, in every analysis configuration.	436	every covered execution that reaches its check, in every analysis configuration.
441	Termination (@sec:certificate) and the components outside the proof		
442	(@fig:intro-trust) are not covered.		

thesis/content/11-instances.typ

77e07947 → 0edeac81

414 old lines, 411 new lines

88% similar lines **+47** **-50** **16 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": fixture, isaconst, isalocale, isathm, isatype, listing, site, thy-badge	3	#import "../lib/code.typ": fixture, isaconst, isalocale, isathm, isatype, listing, site, thy-badge
4	#import "../lib/figures.typ": int-axis, int-strip, printed-set, verdict as verdict-chip	4	#import "../lib/figures.typ": int-axis, int-strip, printed-set, verdict as verdict-chip
5	#import "../lib/math.typ": *	5	#import "../lib/math.typ": *
6	#import "../lib/claims.typ": claim-ref, claim-row	6	#import "../lib/claims.typ": claim-ref, claim-row
7		7	
8	#let verdict(name, loc) = verdict-chip(claim-row(name, loc).verdict)	8	#let verdict(name, loc) = verdict-chip(claim-row(name, loc).verdict)
9	#let state(name, loc) = absval(raw(claim-row(name, loc).state))	9	#let state(name, loc) = absval(raw(claim-row(name, loc).state))
10		10	
11		11	
12	= Numeric and Relational Analyses <ch:instances>	12	= Numeric and Relational Analyses <ch:instances>
13		13	
14	The preceding chapters stated what a domain, an analysis, and a combination of	14	A user of Voblint selects the analyses that run on a program, and the
		15	selection determines which checks the report proves. The preceding chapters stated what a domain, an analysis, and a combination of
15	analyses must prove, and @ch:results showed that every report is sound,	16	analyses must prove, and @ch:results showed that every report is sound,
16	whichever analyses produced it. This chapter introduces and describes the analyses that Voblint	17	whichever analyses produced it. Voblint instantiates this contract with the

Old	77e07947	New	0edeac81
17	implements. A user selects among the	18	non-relational domains Sign, Interval, Parity, and Congruence, their reduced
18	non-relational domains Sign, Interval, Parity, and Congruence, their reduced product	19	product Int, and a relational order analysis. Each base domain records a
19	Int, and a relational order analysis. The chapter compares the four base	20	different property of a variable, and on one loop Int proves a check that none
20	domains on one loop, then describes how Int lets them exchange facts and how	21	of them proves alone. The order analysis proves relations between variables, so
21	the order analysis tracks relations between variables, which shows that the	22	the framework also supports states that describe several variables jointly.
22	framework is not limited to one value per variable.		
23		23	
24	== Four non-relational numeric domains <sec:instances-supply>	24	== Four non-relational numeric domains <sec:instances-supply>
25		25	
26	The four base domains, Sign #thy-badge("Voblint_Domain", "Sign_Lattice"),	26	The four base domains, Sign #thy-badge("Voblint_Domain", "Sign_Lattice"),
27	Interval #thy-badge("Voblint_Domain", "Interval_Lattice"), Parity	27	Interval #thy-badge("Voblint_Domain", "Interval_Lattice"), Parity
28	#thy-badge("Voblint_Domain", "Parity_Lattice"), and Congruence	28	#thy-badge("Voblint_Domain", "Parity_Lattice"), and Congruence
29	#thy-badge("Voblint_Domain", "Congruence_Lattice"), are the classical	29	#thy-badge("Voblint_Domain", "Congruence_Lattice"), are the classical
30	non-relational domains @mine17;	30	non-relational domains @mine17;
31	congruences go back to Granger @granger89. The fifth domain, Int, combines all	31	congruences go back to Granger @granger89. The fifth domain, Int, combines all
32	four. Its value holds one value of each base domain and denotes the	32	four. Its value holds one value of each base domain and denotes the
33	intersection of their meanings (@sec:reduced-product). All five are	33	intersection of their meanings (@sec:reduced-product). All five are
34	non-relational: they describe each variable separately, and the relational	34	non-relational in the sense of @sec:nonrel-state, so each describes every
35	state of the order analysis follows in @sec:rel-state. @fig:gamma shows one	35	variable separately. @fig:gamma shows one
36	value of each and the integers it stands for.	36	value of each and the integers it stands for.
37		37	
38	#figure(	38	#figure(
	59 unchanged lines hidden		
98	(#isathm("sound_nonrelational_ops.dg_analysis_execI")). The only further	98	(#isathm("sound_nonrelational_ops.dg_analysis_execI")). The only further
99	obligation concerns the initial state, which must describe the initial stores.	99	obligation concerns the initial state, which must describe the initial stores.
100		100	
101	The loop below is the fixture #fixture("12-widening/known-imprecision/05-mine_ex410_stride2_parity.vimp", label: "05-mine_ex410_stride2_parity.vimp"),	101	We compare the four domains on one loop, the fixture #fixture("12-widening/known-imprecision/05-mine_ex410_stride2_parity.vimp", label: "05-mine_ex410_stride2_parity.vimp"),
102	adapted from Goblint's regression test #link("https://github.com/goblint/analyzer/blob/d155e9e/tests/regression/56-witness/29-mine-tutorial-ex4.10.c")[`56-witness/29-mine-tutorial-ex4.10.c`]	102	adapted from Goblint's regression test #link("https://github.com/goblint/analyzer/blob/d155e9e/tests/regression/56-witness/29-mine-tutorial-ex4.10.c")[`56-witness/29-mine-tutorial-ex4.10.c`]
103	(revision `d155e9e`) after Example 4.10 of Miné's tutorial @mine17. In every	103	(revision `d155e9e`) after Example 4.10 of Miné's tutorial @mine17. In every
104	run, v takes the odd values $1, 3, dots, 51$ and the loop exits with	104	run, v takes the odd values $1, 3, dots, 51$ and the loop exits with
	68 unchanged lines hidden		
173	answer is sound, yet none of the four proves `v == 51`. Only Int, which puts	173	answer is sound, yet none of the four proves `v == 51`. Only Int, which puts
174	the bound and the oddness together, does (@sec:reduced-product).	174	the bound and the oddness together, does (@sec:reduced-product).
175		175	
176	== Combining the domains in a reduced product #thy-badge("Voblint_Domain", "Int_Lattice") <sec:reduced-product>	176	== The reduced product of the domains #thy-badge("Voblint_Domain", "Int_Lattice") <sec:reduced-product>
177		177	
178	An Int value (#isatype("int_dom")) holds one value of each base domain and	178	An Int value (#isatype("int_dom")) holds one value of each base domain and
179	stands for the integers that all four allow. Running the four side by side	179	stands for the integers that all four allow. Running the four side by side
180	would be sound, but each would work only with its own value, so a fact that	180	would be sound, but each would work only with its own value. A fact that

181	needs two of them would not be found. @fig:stride2 shows this: no	181	needs two of them, such as `v == 51` in @fig:stride2, would then stay unproved.
182	base domain alone proves `v == 51`. In the program below, the guards keep	182	In the program below, the guards keep
183	x between -2 and 10, and the assignment makes x one more than a	183	x between -2 and 10, and the assignment makes x one more than a
184	multiple of 4. Only 1, 5, and 9 satisfy both, so `x >= 1 && x <= 9`	184	multiple of 4. Only 1, 5, and 9 satisfy both, so `x >= 1 && x <= 9`
185	holds. Yet the interval alone still allows -2, and the residue class alone	185	holds. Yet the interval alone still allows -2, and the residue class alone
	30 unchanged lines hidden		
216	facts only through answers to queries, where a transfer asks one. Int is a	216	facts only through answers to queries, where a transfer asks one. Int is a
217	single domain whose value holds all four components, and it reduces after	217	single domain whose value holds all four components, and it reduces after
218	every arithmetic operation and guard. A fact such as "odd" therefore sharpens	218	every arithmetic operation and guard. A fact such as "odd" therefore sharpens
219	the interval stored in the value itself, inside expression evaluation, and not	219	the interval stored in the value itself, during expression evaluation.
220	just the answer to one query. Goblint draws the same line. Its integer	220	Goblint separates the two mechanisms in the same way. Its integer domains form
221	domains form one value domain inside its base analysis, whereas its MCP	221	one value domain inside its base analysis, whereas its MCP combines whole
222	combines whole analyses.	222	analyses.
223		223	
224	A reduction round passes information in two steps. It first combines the range	224	A reduction round first combines the range
225	information of Sign and Interval and uses it to sharpen Sign, Interval, and	225	information of Sign and Interval and uses it to sharpen Sign, Interval, and
226	Parity. It then intersects the residue class with the parity and uses the	226	Parity. It then intersects the residue class with the parity and uses the
227	result to sharpen Interval, Parity, and Congruence. The user chooses how often	227	result to sharpen Interval, Parity, and Congruence. The user chooses how often
228	these rounds run: never, once, or until the value stops changing, which is the	228	these rounds run: never, once, or until the value stops changing, which is the
229	default. Every mode keeps the integers a value stands for and only makes the	229	default. Every mode keeps the integers a value stands for and only makes the
230	value smaller (#isathm("refine_exact"), #isathm("refine_reductive")).	230	value smaller (#isathm("refine_exact"), #isathm("refine_reductive")).
		231	In the generated OCaml, the default mode iterates without a termination proof,
		232	so on some inputs the analyzer might not return. Soundness is unaffected, since
		233	the theorem only speaks about returned reports (@fig:intro-trust).
231		234	
232	The design follows Goblint's integer domain	235	The design follows Goblint's integer domain
233	#link("https://github.com/goblint/analyzer/blob/5503dec/src/cdomain/value/cdomains/int/intDomTuple.ml")[`IntDomTuple`] at revision `5503dec`. Goblint combines up to six optional components, each enabled by an option. Int always carries	236	#link("https://github.com/goblint/analyzer/blob/5503dec/src/cdomain/value/cdomains/int/intDomTuple.ml")[`IntDomTuple`] at revision `5503dec`. Goblint combines up to six optional components, each enabled by an option. Int always carries
234	its four components, and Sign and Parity have no counterpart there. Goblint's	237	its four components, and Sign and Parity have no counterpart there. Goblint's
235	option `ana.int.refinement` offers the same three modes, but Goblint defaults	238	option `ana.int.refinement` offers the same three modes, but Goblint defaults
236	to no reduction, whereas Voblint defaults to repeating rounds until nothing	239	to no reduction, whereas Voblint defaults to repeating rounds until nothing
237	changes.	240	changes.
238		241	
239	In the generated OCaml, the default mode iterates without a termination proof,		
240	so on some inputs the analyzer might not return. Soundness is unaffected, since		
241	the theorem only speaks about returned reports (@fig:intro-trust).		
242			
243	Int reduces the result of every arithmetic operation and of the guard filter,	242	Int reduces the result of every arithmetic operation and of the guard filter,
244	but not the result of a join, a widening, or a narrowing, which all work	243	but not the result of a join, a widening, or a narrowing, which all work
245	componentwise. For join and widening, Goblint makes the same choice. After a	244	componentwise. For join and widening, Goblint makes the same choice. After a
	35 unchanged lines hidden		
281	same stores but are different values. The join can also forget an implied	280	same stores but are different values. The join can also forget an implied

282	relation. The join of $absval({x <= y, y <= z})$ and $absval({x <= y, x <= z})$ keeps only	281	relation. The join of $absval({x <= y, y <= z})$ and $absval({x <= y, x <= z})$ keeps only
283	their common pair and is $absval({x <= y})$, although both operands imply $x <= z$. The	282	their common pair and is $absval({x <= y})$, although both operands imply $x <= z$. The
284	order analysis is therefore not as precise as a closed relational domain could	283	order analysis is therefore less precise than a relational domain that closes
285	be. It demonstrates that the framework supports relational	284	its pairs under transitivity. Its purpose is to show that the framework
286	states.	285	supports relational states.
287			
288	Pairs enter only at guards, and an assignment to x removes every pair that		
289	mentions x, so P is finite in every value the analysis constructs, and generated code can		
290	store it as a list.		
291		286	
292	#let _rnode(pos, name, body) = node(pos, text(size: 7pt, body), name: name, inset: 3pt)	287	#let _rnode(pos, name, body) = node(pos, text(size: 7pt, body), name: name, inset: 3pt)
293	#figure(	288	#figure(
	48 unchanged lines hidden		
342	$x = y = z$.],	337	$x = y = z$.],
343	) <fig:rel-state>	338	) <fig:rel-state>
344		339	
345	The emptiness		
346	test only compares with #ctor("RelBot") (#isathm("is_empty_relc_gamma")).		
347	Unlike the pointwise state, the carrier has a single representation of		
348	unreachability, so it needs no lifting.		
349			
350	A relational state stores no value per variable, so it cannot evaluate an	340	A relational state stores no value per variable, so it cannot evaluate an
351	expression to an abstract value, and its transfers work on the pairs	341	expression to an abstract value, and its transfers work on the pairs
352	directly. An assignment to x invalidates every relation that involves x,	342	directly. An assignment to x invalidates every relation that involves x,
353	so it forgets every pair that mentions x. New relations come from guards. On	343	so it forgets every pair that mentions x. New relations come from guards. On
354	the true arm of `if (x < y)`, the analysis knows that x is smaller than y	344	the true arm of `if (x < y)`, the analysis knows that x is smaller than y
355	and records this by meeting its state with $absval(ctor("RelC"){x <= y})$	345	and records this by meeting its state with $absval(ctor("RelC"){x <= y})$
356	(#isaconst("assume_step")). Since the carrier stores only non-strict pairs, it	346	(#isaconst("assume_step")). Since the carrier stores only non-strict pairs, it
357	keeps $x <= y$ and loses the strictness, which costs precision but not	347	keeps $x <= y$ and loses the strictness, which costs precision but not
358	soundness. This guard filter meets the same soundness statement as the	348	soundness. This guard filter meets the same soundness statement as the
359	numeric guard filter of @sec:branches (#isathm("relc_branch_step_sound")). The type	349	numeric guard filter of @sec:branches (#isathm("relc_branch_step_sound")).
360	instantiates the solver's classes and #isalocale("executable_domain") of		
361	@fig:domain-carrier, but it is not a numeric domain.		
362		350	
363	== Proving order relations between variables #thy-badge("Voblint_Analysis_Relational", "Rel_Order_Local") <sec:relational>	351	Since pairs enter only at guards and assignments remove them, P is finite in
		352	every value the analysis constructs, and generated code can store it as a
		353	list. The emptiness test only compares with #ctor("RelBot")
		354	(#isathm("is_empty_relc_gamma")). The carrier has a single representation of
		355	unreachability, so it needs no lifting by a bottom element as the pointwise
		356	state of @sec:nonrel-state does. The type instantiates the solver's classes and
		357	#isalocale("executable_domain") of @fig:domain-carrier, but it is not a numeric
		358	domain.

364		359	
365	@sec:rel-state introduced the relational state as an abstract domain. This	360	== Order relations between variables #thy-badge("Voblint_Analysis_Relational", "Rel_Order_Local") <sec:relational>
366	section builds the order analysis on top of it and demonstrates it on a	361	
367	program. After an	362	The order analysis runs on the relational state of @sec:rel-state and proves
368	assignment has forgotten the pairs involving the changed variable, it asks the	363	order checks that Interval cannot decide. Its guard and assignment transfers
369	shared channel how the new value relates to each variable, and	364	are those of @sec:rel-state. After an assignment has forgotten the pairs
370	every selected analysis, the order analysis included, may give an answer	365	involving the changed variable, the analysis asks the channel how the new
371	(@sec:coop-mcp). In the program below, the guard `x < y` tells Interval	366	value relates to each variable. Every selected analysis, the order analysis
		367	included, may give an answer (@sec:coop-mcp). In the program below, the guard `x < y` tells Interval
372	nothing about either variable, so it keeps	368	nothing about either variable, so it keeps
373	#state("order-alone-interval", "12:5") and reports	369	#state("order-alone-interval", "12:5") and reports
374	#verdict("order-alone-interval", "12:5") at the first check. The order	370	#verdict("order-alone-interval", "12:5") at the first check. The order
	34 unchanged lines hidden		
409	domain can express. A user does not have to choose between them. The	405	domain can express. A user does not have to choose between them. The
410	product state and the channel of @ch:cooperation let Int and the order	406	product state and the channel of @ch:cooperation let Int and the order
411	analysis run together. They exchange facts	407	analysis run together. They exchange facts
412	through queries, and every check is decided by the meet of their answers. Selecting both gets the most out of the analyses Voblint	408	through queries, and every check is decided by the meet of their answers.
413	ships, although no theorem guarantees that a combination is more precise than	409	Selecting both thus lets a check use the facts of either analysis, although no
414	its parts (@sec:coop-mcp).	410	theorem guarantees that a combination is more precise than its parts
		411	(@sec:coop-mcp).

thesis/content/12-executable.typ

77e07947 → 0edeac81

328 old lines, 331 new lines

88% similar lines **+41** **-38** **18 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/math.typ": *	2	#import "../lib/math.typ": *
3	#import "../lib/figures.typ": *	3	#import "../lib/figures.typ": *
	45 unchanged lines hidden		
49	= The Executable Analyzer <ch:executable>	49	= The Executable Analyzer <ch:executable>
50		50	
51	@ch:results proved that every report #isaconst("run_voblint") returns is	51	@ch:results proved that every report #isaconst("run_voblint") returns is
52	sound. A user, however, does not evaluate a HOL function. The user writes	52	sound. Someone who analyzes a VIMP program, however, writes source text and
53	source text and reads a report in a terminal or a browser tab. Between the	53	reads a report in a terminal or a browser tab. This chapter asks how the
54	two, the proved function has to become a program. Its abstract states are	54	proved HOL function becomes a program between the two, and which components the
		55	soundness guarantee for that program rests on. The proved function's abstract states are
55	mathematical functions, which code cannot compute with, so they are first	56	mathematical functions, which code cannot compute with, so they are first
56	replaced by finite representations (@sec:represented-function). Isabelle's	57	replaced by finite representations (@sec:represented-function). Isabelle's
57	code generator then exports #isaconst("run_voblint") to OCaml	58	code generator then exports #isaconst("run_voblint") to OCaml

```
(@sec:codegen), and an unverified shell of handwritten OCaml parses the source text and displays
the report (@sec:ocaml-boundary). The change of representation is proved
to preserve the theorem. The code generator and the shell are not proved
correct, so the guarantee for the running tool also rests on them, as
@fig:intro-trust shows.

== Finite representations of abstract states <sec:represented-function>

To export #isaconst("run_voblint") as code, every function it calls must be
computable. Most of the development already is, since its datatypes and
recursive functions translate directly into a functional program, but the
concretizations and the pointwise states do not. The concretization $conc$ maps an abstract value to a possibly infinite set of integers, and in general
no program can compute such a set. The analyzer does not need to, because
$conc$ only states what a value means. The domain interface therefore
separates #isalocale("executable_domain") from the classes and locales
below it in @fig:domain-carrier, which add $conc$ and its laws. Code
generation exports the executable part together with each domain's operation
record, and the rest serves only the proofs.

The pointwise states of @ch:domains are total functions on
infinitely many variable names, so the solver could not even test whether two
```

12 unchanged lines hidden

```
@ch:analysis-interface carry over to the code that runs
(#isathm("exec_local_spec_sound")).

With concretizations left to the proofs and pointwise states replaced, nothing
else stands in the way. The verified TD solver already comes with an executable
version, #isaconst("solve_c", thy: "TD_side_upd_rule"), which Tilscher et al.
refine from its definition @tilscher26, and @sec:certificate shows that every
run of it that returns satisfies the solver's correctness theorem. The values of the numeric domains (@sec:instances-supply) and the
states of the order analysis (@sec:rel-state) are already finite data.

== Exporting the analyzer to OCaml and WebAssembly #thy-badge("Voblint_Codegen", "Voblint_Codegen") <sec:codegen>

With these replacements, every function that #isaconst("run_voblint") uses
has code equations, and Isabelle's
```

12 unchanged lines hidden

```
  str(calc.round(r.before.mean_s / r.after.mean_s, digits: 0))
}
```

```
(@sec:codegen), and an unverified shell of handwritten OCaml parses the source text and displays
the report (@sec:ocaml-boundary). The change of representation is proved
to preserve the theorem. The code generator and the shell are not proved
correct, so the soundness guarantee for the running tool also rests on them, as
@fig:intro-trust shows.

== Finite representations of abstract states <sec:represented-function>

To export #isaconst("run_voblint") as code, every function it calls must be
computable. Most of the development already is, since its datatypes and
recursive functions translate directly into a functional program, but the
concretizations and the pointwise states do not. The concretization $conc$ maps an abstract value to a possibly infinite set of integers, and in general
no program can compute such a set. The analyzer does not need to, because
$conc$ only states what a value means. The domain interface therefore
separates #isalocale("executable_domain") from the classes and locales
below it in @fig:domain-carrier, which add $conc$ and its laws. Code
generation exports the executable part together with each domain's operation
record. The concretizations and their laws serve only the proofs.

The pointwise states of @ch:domains are total functions on
infinitely many variable names, so the solver could not even test whether two
```

12 unchanged lines hidden

```
@ch:analysis-interface carry over to the code that runs
(#isathm("exec_local_spec_sound")).

The remaining parts are executable already. The analyzer runs the TD
solver's executable form #isaconst("solve_c", thy: "TD_side_upd_rule")
(@sec:certificate), which Tilscher et al. refine from its definition
@tilscher26. The values of the numeric domains (@sec:instances-supply) and the
states of the order analysis (@sec:rel-state) are finite data.

== Export to OCaml and WebAssembly #thy-badge("Voblint_Codegen", "Voblint_Codegen") <sec:codegen>

With these replacements, every function that #isaconst("run_voblint") uses
has code equations, and Isabelle's
```

12 unchanged lines hidden

```
  str(calc.round(r.before.mean_s / r.after.mean_s, digits: 0))
}
```

120	The generated module has #stat("generated_ocaml") lines and follows the proofs rather than the	120	The generated module has #stat("generated_ocaml") lines. Its code mirrors the
121	habits of OCaml programmers, which makes it hard to read. Type classes become records of operations passed	121	definitions of the proofs, which makes it hard to read. Type classes become records of operations passed
122	as arguments, finite sets and maps become lists that every lookup scans, and	122	as arguments, finite sets and maps become lists that every lookup scans, and
123	the solver keeps its valuation as a chain of closures, one per update.	123	the solver keeps its valuation as a chain of closures, one per update.
124		124	
	44 unchanged lines hidden		
169	most of the CPU time to a single function. The generated	169	most of the CPU time to a single function. The generated
170	code of #isaconst("init_publications") recompiled the whole program at every	170	code of #isaconst("init_publications") recompiled the whole program at every
171	equation evaluation, because its definition names the program's entry through the compiler from	171	equation evaluation, because its definition names the program's entry through the compiler from
172	VIMP to control-flow graphs (@sec:compile). A proved code equation, #isathm("init_publications_code"), that names	172	VIMP to control-flow graphs (@sec:compile). A proved code equation, #isathm("init_publications_code"), names
173	the entry directly as #ctor("FunctionEntry") #isaconst("prog_main_name") fixed this	173	the entry directly as #ctor("FunctionEntry") #isaconst("prog_main_name") and fixed this
174	without changing any theorem, and the chain of 256 assignments then ran	174	without changing any theorem. The chain of 256 assignments then ran
175	#_speedup("chain256") times faster and a program of 512 assignments in	175	#_speedup("chain256") times faster, and a program of 512 assignments in
176	procedures of eight statements #_speedup("procs512") times faster, in one	176	procedures of eight statements ran #_speedup("procs512") times faster, in one
177	measurement each	177	measurement each
178	(#link(repo-blob + "thesis/shared/generated/bench-init-publications.json")[recorded]).	178	(#link(repo-blob + "thesis/shared/generated/bench-init-publications.json")[recorded]).
179	Finding further inefficiencies and improving the generated code is future work.	179	Finding further inefficiencies and improving the generated code is future work.
	24 unchanged lines hidden		
204	project site.],	204	project site.],
205	) <fig:build>	205	) <fig:build>
206		206	
207	== Tracing solver steps without changing the proofs #thy-badge("Voblint_Solver", "Solver_Trace") <sec:tracing>	207	== Traced solver runs #thy-badge("Voblint_Solver", "Solver_Trace") <sec:tracing>
208		208	
209	To explain a run, the command-line tool and the playground can trace the	209	To explain a run, the command-line tool and the playground can trace the
210	solver's steps. The solver trace records when each unknown is queried, which update	210	solver's steps. The solver trace records when each unknown is queried, which update
211	destabilizes which unknown, and where widening applies. Goblint's tracing of	211	destabilizes which unknown, and where widening applies. Goblint's tracing of
212	its top-down solvers reports the same events. Tracing must leave the definitions and proofs of	212	its top-down solvers reports the same events. Tracing must leave the definitions and proofs of
213	the vendored TD solver @tilscher26 untouched. Voblint therefore adds tracing only to the equations	213	the vendored TD solver @tilscher26 untouched. Voblint therefore adds tracing only to the equations
214	that code generation translates. Every traced equation reports each step	214	that code generation translates. Every traced equation reports each step
215	through one constant, #isaconst("trace_event"), which is defined as the unit	215	through one constant, #isaconst("trace_event"), which is defined as the unit
216	value $()$ and so does nothing in the logic. Each traced equation is thus	216	value $()$ and so does nothing in the logic. Each traced equation is thus
217	provably equal to the vendored one it copies (for example,	217	provably equal to the vendored one it copies (for example,
218	#isathm("solve_rec_c_traced")). The traced copies make up a second entry point, #isaconst("run_voblint_traced"), which	218	#isathm("solve_rec_c_traced")).
		219	
		220	The traced copies make up a second entry point, #isaconst("run_voblint_traced"), which
219	equals #isaconst("run_voblint") in the logic	221	equals #isaconst("run_voblint") in the logic

```
(#isathm("run_voblint_traced_eq")). Code generation exports both. A
#isacmd("code_printing") declaration then tells the code generator to emit, in
```

12 unchanged lines hidden

```
#pagebreak(weak: true)
== Unverified parsing, rendering, and playground <sec:ocaml-boundary>

The generated #isaconst("run_voblint") works on a syntax tree and returns a
typed answer, whereas a user writes source text and reads a report.
Unverified OCaml bridges both ends. On the input side, an `ocamllex` lexer
and a Menhir parser turn the source text into an
#isatype("imp_prog") and record each check's source position in its label.
Both are generated from the grammar manifest
#link(repo-blob + "manifests/vimp-grammar.yaml")[`vimp-grammar.yaml`] by
#link(repo-blob + "scripts/gen_vimp_menhir.py")[`gen_vimp_menhir.py`]. On
the output side, the generated #isaconst("render_report") turns the report's
states into text, and handwritten OCaml either prints the command-line report,
placing each check at the position its label carries, or serializes the answer
as JSON, from which the playground draws the graph and marks the verdicts and
diagnostics in the editor. The command-line tool and the playground share all
of this code and differ only in how they receive a request. The code generator maps HOL's integers to
arbitrary-precision Zarith integers, as VIMP requires (@sec:vimp-vs-c).

@fig:shell shows, for the program below, the call that #isaconst("run_voblint")
receives and the answer it returns once #isaconst("render_report") has turned
```

21 unchanged lines hidden

```
  },
  kind: image,
  placement: none,
  caption: [The input and output of the generated analyzer for the program
    above (claim #claim-ref("shell-json")). The call carries the configuration
    and the syntax tree `p` the parser built, of the kind @fig:vimp-ast shows in
    full; the answer carries the report with its states rendered as text, here
    with its one check in full. Everything before the call, and the rendering
    from the answer on, is unverified. `{…}` and `[…×n]` fold parts of the JSON.],
) <fig:shell>

The #link(site("playground.html"))[playground] runs the analyzer in the
browser. Its toolbar sets the configuration of #isaconst("run_voblint") and
toggles the tracing of @sec:tracing, and the end-to-end soundness theorem covers every
```

```
(#isathm("run_voblint_traced_eq")). Code generation exports both. A
#isacmd("code_printing") declaration then tells the code generator to emit, in
```

12 unchanged lines hidden

```
#pagebreak(weak: true)
== Unverified parser, rendering, and playground <sec:ocaml-boundary>

The generated #isaconst("run_voblint") works on a syntax tree and returns a
typed answer, whereas a user writes source text and reads a report.
Unverified OCaml bridges both ends. On the input side, an `ocamllex` lexer
and a Menhir parser turn the source text into an
#isatype("imp_prog") and record each check's source position in its label.
Both are generated from the grammar manifest
#link(repo-blob + "manifests/vimp-grammar.yaml")[`vimp-grammar.yaml`] by
#link(repo-blob + "scripts/gen_vimp_menhir.py")[`gen_vimp_menhir.py`]. On
the output side, the generated #isaconst("render_report") turns the report's
states into text. Handwritten OCaml then either prints the command-line report,
placing each check at the position its label carries, or serializes the answer
as JSON. From the JSON, the playground draws the graph and marks the verdicts and
diagnostics in the editor. The command-line tool and the playground share all
of this code and differ only in how they receive a request. Inside the generated
code, the code generator maps HOL's integers to arbitrary-precision Zarith
integers, as VIMP requires (@sec:vimp-vs-c).

@fig:shell shows, for the program below, the call that #isaconst("run_voblint")
receives and the answer it returns once #isaconst("render_report") has turned
```

21 unchanged lines hidden

```
  },
  kind: image,
  placement: none,
  caption: [Input and output of the generated analyzer for the program above
    (recorded analyzer output #claim-ref("shell-json")). The call carries the
    configuration and the parsed syntax tree `p` (cf. @fig:vimp-ast). The answer
    carries the report with its one check in full. Everything before the call
    and after the answer is unverified. `{…}` and `[…×n]` fold parts of the JSON.],
) <fig:shell>

The #link(site("playground.html"))[playground] is the browser front end of
@fig:build. Its toolbar sets the configuration of #isaconst("run_voblint") and
toggles the tracing of @sec:tracing, and the end-to-end soundness theorem covers every
```

292	configuration it can select (#isathm("run_voblint_source_sound")).	294	configuration it can select (#isathm("run_voblint_source_sound")).
293	@fig:pg-overview shows one run.	295	@fig:pg-overview shows one run.
	23 unchanged lines hidden		
317	label: <fig:playground>,	319	label: <fig:playground>,
318	)	320	)
319		321	
320	== Testing the unverified components <sec:eval-corpus>	322	== Tests of the unverified components <sec:eval-corpus>
321		323	
322	The handwritten OCaml around the generated analyzer (@tab:sizes) is tested.	324	The handwritten OCaml around the generated analyzer (@tab:sizes) is tested.
323	A #link(repo-blob + "tests/regression")[regression corpus] of	325	A #link(repo-blob + "tests/regression")[regression corpus] of
324	#stat("corpus.cases") VIMP fixtures pins the expected verdicts: a definite one	326	#stat("corpus.cases") VIMP programs pins the expected verdicts: a definite one
325	in `precision/`, #verdict("UNKNOWN") in `soundness/`, and a documented loss in	327	in `precision/`, #verdict("UNKNOWN") in `soundness/`, and a documented loss in
326	`known-imprecision/`. #link(repo-blob + "tests/solver-trace")[Golden files] pin	328	`known-imprecision/`. Stored expected traces
327	solver traces (@sec:tracing), and property tests check that the parser reads	329	(#link(repo-blob + "tests/solver-trace")[golden files]) pin the solver traces of
		330	@sec:tracing, and property tests check that the parser reads
328	back random syntax trees.	331	back random syntax trees.

thesis/content/13-evaluation.typ

77e07947 → 0edeac81

825 old lines, 819 new lines

90% similar lines **+78** **-84** **33 changed blocks**

Old	77e07947	New	0edeac81
1	#import "../lib/code.typ": *	1	#import "../lib/code.typ": *
2	#import "../lib/stats.typ": stat	2	#import "../lib/stats.typ": stat
3	#import "../lib/math.typ": absval, conc, ivl, parity-name, signval	3	#import "../lib/math.typ": absval, conc, ivl, parity-name, signval
	16 unchanged lines hidden		
20	satisfies it as well (#isathm("unknown_everywhere_sound")). Following	20	satisfies it as well (#isathm("unknown_everywhere_sound")). Following
21	#cite(<kim26regulatory>, form: "prose", supplement: [§6]), this chapter first	21	#cite(<kim26regulatory>, form: "prose", supplement: [§6]), this chapter first
22	checks that the premises can hold and that the obligations exclude real	22	checks that the premises can hold and that the obligations exclude real
23	errors. It then shows, on concrete programs, what precision the analyzer reaches	23	errors. It then shows, on concrete programs and in comparison with Goblint,
24	and where it ends. Several facts below concern one	24	what precision the analyzer reaches and where it ends, and finally how long the
25	fixed program and are proved by `eval`, which runs generated code and therefore	25	generated code takes. Several facts below concern one fixed program and are
		26	proved by Isabelle's `eval` method, which runs generated code and therefore
26	also trusts the code generator (@fig:intro-trust).	27	also trusts the code generator (@fig:intro-trust).
27		28	
28	== The soundness theorems are not vacuous #thy-badge("Voblint_Examples", "Example_Non_Vacuity") <sec:nonvacuity>	29	== Non-vacuity of the soundness theorems #thy-badge("Voblint_Examples", "Example_Non_Vacuity") <sec:nonvacuity>
29		30	
30	A theorem whose premises can never hold together is true but says nothing. The	31	A theorem whose premises can never hold together is true but says nothing. The
31	end-to-end soundness theorem has three premises: an initial store, a source execution from it,	32	end-to-end soundness theorem has three premises: an initial store, a source execution from it,
	21 unchanged lines hidden		
53	semantics at that node is empty. In all three instances, the source executions are	54	semantics at that node is empty. In all three instances, the source executions are
54	proved by rule application, while `eval` computes the compiled graphs and the	55	proved by rule application, while `eval` computes the compiled graphs and the

```
55  analyzer's answers. The end-to-end soundness theorem and its companions therefore do not hold vacuously, since
56  each applies to at least one program on which the solve actually terminates.
57
58  == The obligations reject a real Goblint bug <sec:eval-1161>
59
60  Non-vacuity does not show that the obligations are strong enough to reject a
61  wrong analysis. One way to test them is to check them against a real error. @ch:intro opened
62  with #link("https://github.com/goblint/analyzer/issues/1156")[issue 1156], in
63  which Goblint claimed `c % 2 == 1` for $c in {-5, -7}$, although C's truncating
64  remainder gives $-1$ for both values.
65  #link("https://github.com/goblint/analyzer/pull/1161")[Pull request 1161]
66  restricted when the congruence domain returns a constant remainder and added
67  the program as regression test `37-congruence/14-negative.c` @goblint1161.
```

68 unchanged lines hidden

```
136     both executions.],
137 ) <fig:goblint-1161>
138
139 == Measuring precision on concrete programs <sec:eval-precision>
140
141 The theorem says nothing about precision, so the examples below measure it on
142 single programs whose concrete behavior is known.
143
144 Earlier chapters already showed precise answers. Narrowing recovers the bound of the counting loop after widening
145 (@sec:widening), entry-state contexts keep the two calls of `bump` apart
```

17 unchanged lines hidden

```
163 its call site in `f`, so both activations of `g` share one context. Sign joins
164 #signval("+") and #signval("-") there to $absval(top)$, and neither check is
165 proved. Call strings of length 2 also record the call in `main`, keep the two
166 activations apart, and prove both checks. The price is a second copy of `g`.
167 The theories prove the same separation for #isaconst("run_voblint") itself
168 (#isathm("run_voblint_call_string_2_decides"), #isathm("cs_nest_checks_hold"))
169 and at the entry of `g` on hand-built equation systems
```

36 unchanged lines hidden

```
206
207 More contexts also cost more, since every context of a procedure gets its own
208 copy of the procedure's unknowns. The program `down(100)`, which recurses down
209 to `down(0)` and checks `n >= 0` at every level, shows that this cost does not
210 buy precision gradually. With call strings of length 100, every level gets its
211 own context, and the check is #verdict(snapshot-verdict(_k100, "n >= 0")) with
```

```
#_k100.clusters.len() procedure copies. With length 99, the analyzer still
builds #_k99.clusters.len() copies, but the deepest calls share one context.
```

16 unchanged lines hidden

```
explosion by omitting intervals from the contexts of recursive functions
@saan26phd[§3.4.3].

=== Comparing the update rules for global unknowns <sec:eval-rules>

#let _verdict(key, rule, cond) = {
  let name = "rules-" + key + "-" + rule
```

95 unchanged lines hidden

```
    the run. "No answer in 5 s": stopped by `--timeout 5`.],
) <fig:rules-programs>

== Comparing precision with Goblint <sec:eval-flavors>

#let _gob = {
  let rows = (:)
```

21 unchanged lines hidden

```
interpretation walks through finite lattices, relational domains, widening and
narrowing, and alternatives to iterative solving. For each, it gives a small
program on which a standard analysis gains or loses precision @monniaux24.
Every program of the tutorial that VIMP can express is transliterated into a
regression fixture of
#link(repo-blob + "tests/regression/28-flavors")[`28-flavors`], and Voblint's
verdicts on them are registered as claims. Each check holds in every execution, so each
#verdict("UNKNOWN") below is lost precision.
```

102 unchanged lines hidden

```
because an abstract value may reach the limit of `int` although no execution
does. Goblint then applies its default setting `assume_top` of
#link("https://github.com/goblint/analyzer/blob/6127869c56c61fa184a830a3fcfb7df44de33829/src/config/options.schema.json#L1640")[`sem.int.signed_overflow`], which gives a
signed operation that may overflow the whole range of its type. This happens
for one check, discussed in @sec:flavors-widening.

The settings of @tab:flavors are chosen such that each analyzer has its best
chance on each flavor. Voblint runs Sign, Interval, Int, the reduced product of its four
non-relational domains (@sec:reduced-product), and Int with the order
analysis, its only relational information. No other Voblint option affects
these programs. Its solver has neither widening delays nor thresholds, and
calling contexts and the update rules for global unknowns need procedures,
which a second benchmark compares (@tab:flavors-ctx). The Goblint rows
"delay 10", "thresholds", and "comparisons" apply the repairs the literature
```

```
#_k100.clusters.len() procedure copies. With length 99, the analyzer still
builds #_k99.clusters.len() copies, but the deepest calls share one context.
```

16 unchanged lines hidden

```
explosion by omitting intervals from the contexts of recursive functions
@saan26phd[§3.4.3].

=== Precision of the update rules <sec:eval-rules>

#let _verdict(key, rule, cond) = {
  let name = "rules-" + key + "-" + rule
```

95 unchanged lines hidden

```
    the run. "No answer in 5 s": stopped by `--timeout 5`.],
) <fig:rules-programs>

== Precision compared with Goblint <sec:eval-flavors>

#let _gob = {
  let rows = (:)
```

21 unchanged lines hidden

```
interpretation walks through finite lattices, relational domains, widening and
narrowing, and alternatives to iterative solving. For each, it gives a small
program on which a standard analysis gains or loses precision @monniaux24.
The tutorial programs expressible in VIMP are adapted into regression fixtures in
#link(repo-blob + "tests/regression/28-flavors")[`28-flavors`], and Voblint's
verdicts on them are registered as claims. Each check holds in every execution, so each
#verdict("UNKNOWN") below is lost precision.
```

102 unchanged lines hidden

```
because an abstract value may reach the limit of `int` although no execution
does. Goblint then applies its default setting `assume_top` of
#link("https://github.com/goblint/analyzer/blob/6127869c56c61fa184a830a3fcfb7df44de33829/src/config/options.schema.json#L1640")[`sem.int.signed_overflow`], which gives a
signed operation that may overflow the whole range of its type. Only one check depends on
this setting. @sec:flavors-widening discusses it.

The settings of @tab:flavors are chosen such that each analyzer has its best
chance on each flavor. Voblint runs Sign, Interval, Int, the reduced product of its four
non-relational domains (@sec:reduced-product), and Int with the order
analysis, its only relational information. No other Voblint option affects
these programs. Its solver has neither widening delays nor thresholds, and
calling contexts and the update rules for global unknowns need procedures,
which a second benchmark compares (@tab:flavors-ctx). The Goblint rows
"delay 10", "thresholds", and "comparisons" apply the repairs the literature
```

484	proposes for failed narrowing, and 10 is the smallest delay that proves a	482	proposes for failed narrowing, and 10 is the smallest delay that proves a
485	check Interval leaves unknown. In "Int, delay 3", Goblint's closest	483	check Interval leaves unknown. In "Int, delay 3", Goblint's closest
486	counterpart of Voblint's Int, Interval and Congruence refine each other. The last two rows add Apron's octagons @jeannet09apron, the second together with Goblint's	484	counterpart of Voblint's Int, Interval and Congruence refine each other. The last two rows add the octagon domain of the Apron library @jeannet09apron,
		485	a relational domain of bounds on sums and differences of two variables, the
		486	second row together with Goblint's
487	#link("https://github.com/goblint/analyzer/blob/6127869c56c61fa184a830a3fcfb7df44de33829/src/analyses/branchSet.ml")[`branchSet`] analysis, which keeps	487	#link("https://github.com/goblint/analyzer/blob/6127869c56c61fa184a830a3fcfb7df44de33829/src/analyses/branchSet.ml")[`branchSet`] analysis, which keeps
488	a separate state for each set of branches taken.	488	a separate state for each set of branches taken.
489		489	
	13 unchanged lines hidden		
503	transfer function that contains the initial states. The abstract value that an	503	transfer function that contains the initial states. The abstract value that an
504	analyzer computes at a loop head describes such an invariant, so a check that	504	analyzer computes at a loop head describes such an invariant, so a check that
505	the analyzer proves follows from an inductive invariant that its domain can	505	the analyzer proves follows from an inductive invariant that its domain can
506	express. The converse fails: widening may miss an invariant the domain can	506	express. The converse fails, because widening may miss an invariant the domain can
507	express (@sec:flavors-foiled).	507	express (@sec:flavors-foiled).
508		508	
509	#_flavor("flavors-not-inductive-sign")	509	#_flavor("flavors-not-inductive-sign")
510		510	
511	In this loop from the tutorial @monniaux24[slide 27], `i` takes the values 4	511	Adapting the tutorial loop @monniaux24[slide 27] to start at 4 rather than
512	and 2 at the check, so `i > 0` holds. The property	512	6, `i` takes the values 4 and 2 at the check, so `i > 0` holds. The property
513	is not inductive, however. If all that is known at the loop head is that `i`	513	is not inductive, however. If all that is known at the loop head is that `i`
514	is positive, `i` may be 1, and the next iteration then yields -1. Sign can	514	is positive, `i` may be 1, and the next iteration then yields -1. Sign can
515	express `i > 0` but nothing stronger about `i`, so the loop head reaches every	515	express `i > 0` but nothing stronger about `i`, so the loop head reaches every
	44 unchanged lines hidden		
560		560	
561	#_flavor("flavors-reset-bound-interval")	561	#_flavor("flavors-reset-bound-interval")
562		562	
563	Widening sends the upper bound of `i` to $+infinity$, and narrowing through the	563	Widening sends the upper bound of `i` to $+infinity$; narrowing through
564	test `i >= 10` brings it back to 9 @monniaux24[slide 59]. Both checks are	564	`i >= 10` restores 9. The tutorial uses a machine-integer upper bound
		565	@monniaux24[slide 59]. Both checks are
565	#_vv("flavors-reset-bound-interval", "i <= 9") over	566	#_vv("flavors-reset-bound-interval", "i <= 9") over
566	#absval(raw(claim-check("flavors-reset-bound-interval", "i >= 0").at(4).split("i=").at(1)))	567	#absval(raw(claim-check("flavors-reset-bound-interval", "i >= 0").at(4).split("i=").at(1)))
567	(claim #claim-ref("flavors-reset-bound-interval")). Goblint's	568	(claim #claim-ref("flavors-reset-bound-interval")). Goblint's
	32 unchanged lines hidden		
600	when it delays widening by 10 rounds @blanchet03[§7.1.3], but not by 9, so	601	when it delays widening by 10 rounds @blanchet03[§7.1.3], but not by 9, so
601	the delay must exceed the loop bound. It also proves them with threshold	602	the delay must exceed the loop bound. It also proves them with threshold
602	widening @blanchet03[§7.1.2], but only when the thresholds include the 9 of	603	widening @blanchet03[§7.1.2], but only when the thresholds include the 9 of
603	the check itself. Policy iteration @costan05[§3.2] and guided static analysis	604	the check itself. Policy iteration @costan05[§3.2] solves successive selected policies; guided
604	@gopan07 replace the decreasing iteration altogether. In Voblint, each repair	605	static analysis @gopan07 analyzes growing program fragments and can use
605	would change the vendored solver and its partial-correctness proof.	606	narrowing within each phase. Neither is implemented in Voblint.
606		607	
607	The flavors benchmark has eight checks on small programs, so it compares the	608	The flavors benchmark has eight checks on small programs, so it compares the

```
mechanisms of the two analyzers and says little about their precision on
larger programs. On these checks, Voblint loses no precision to verification
on the mechanisms both analyzers share. With Int and the order
analysis, it proves five of the eight checks of @tab:flavors, and Goblint's
plain Interval proves three. Across all its settings, Goblint proves all eight,
though no single setting does, using widening delays, thresholds, and path
partitioning. Voblint lacks these techniques, and each would bring its own
proof obligation.

=== Procedures <sec:flavors-procedures>
```

9 unchanged lines hidden

```
In _three contexts_, a loop steps `i` by `f(i) = i + 1` and then checks
`i == 10`. The _deep call chain_ consists of recursive procedures that call
each other up to $21$ levels deep, and its ten checks ask for their results. In the _chain
with guards_, `f` calls `g`, `g` calls `h`, and
`h` counts its argument down to $0$ behind the guards `i == 0` and `0 < i`, and all seven
calls, with arguments up to $300$, return $3$. In the
_mutual recursion_, $f(i) = g(i-2) + g(i-3)$ and $g(i) = f(i+1)$, and twelve
checks ask for exact values. The _two-formal countdown_
`f(x, y)` decrements both arguments until `x` reaches $0$ and returns `y`, so its three calls
return $0$.
```

92 unchanged lines hidden

```
  kind: table,
  placement: bottom,
  caption: [Checks proved out of those that hold, on procedure programs adapted
    from Goblint's regression suite (Voblint with Int, Goblint with Interval).
    Voblint: no contexts, also with bounded
    narrowing; call strings of length $k$; entry-state contexts. Goblint
    (#link(repo-blob + "thesis/shared/code/goblint-contexts/results.tsv")[`results.tsv`]):
    context gas $0$, also with narrowing of global unknowns; call strings
    of length $k$; its default full contexts; context gas $10$.],
) <tab:flavors-ctx>

Goblint's context gas $0$, call strings, full contexts, and narrowing of
global unknowns
(#link("https://github.com/goblint/analyzer/blob/6127869c56c61fa184a830a3fcfb7df44de33829/
src/config/options.schema.json#L2595")[`solvers.td3.narrow-globs`])
correspond to Voblint's policy without contexts, its call strings,
```

```
  kind: table,
  placement: bottom,
  caption: [Checks proved out of those that hold, on procedure programs adapted
    from Goblint's regression suite (Voblint with Int, Goblint with Interval,
    #link(repo-blob + "thesis/shared/code/goblint-contexts/results.tsv")[`results.tsv`]).
    Rows are context policies, where _none_ is Goblint's context gas $0$, and
    _call string_ keeps the $k$ most recent call sites.],
) <tab:flavors-ctx>

Goblint's context gas
(#link("https://github.com/goblint/analyzer/blob/6127869c56c61fa184a830a3fcfb7df44de33829/
src/config/options.schema.json#L1061")[`ana.context.gas_value`])
limits how many recursive calls keep full contexts. Gas $10$ keeps them only
for the first ten recursive calls and has no counterpart in Voblint. Gas $0$,
call strings, full contexts, and narrowing of global unknowns
(#link("https://github.com/goblint/analyzer/blob/6127869c56c61fa184a830a3fcfb7df44de33829/
src/config/options.schema.json#L2595")[`solvers.td3.narrow-globs`])
correspond to Voblint's policy without contexts, its call strings,
```

```
entry-state contexts, and bounded narrowing (@sec:update-rules). Context gas $10$
(#link("https://github.com/goblint/analyzer/blob/6127869c56c61fa184a830a3fcfb7df44de33829/
src/config/options.schema.json#L1061")[`ana.context.gas_value`]),
which keeps full contexts only for the first ten recursive calls, has no
counterpart.

With entry-state contexts, Voblint proves every check of all five programs.
Goblint's full contexts prove every check except the one of the three-contexts
program. After its loop,
an interval only bounds `i` to $absval(ivl(10, 11))$. Voblint's Int also knows
that `i` is even, which leaves $10$, and Goblint's configuration has no parity
or congruence component. With bounded contexts,
Goblint proves more on the deep call chain and Voblint more on the mutual
recursion. The two-formal
countdown recurses up to $300$ levels deep and needs a context per level,
which only entry-state contexts provide.

On this benchmark, Voblint's entry-state contexts prove every check that
Goblint's full contexts prove, and one more. With call strings, each analyzer
proves checks the other loses, and without contexts Voblint proves more. Goblint's widening
then loses the lower bound $0$ of `i` in `h` unless it narrows global unknowns
(#link(repo-blob + "thesis/shared/code/goblint-contexts/results.tsv")[`results.tsv`]),
whereas Voblint keeps it and turns `i != 0` into `i > 0`.

== Running time <sec:eval-runtime>

A sound result is of little use if the analyzer takes too long to produce it. Speed,
however, was not a goal of this thesis. Apart from the defect fixed in @sec:codegen, the
generated code runs as Isabelle emits it. The measurements below therefore ask only
whether this unoptimized code runs in acceptable time on the regression corpus.

The #link(site("profiling.html"))[profiling page] reports the measurements in full. The
profiler runs the #prof-int("fixtures") well-formed regression fixtures that set no timeout
of their
own under all #prof-int("configurations") combinations of domain and context policy, with
the default update rule warrow. Each pair of a fixture and a configuration is one case.
```

```
entry-state contexts, and bounded narrowing (@sec:update-rules).

With entry-state contexts, Voblint proves every check of all five programs.
Goblint's full contexts prove every check except the one of the three-contexts
program. After its loop,
an interval only bounds `i` to $absval(ivl(10, 11))$. Voblint's Int also knows
that `i` is even, which leaves $10$, and Goblint's configuration has no parity
or congruence component. With bounded contexts,
Goblint proves more on the deep call chain and Voblint more on the mutual
recursion. The two-formal
countdown recurses up to $300$ levels deep and needs a context per level,
which only entry-state contexts provide.

Without contexts, Voblint proves more checks than Goblint. Goblint's widening
then loses the lower bound $0$ of `i` in `h` unless it narrows global unknowns
(#link(repo-blob + "thesis/shared/code/goblint-contexts/results.tsv")[`results.tsv`]),
whereas Voblint keeps it and turns `i != 0` into `i > 0`.

== Running-time benchmarks <sec:eval-runtime>

#figure(
  scaling-plot(width: 12.5cm, height: 3.2cm),
  kind: image,
  placement: bottom,
  caption: [Fitted exponents $k$ of $n^k$ for solve time and solver evaluations, each family
    growing one program size $n$ under Interval. The fits describe the measured ranges and
    are no complexity bounds.],
) <fig:runtime-scaling>

Speed was not a goal of this thesis. Apart from the defect fixed in @sec:codegen, the
generated code runs as Isabelle emits it, and the measurements below ask only whether it
runs in acceptable time on the regression corpus.

The #link(site("profiling.html"))[profiling page] runs the #prof-int("fixtures")
well-formed regression fixtures without a timeout of their own under all
#prof-int("configurations") combinations of domain and context policy, with the default
update rule warrow, which gives one case per fixture and configuration.
```

```
#if prof-value("timeouts") > 0 [Of the #prof-int("cases") cases, #prof-int("timeouts") pass
  the limit of #prof-int("meta.timeout_s")~s, all of them under
  #prof-list("timeout_contexts") contexts.]
The other #prof-int("cases_solved") cases finish, and each is timed as #prof-runs().
#if prof-value("solve_median_ms") < 5 [Most of them are solved within a few
  milliseconds,] else [Their median solve time is #prof-ms("solve_median_ms"),] except for
recursive programs
under entry-state contexts, where successive calls can create distinct contexts. A procedure
`h` that calls itself $n$ times with
arguments $n, n - 1, dots, 0$ shows the effect. Sign and Parity have only a few abstract
values, so the number of contexts stays bounded and the solve time is
#prof-growth("depth_fits.sign.time"). Congruence, Interval and Int tell the successive
arguments apart and create one context per level. Their solve time grows
#prof-growth("depth_fits.interval.time"), although the number of solver evaluations grows
only #prof-growth("depth_fits.interval.evals").

#if prof-value("scaling.broken").len() == 0 [
  Generated programs that grow other sizes, such as variables or statements, show no such
  gap, so the growth is tied to contexts.
] else if prof-value("scaling.flat").len() == 0 [
  The same gap appears without recursion. The profiling page grows
  #prof-int("scaling.count") sizes of a program independently, from variables and
  statements to call chains. In all of them the number of evaluations grows about linearly,
  while solve time grows as #prof-exp("scaling.time_min") to #prof-exp("scaling.time_max")
  (@fig:runtime-scaling). Each evaluation therefore gets more expensive as the program
  grows, along every size measured.
] else [
  Generated programs that grow one size at a time show the same gap with
  #prof-list("scaling.broken"), while time and evaluations grow alike with
  #prof-list("scaling.flat") (@fig:runtime-scaling).
]
The CPU profiles trace this cost to the data structures of the generated code. Its finite
sets and maps are lists (@sec:codegen), so a membership test scans the list, a union
inserts element by element, and removing duplicates compares every pair. Balanced trees or
hash tables would avoid these scans. The Collections Framework @lammich10collections
refines abstract sets and maps into such structures, and the Light-weight Containers
@lochbihler13containers can replace the list-based default of code generation altogether.

Either route needs only targeted replacements in the code setup and no change to the
soundness proofs.
```

```
#if prof-value("timeouts") > 0 [Of the #prof-int("cases") cases, #prof-int("timeouts") pass
  the limit of #prof-int("meta.timeout_s")~s, all of them under
  #prof-list("timeout_contexts") contexts.]
The other #prof-int("cases_solved") cases finish, and each is timed as #prof-runs().
#if prof-value("solve_median_ms") < 5 [Most of them are solved within a few
  milliseconds,] else [Their median solve time is #prof-ms("solve_median_ms"),] except for
recursive programs
under entry-state contexts, where successive calls can create distinct contexts. For a
procedure `h` that calls itself with arguments $n, n - 1, dots, 0$, Sign and Parity keep
the number of contexts bounded, and the solve time is
#prof-growth("depth_fits.sign.time"). Congruence, Interval and Int tell the successive
arguments apart and create one context per level. Their solve time grows
#prof-growth("depth_fits.interval.time"), although the number of solver evaluations grows
only #prof-growth("depth_fits.interval.evals").

#if prof-value("scaling.broken").len() == 0 [
  Generated programs that grow other sizes, such as variables or statements, show no such
  gap, so the growth is tied to contexts.
] else if prof-value("scaling.flat").len() == 0 [
  The same gap appears in programs that grow one of #prof-int("scaling.count") sizes
  without recursion, from variables to call chains. In all of them the number of evaluations
  grows about linearly,
  while solve time grows as #prof-exp("scaling.time_min") to #prof-exp("scaling.time_max")
  (@fig:runtime-scaling), so each evaluation gets more expensive as the program grows.
] else [
  Generated programs that grow one size at a time show the same gap with
  #prof-list("scaling.broken"), while time and evaluations grow alike with
  #prof-list("scaling.flat") (@fig:runtime-scaling).
]
The CPU profiles trace this cost to the data structures of the generated code. Its finite
sets and maps are lists (@sec:codegen), which membership tests, unions, and duplicate
removal scan element by element. The Collections
Framework @lammich10collections refines abstract sets and maps into balanced trees or hash
tables, and the Light-weight Containers @lochbihler13containers replace the list-based
default of code generation. The former requires refinement proofs, the latter a code-
generation setup.

Neither has been evaluated for Voblint.
```

814	The generated analyzer therefore handles the regression corpus in acceptable time, while	817	On one machine, the generated analyzer thus handles the small programs of the regression
815	its data structures limit how far it scales. These measurements cover small programs on	818	corpus in acceptable time, while its data structures limit how far it scales.
816	one machine and give no performance guarantee beyond them.		
817		819	
818	#figure(		
819	scaling-plot(width: 12.5cm, height: 3.6cm),		
820	kind: image,		
821	placement: bottom,		
822	caption: [Fitted exponents k of n^k for solve time and solver evaluations, each family		
823	growing one program size n under Interval. The fits describe the measured ranges and		
824	are no complexity bounds.],		
825	) <fig:runtime-scaling>		

thesis/content/14-related.typ

77e07947 → 0edeac81

399 old lines, 397 new lines

85% similar lines **+60** **-62** **20 changed blocks**

Old	77e07947	New	0edeac81
1	#import "../lib/code.typ": *	1	#import "../lib/code.typ": *
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/math.typ": ctor	3	#import "../lib/math.typ": ctor
4	#import "../lib/alignment.typ": alignment, alignment-count	4	#import "../lib/alignment.typ": alignment, alignment-count
5		5	
6		6	
7	= Related Work <ch:related>	7	= Related Work <ch:related>
8		8	
9	Most parts of Voblint build on earlier work, from verified analyzers and	9	This chapter separates the parts of Voblint that come from earlier work from the
10	solvers to Goblint and its local-trace semantics. The sections below compare	10	new ones. Most parts
11	each line of work with Voblint and separate what Voblint inherits from what it	11	have a precedent, from verified analyzers and solvers to Goblint and its
12	adds. The literature was found through web and OpenAlex searches. AI agents supported this work heavily, from the search	12	local-trace semantics. The new part is their connection in one mechanized
		13	soundness proof (@sec:rel-summary). The literature was found through web and
		14	OpenAlex searches. AI agents supported this work heavily, from the search
13	to checking each claim against its source	15	to checking each claim against its source
14	(#link(<ai-use>)[statement on the use of generative AI]).	16	(#link(<ai-use>)[statement on the use of generative AI]).
15		17	
16	== Unverified analyzers for C <sec:rel-practice>	18	== Unverified analyzers for C <sec:rel-practice>
17		19	
18	Most analyzers applied to real C programs have no machine-checked soundness	20	The analyzers applied to real C programs are tested rather than proved
19	proof, and they differ from Voblint most in how they treat procedures.	21	(@sec:state-of-art), and they differ from Voblint most in how they treat
		22	procedures.
20	@tab:production-analyzers compares six of them with Voblint. Five of them claim	23	@tab:production-analyzers compares six of them with Voblint. Five of them claim
21	to be sound. Pulse-X is a bug finder instead, and its under-approximating	24	to be sound. Pulse-X is a bug finder instead, and its under-approximating

Old	77e07947	New	0edeac81
22	soundness theorem is proved on paper @le22[pp. 81:2--81:3]. Four of them inline	25	soundness theorem is proved on paper @le22[pp. 81:2--81:3]. Four reanalyze the callee at each call
23	the callee at every call and therefore need no context policy	26	@blanchet03[§5.4] @eva-manual[§5.3] @journault19[§5.1] @brat14[§3]. Inlining
24	@blanchet03[§5.3] @eva-manual[§5.3] @journault19[§5.1] @brat14[§3]. Inlining		
25	cannot handle unbounded recursion, and these tools exclude recursion, unroll it	27	cannot handle unbounded recursion, and these tools exclude recursion, unroll it
26	to a bounded depth, rely on user-written contracts, or assume that a recursive	28	to a bounded depth, rely on user-written contracts, or assume that a recursive
27	call may change any value @blanchet03[§5.3] @eva-manual[§6.3.9]	29	call may change any value @blanchet03[§5.4] @eva-manual[§6.3.9]
28	@mopsa-manual[Limitations] (#link("https://github.com/NASA-SW-VnV/ikos/blob/ac7f7c1738976cabc58c6a53413df6e458995c38/analyzer/README.md#analysis-assumptions")[IKOS README]). Goblint, in	30	@mopsa-manual[Limitations] (#link("https://github.com/NASA-SW-VnV/ikos/blob/ac7f7c1738976cabc58c6a53413df6e458995c38/analyzer/README.md#analysis-assumptions")[IKOS README]). Goblint and
29	contrast, keeps one abstract state per program point and calling context and	31	Voblint, in contrast, keep one abstract state per program point and calling
30	analyzes recursive programs.	32	context (@sec:state-of-art), and Voblint's theorem covers recursive calls.
31		33	
32	Without proof, these tools are tested against independent evidence, such as	34	These tools are tested against independent evidence, such as
33	compiled execution, other analyzers, or the known verdicts of benchmark	35	compiled execution, other analyzers, or the known verdicts of benchmark
34	programs. Such testing keeps finding errors. On eight analyzers for C and C++,	36	programs. Such testing keeps finding errors. On eight analyzers for C and C++,
35	metamorphic testing found 24 issues, 16 of them related to soundness, and	37	interrogation testing found 24 issues, 16 of them related to soundness, and
36	seven of the analyzers had soundness issues, MOPSA among them	38	seven of the analyzers had soundness issues, MOPSA among them
37	@kaindlstorfer24[p. 326, Tab. 2; p. 329].	39	@kaindlstorfer24[p. 326, Tab. 2; p. 329].
38		40	
	21 unchanged lines hidden		
60	[Pulse-X @le22], [C], [bug finding], [summaries], [unrolled], [pure conditions],	62	[Pulse-X @le22], [C], [bug finding], [summaries], [unrolled], [pure conditions],
61	[one domain], [none],	63	[one domain], [none],
62	[Goblint @saan23], [C], [sound], [per context], [analyzed], [Apron], [products, queries],	64	[Goblint @saan23], [C], [sound], [per context], [analyzed], [Apron], [products, queries],
63	[TD algorithm],	65	[TD algorithm @stade24 @tilscher26],
64	table.hline(stroke: 0.4pt),	66	table.hline(stroke: 0.4pt),
65	[*Voblint*], [VIMP], [proved sound], [per context], [covered], [order analysis],	67	[*Voblint*], [VIMP], [proved sound], [per context], [covered], [order analysis],
66	[products, queries], [end to end],	68	[products, queries], [end to end],
	24 unchanged lines hidden		
91		93	
92	== Verified analyzers in the CompCert line	94	== Verified analyzers in the CompCert line
93		95	
94	CompCert and Verasco state their theorems about the tool that users run, and	96	CompCert and Verasco (@sec:state-of-art) state their theorems about the code
95	Voblint follows their pattern for a language with recursive procedures and	97	that actually runs, and Voblint follows their pattern for a language with recursive procedures and
96	context-sensitive calls. CompCert is a compiler from Clight, a large subset of	98	context-sensitive calls. CompCert is a compiler from Clight, a large subset of
97	C, to PowerPC assembly, programmed and proved correct in Coq, now Rocq	99	C, to PowerPC assembly, programmed and proved correct in Coq, now Rocq
98	@leroy09. Its theorem is semantic preservation, so safety properties proved of	100	@leroy09. Its theorem is semantic preservation, so safety properties proved of
	30 unchanged lines hidden		
129		131	
130	== Mechanized abstract interpreters <sec:rel-mechanized>	132	== Mechanized abstract interpreters <sec:rel-mechanized>
131		133	
132	Abstract interpreters have been mechanized in several provers.	134	@tab:state-of-art compares the mechanized abstract interpreters closest to

```
@tab:state-of-art compares those most closely related to Voblint. Their proofs
cover small or intermediate languages, from While and IMP to C\#minor, and the
differences that matter for Voblint lie in recursion, contexts, and the
fixpoint computation. Three of the works have no procedures at all. In every work of the
table, a call is analyzed in at most one context. An
analysis that splits a call into several alternatives, however, needs one
context for each. Voblint's context policy may therefore admit several
contexts for one call, and its semantics and proofs cover this case, although
the shipped analyses do not use it (@sec:contexts). Only Cachera et al. compute the fixpoint
with a verified
solver, and that solver handles ordinary inequations, not side-effecting ones
@cachera05.

#figure(
```

18 unchanged lines hidden

```
      [Cachera, Pichardie @cachera10], [Coq], [While], [no procedures], [none],
      [structural], [yes],
      [Dabrowski, Pichardie @dabrowski09], [Coq], [Java-like], [covered],
      [object-sensitive], [specified], [no],
      [Nipkow, Klein @nipkow14], [Isabelle], [IMP], [no procedures], [none],
      [iterator], [in prover],
      [Lammich, Müller-Olm @lammich07afp], [Isabelle], [flowgraphs], [exact],
```

14 unchanged lines hidden

```
    recursive programs, _exact_ interprets calls and returns exactly, and _no
    procedures_ means the language has none. _Contexts_ says how calls are
    distinguished: _per call site_ reanalyzes a function at every call site,
    _object-sensitive_ computes contexts by a function, _summaries_ computes
    one summary per procedure, _set per call_ lets the context policy return
    several contexts for one call, and _none_ and _intraprocedural_ mean that
    calls are not distinguished or not analyzed. _Fixpoint_ says how the
```

16 unchanged lines hidden

```
activation traces of a procedure-aware control-flow graph (@ch:traces),
because a return must know which caller resumes.

Other works make soundness proofs reusable by sharing one interpreter between
the concrete and the abstract semantics @michelland24 @keidel18 @darais15.
Voblint instead splits analysis, context policy, and solver for a
constraint-based analyzer and composes them in one theorem,
#isathm("activation_collect_dg_sound"). Executable verified analyzers are not
new either. Cachera and Pichardie extract theirs to OCaml @cachera10[§§1, 6],
and the analyzer of Franceschino et al. runs in a browser @franceschino21.
Interprocedural analyses have also been verified in Isabelle before
@lammich07afp @wasserrab09afp @breitner10afp. None of these entries claims an
```

```
@tab:state-of-art compares those most closely related to Voblint. Their proofs cover small or intermediate languages, from While and IMP
to C\#minor, and they differ from Voblint in recursion, contexts, and the
fixpoint computation. Three of the works have no procedures at all. Every work
in the table analyzes a call in at most one context. An analysis that splits a
call into several alternatives, however, needs one context for each. Voblint's
context policy may therefore admit several contexts for one call, and its
semantics and proofs cover this case, although the shipped analyses do not use
it (@sec:contexts). Only Cachera et al. compute the fixpoint with a verified
solver, and that solver handles only ordinary inequations without side effects
@cachera05.

#figure(
```

18 unchanged lines hidden

```
      [Cachera, Pichardie @cachera10], [Coq], [While], [no procedures], [none],
      [structural], [yes],
      [Dabrowski, Pichardie @dabrowski09], [Coq], [Java-like], [covered],
      [functional policy], [specified], [no],
      [Nipkow, Klein @nipkow14], [Isabelle], [IMP], [no procedures], [none],
      [iterator], [in prover],
      [Lammich, Müller-Olm @lammich07afp], [Isabelle], [flowgraphs], [exact],
```

14 unchanged lines hidden

```
    recursive programs, _exact_ interprets calls and returns exactly, and _no
    procedures_ means the language has none. _Contexts_ says how calls are
    distinguished: _per call site_ reanalyzes a function at every call site,
    _functional policy_ computes a context by a function, _summaries_ computes
    one summary per procedure, _set per call_ lets the context policy return
    several contexts for one call, and _none_ and _intraprocedural_ mean that
    calls are not distinguished or not analyzed. _Fixpoint_ says how the
```

16 unchanged lines hidden

```
activation traces of a procedure-aware control-flow graph (@ch:traces),
because a return must know which caller resumes.

Other works derive concrete and abstract interpreters from shared semantic
definitions @michelland24 @keidel18 @darais15.
Voblint instead splits analysis, context policy, and solver for a
constraint-based analyzer and composes them in one theorem,
#isathm("activation_collect_dg_sound"). Executable verified analyzers are not
new either. Cachera and Pichardie extract theirs to OCaml @cachera10[§§1, 6],
and the analyzer of Franceschino et al. runs in a browser @franceschino21.
Other Isabelle developments verify conflict analysis @lammich07afp,
HRB interprocedural slicing @wasserrab09afp, and Shivers' control-flow
```

219	extracted analyzer, however, and none of their semantics represents one	221	analysis @breitner10afp. The slicing result preserves observations by weak
220	activation with its caller chain, the object Voblint's context relation	222	simulation; Voblint bounds stores in each admitted calling context.
221	#isaconst("activation_context_rel") reads.		
222		223	
223	== Verified solvers and termination results <sec:rel-solvers>	224	== Verified solvers and termination results <sec:rel-solvers>
224		225	
225	A verified solver guarantees a property of its result for arbitrary	226	A verified solver guarantees a property of its result for arbitrary
226	equations, and the analyzer must give the equations their meaning. Hofmann et	227	equations, and the analyzer must give the equations their meaning. Hofmann et
227	al. verify the local generic solver RLD in Coq @hofmann10, and de Vilhena et	228	al. verify the local generic solver RLD in Coq @hofmann10, and de Vilhena et
228	al. verify in Iris a local solver that discovers dependencies dynamically	229	al. verify in Iris a local solver that discovers dependencies dynamically
229	@devilhena20. Stade et al. prove the top-down solver partially correct in	230	@devilhena20. Stade et al. prove the top-down solver partially correct in
230	Isabelle/HOL @stade24. Tilscher et al. @tilscher26 extend this line to	231	Isabelle/HOL @stade24. Tilscher et al. @tilscher26 extend this line to
231	$"TD"_"side"$, which accepts contributions to global unknowns, verify it	232	$"TD"_"side"$, which accepts contributions to global unknowns, verify it
232	against a generic interface for update rules, prove five update rules of	233	against a generic interface for update rules, prove five update rules of
233	Stemmler et al. @stemmler25 sound, and refine the solver to executable code.	234	Stemmler et al. @stemmler25 sound, and refine the solver to executable code.
234	For updates without widening and narrowing, they also prove that a terminating	235	For their abort-enabled variant with precise updates and monotonic right-hand
235	solve returns the least partial post-solution, minimal in both valuation and	236	sides, dependencies, and side effects, they also prove minimality of the
236	solved set. CompCert similarly hides its	237	valuation and solved set. CompCert similarly hides its
237	intraprocedural dataflow solvers behind a module type whose clients rely only	238	intraprocedural dataflow solvers behind a module type whose clients rely only
238	on fixpoint properties of the result @compcertKildall @laspina25.	239	on fixpoint properties of the result @compcertKildall @laspina25.
239		240	
240	Termination results exist, but none applies to Voblint. Tilscher et al. prove	241	Tilscher et al. prove termination for top-down solvers without side effects
241	termination for top-down solvers without side effects over a finite type of	242	over a finite set of unknowns, with further hypotheses on widening and
242	unknowns @tilscher26jar[Thm. 2, Cor. 1], whereas Voblint's unknowns pair nodes	243	monotonicity @tilscher26jar[Thm. 2, Cor. 1]. These conditions have not been
243	of an infinite type with contexts (@sec:certificate). Seidl and Vogler prove on	244	established for Voblint (@sec:certificate). Seidl and Vogler prove on
244	paper that their side-effecting solver terminates when side effects target	245	paper that their side-effecting solver terminates when side effects target
245	only unknowns without a right-hand side and only finitely many unknowns are	246	only unknowns without a right-hand side and only finitely many unknowns are
246	encountered @seidl21[§9, Thm. 5]. Their result assumes widening and narrowing	247	encountered @seidl21[§9, Thm. 5]. Their result assumes widening and narrowing
	20 unchanged lines hidden		
267	callee's exit state (@sec:calls), the shape of the return function of	268	callee's exit state (@sec:calls), the shape of the return function of
268	#cite(<knoop92>, form: "prose", supplement: [§4.1]). Reps et al. solve such	269	#cite(<knoop92>, form: "prose", supplement: [§4.1]). Reps et al. solve such
269	problems precisely when the facts are finite and the transfer functions	270	problems precisely when the facts are finite and the transfer functions
270	distributive @reps95, whereas Voblint requires neither and proves no precision	271	distributive @reps95, whereas Voblint requires neither and proves no general
271	result. Seidl and Fecht model procedures by a pushdown automaton that pushes an	272	precision guarantee. Seidl and Fecht model procedures by a pushdown automaton that pushes an
272	activation record at each call and pops it at each return @seidl00[§4], and	273	activation record at each call and pops it at each return @seidl00[§4], and
273	Voblint's activation traces keep this structure per activation (@ch:traces).	274	Voblint's activation traces keep this structure per activation (@ch:traces).
274	The entry-state policy's unknowns, a node	275	The entry-state policy's unknowns, a node
275	paired with the abstract entry state of its procedure, resemble the unknowns	276	paired with the abstract entry state of its procedure, resemble the unknowns
276	(v, d) of their forward accumulation @seidl00[§7.1].	277	(v, d) of their forward accumulation @seidl00[§7.1].
277		278	
278	Trace partitioning abstracts a set of traces by indexing it with control	279	Trace partitioning abstracts a set of traces by indexing it with control
279	history before abstracting states @rival07. Rival and Mauborgne define both	280	history before abstracting states @rival07. Rival and Mauborgne define both
280	partitions and coverings, in which one trace may belong to several indices,	281	partitions and coverings, in which one trace may belong to several indices,

281	and present calling-context sensitivity as one such indexing. They also note	282	and present calling-context sensitivity as one such indexing. Voblint's activation collecting semantics
282	that the function mapping tokens to tokens in a covering may be replaced by a	283	forms an overlapping covering: a context policy may admit one activation in
283	relation @rival07[Rem. 3.2.4]. Voblint's activation collecting semantics	284	several contexts. Voblint mechanizes the indexing over
284	is a covering of this relational kind, since a context policy may admit one		
285	activation in several contexts. Voblint mechanizes the indexing over		
286	activation traces and proves that the solved, routed result bounds the	285	activation traces and proves that the solved, routed result bounds the
287	activation collecting semantics of every admitted context. Contexts also	286	activation collecting semantics of every admitted context. Contexts also
288	determine how many unknowns a solve creates. The context lifters of Erhard et	287	determine how many unknowns a solve creates. The context lifters of Erhard et
	9 unchanged lines hidden		
298		297	
299	=== Correspondence with Goblint's architecture	298	=== Correspondence with Goblint's architecture
300		299	
301	Side-effecting constraint systems let one equation both define a local unknown	300	Goblint uses the side-effecting constraint systems of @sec:side-effects
302	and publish contributions to global unknowns @seidl03 @apinis12. Seidl et al.	301	@seidl03 @apinis12 to decouple a mixed flow-sensitive analysis from the solver.
303	@seidl26 describe how Goblint uses them to decouple a mixed flow-sensitive	302	Digests (@sec:rel-digests) on the analysis side and update rules on the solver
304	analysis from the solver, with digests on the analysis side and update rules on	303	side recover precision @seidl26. Voblint adopts the split. An analysis supplies
305	the solver side, recovering precision. Voblint adopts the split. An analysis	304	the transfer behavior, and the solver applies one of the update rules of
306	supplies the transfer behavior, and the solver organizes the interaction.	305	@sec:update-rules.
307		306	
308	The correspondence with Goblint's implementation is architectural. Voblint was compared	307	The correspondence with Goblint's implementation is architectural. Voblint was compared
309	with the framework interface of Goblint's	308	with the framework interface of Goblint's
	30 unchanged lines hidden		
340	(#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/analyses/mCP.ml")[`mCP.ml`]	339	(#link("https://github.com/goblint/analyzer/blob/5320a6b741e50dc049f7a1b85e1709e9565cc54a/src/analyses/mCP.ml")[`mCP.ml`]
341	at revision `5320a6b7`). Voblint's combination follows this design and adds a	340	at revision `5320a6b7`). Voblint's combination follows this design and adds a
342	proof obligation per analysis that quantifies over every channel that holds.	341	proof obligation per analysis that quantifies over every channel that holds.
343	Unlike a reduced product @cousot79 @rival20[§5.1.2], which Int uses	342	Int uses an explicit reduced-product operator @cousot79 @rival20[§5.1.2]
344	(@sec:reduced-product), the product state keeps separate carriers and	343	(@sec:reduced-product). The cooperating analyses exchange facts through
345	exchanges facts only through answers, so no reduction operator has to be	344	queries, without a pair-specific reduction operator. Verasco threads its channels so that the second
346	proved for a pair of analyses. Verasco threads its channels so that the second		
347	domain can query the state the first has just computed @jourdan15[§7], whereas	345	domain can query the state the first has just computed @jourdan15[§7], whereas
348	Voblint's answers describe the predecessor state only, as Goblint's do.	346	Voblint's answers describe the predecessor state only, as Goblint's do.
349		347	
350	=== Digests and the context relation	348	=== Digests and the context relation <sec:rel-digests>
351		349	
352	Local traces give each thread of a concurrent program a semantics against which	350	Local traces give each thread of a concurrent program a semantics against which
353	thread-modular analyses are proved sound and compared @schwarz21, and they	351	thread-modular analyses are proved sound and compared @schwarz21, and they
	43 unchanged lines hidden		
397	before, namely a soundness proof that leads from a source semantics, through the	395	before, namely a soundness proof that leads from a source semantics, through the
398	activation collecting semantics, which indexes activation traces by context, to a	396	activation collecting semantics, which indexes activation traces by context, to a
399	side-effecting constraint system and its verified TD solver.	397	side-effecting constraint system and its verified TD solver.

thesis/content/15-conclusion.typ

77e07947 → 0edeac81

231 old lines, 215 new lines

65% similar lines **+69** **-85** **19 changed blocks**

Old	77e07947	New	0edeac81
1	#import "../lib/code.typ": isaconst, isalocale, isathm, isatype, oblig, repo-blob, site	1	#import "../lib/code.typ": isaconst, isalocale, isathm, isatype, oblig, repo-blob, site
2	#import "../lib/figures.typ": verdict	2	#import "../lib/figures.typ": verdict
3	#import "../lib/stats.typ": stat	3	#import "../lib/stats.typ": stat
4		4	
5	= Conclusion <ch:conclusion>	5	= Conclusion <ch:conclusion>
6		6	
7	Static analyzers such as Goblint report properties intended to hold for every	7	Voblint shows that the soundness of a Goblint-style analyzer with calling
8	execution of a program, but their implementations generally lack machine-checked	8	contexts, side effects, and a verified TD solver can be machine-checked from
9	end-to-end soundness proofs. This thesis shows that the soundness of a	9	source executions to the verdicts of the analysis function. The result covers
10	Goblint-style analyzer with calling contexts, side effects, and a verified TD	10	VIMP, a scalar language with recursive procedures. It is a partial-correctness
11	solver can be machine-checked from source executions to the verdicts of the	11	result, so its guarantee holds whenever the analyzer returns an answer. For a
12	analysis function. The result holds for a scalar language with recursive	12	user, a #verdict("PROVED") verdict therefore establishes the reported property
13	procedures. It is a partial-correctness result, so its guarantee holds whenever	13	for every covered execution that reaches the check, instead of resting on the
14	the analyzer returns an answer. For a user, a #verdict("PROVED") verdict therefore	14	tests the analyzer has passed.
15	establishes the reported property for every covered execution that reaches the		
16	check, instead of resting on		
17	the tests the analyzer has passed.		
18		15	
19	The current implementation of Voblint lies between existing formalized	16	The current implementation of Voblint lies between existing formalized
20	analyzers, such as Verasco @jourdan15 and the abstract interpreter of Nipkow and	17	analyzers, such as Verasco @jourdan15 and the abstract interpreter of Nipkow and
21	Klein @nipkow14, and Goblint. The formalized analyzers state their theorem about	18	Klein @nipkow14, and Goblint. The formalized analyzers state their theorem about
22	the analyzer they run, but avoid the mechanisms Goblint relies on for	19	the analyzer they run, but avoid the mechanisms Goblint relies on for
23	interprocedural precision. Verasco reanalyzes a function at every call site	20	interprocedural precision. Verasco reanalyzes a function at every call site
24	and raises an alarm at a possible recursive call, and the interpreter of	21	and raises an alarm at a possible recursive call, and the interpreter of
25	Nipkow and Klein analyzes IMP, which has no procedures. Goblint instead solves	22	Nipkow and Klein analyzes IMP, which has no procedures. Goblint instead solves
26	equations over unknowns indexed by program point and calling context, publishes	23	equations over unknowns indexed by program point and calling context, publishes
27	side effects to global unknowns, and computes the solution with a top-down	24	side effects to global unknowns, and computes the solution with a top-down
28	solver. The TD solver itself has been verified @tilscher26, but no proof connects	25	solver. A formalization of TD has been verified @tilscher26, but no proof connects
29	Goblint's solved equations to the executions of the program. Voblint keeps Goblint's architecture and	26	Goblint's solved equations to the executions of the program. Voblint keeps Goblint's architecture and
30	states a CompCert-style theorem about the analyzer it runs. As far as the author is aware, a proof that connects context-indexed,	27	states a CompCert-style theorem about the analyzer it runs. As far as the author is aware, a proof that connects context-indexed,
31	side-effecting equations and their solver to source executions has not been	28	side-effecting equations and their solver to source executions has not been
	20 unchanged lines hidden		
52	acceptable time on that corpus, although its list-based sets and maps limit how far it	49	acceptable time on that corpus, although its list-based sets and maps limit how far it
53	scales (@sec:eval-runtime).	50	scales (@sec:eval-runtime).
54		51	
55	== Lessons from the formalization <sec:lessons>	52	== Recommendations for similar formalizations <sec:lessons>
56		53	
57	Formalizing Voblint required more than proving the components of an analyzer	54	Much of the work on Voblint went into finding definitions and interfaces that

58	sound. Much of the work went into finding definitions and interfaces that let	55	let the component proofs compose, and into checking that the theorems describe
59	these proofs compose, and into checking that the theorems describe the intended	56	the intended analyzer. The following recommendations for a similar
60	analyzer. The following observations summarize this experience and suggest what	57	formalization come from this experience.
61	a similar formalization might do differently.		
62		58	
63	*Reuse the architecture, but reconsider the model.* Goblint's architecture is	59	*Reuse the architecture, but reconsider the model.* Goblint's architecture is
64	documented in the literature and has been exercised on real programs. Following	60	documented in the literature and has been exercised on real programs. Following
	16 unchanged lines hidden		
81	following Goblint's architecture, with every proof obligation left open as	77	following Goblint's architecture, with every proof obligation left open as
82	`sorry`. This	78	`sorry`. This
83	made it inexpensive to try definitions, find missing assumptions, and move	79	made it inexpensive to try definitions, find missing assumptions, and move
84	interfaces without repairing finished proofs. The largest revisions indeed	80	interfaces without repairing finished proofs. Because Goblint's structure fixed
85	concerned the semantics rather than individual arguments. Because Voblint	81	most interfaces early, the largest revisions concerned the semantics, and later
86	followed Goblint's structure, most interfaces were settled before the proofs	82	changes seldom reached finished proofs. The end-to-end theorem itself, however,
87	were written, and later changes seldom reached finished proofs. The end-to-end	83	took shape only as the components were connected.
88	theorem itself, however, took shape only as the components were connected.		
89	Stating it first and deriving each component's obligations from it would likely	84	Stating it first and deriving each component's obligations from it would likely
90	have saved some rework.	85	have saved some rework.
91		86	
	9 unchanged lines hidden		
101	function #isaconst("run_voblint"), and the command-line tool, the playground, and	96	function #isaconst("run_voblint"), and the command-line tool, the playground, and
102	the regression tests all run code generated from this function. Regression tests committed to the repository	97	the regression tests all run code generated from this function. Regression tests committed to the repository
103	record the expected verdicts on representative programs and detect changes in	98	record the expected verdicts on representative programs and detect changes in
104	the analyzer's observed precision. Generated theorem statements, registered	99	the analyzer's observed precision. Generated theorem statements, analyzer
105	claims, and repository checks keep the thesis and documentation from drifting	100	output that every build recomputes, and repository checks keep the thesis and
106	away from the theories (@sec:voblint-repo).	101	documentation consistent with the theories (@sec:voblint-repo).
107		102	
108	*Make the solver observable.* The	103	*Make the solver observable.* The
109	#link(repo-blob + "cli")[command-line tool] and the browser	104	#link(repo-blob + "cli")[command-line tool] prints a solver trace, and the browser
110	#link(site("playground.html"))[playground] were also important for understanding the analyzer's behavior. The	105	#link(site("playground.html"))[playground] renders the graph and replays the
111	solver trace of the command-line tool and the playground's rendered graph and	106	trace (@sec:tracing). Both show where an analysis loses precision or where the
112	trace replay (@sec:tracing) show where an analysis loses precision or where the		
113	solver does needless work. The trace hook lies outside the proof. Only traced runs execute it, and	107	solver does needless work. The trace hook lies outside the proof. Only traced runs execute it, and
114	for them the trusted base assumes that it leaves the analysis results unchanged	108	for them the trusted base assumes that it leaves the analysis results unchanged
115	(@sec:tracing). Because the analyzer is exported to OCaml, standard profiling and	109	(@sec:tracing). Because the analyzer is exported to OCaml, standard profiling and
116	debugging tools also remain available. The author used them to profile the	110	debugging tools also remain available. The author used them to profile the
117	generated code and to attach a debugger to solves that did not terminate. The trace of	111	generated code and to attach a debugger to solves that did not terminate. The trace of
118	#link("https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis/issues/251") [issue 251],	112	#link("https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis/issues/251") [issue 251],
119	for instance, showed every newly entered context solved once from bot before	113	for instance, showed every newly entered context solved once from bot before
120	its seed was published, and	114	its seed was published, and

121	#link(	115	#link(
122	"https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis/pull/256",	116	"https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis/pull/256",
123	)[pull request 256]	117	)[pull request 256]
124	reordered the two, as Goblint does.	118	reordered the two into Goblint's order.
125		119	
126	*Proof correctness and proof quality require different checks.* Isabelle checks	120	*Proof correctness and proof quality require different checks.* Isabelle checks
127	that the theorems follow from the definitions and assumptions, but not that the	121	that the theorems follow from the definitions and assumptions. Human review could
128	definitions model the intended analyzer. Once the definitions and interfaces had	122	therefore concentrate on the definitions, the assumptions, and the intended
129	stabilized, generative AI tools produced many of the proofs. Isabelle checked	123	meaning of the theorems, while AI agents produced many of the proofs
130	their validity relative to the formal assumptions, while human review focused on	124	(#link(<ai-use>)[statement on the use of generative AI]). Acceptance by Isabelle,
131	the definitions, the assumptions, and the intended meaning of the theorems. Acceptance by Isabelle does not make a proof readable	125	however, does not make a proof readable or reusable. Unused and duplicated
132	or reusable, however. Unused and duplicated lemmas accumulated during the	126	lemmas accumulated during the development, and some arguments stayed more
133	development, and some arguments stayed more specialized than necessary. To find	127	specialized than necessary. To find such problems, the author developed
134	such problems, the author developed		
135	#link("https://github.com/ManuelLerchner/isar-tools")[`isar-tools`] alongside	128	#link("https://github.com/ManuelLerchner/isar-tools")[`isar-tools`] alongside
136	the formalization (@sec:isar-tools). It	129	the formalization (@sec:isar-tools). It detects duplicated, subsumed, and unused
137	detects maintenance issues such as duplicated or subsumed lemmas, unused		
138	lemmas, leftover proof-search and diagnostic commands, redundant brackets, and	130	lemmas, leftover proof-search and diagnostic commands, redundant brackets, and
139	inconsistent indentation.	131	inconsistent indentation. Its checks are deterministic and complete in seconds,
140	Full Isabelle builds and these maintenance checks complement each other. A full	132	whereas a full build of the development takes several minutes on the author's
141	build of the development takes several minutes on the author's laptop (an Apple M4 Pro),	133	laptop (an Apple M4 Pro). Agents that edit the theories can therefore run them
142	whereas the checks of `isar-tools` are deterministic and complete in seconds.	134	after individual changes, while full Isabelle builds remain necessary to
143	Agents that edit the theories can therefore run them after individual changes,	135	establish correctness.
144	while full Isabelle builds remain necessary to establish correctness.		
145		136	
146	In retrospect, three questions required different kinds of evidence. The	137	In retrospect, three questions required different kinds of evidence. The
147	soundness theorem establishes that the analysis results satisfy their formal	138	soundness theorem establishes that the analysis results satisfy their formal
148	specification. The design of the semantics, the comparison with Goblint, and	139	specification. The design of the semantics, the comparison with Goblint, and
149	executable examples support the adequacy of that specification, which proved	140	executable examples support the adequacy of that specification, which proved
150	the most consequential challenge. Benchmarks, regression tests, tracing, and	141	the most consequential challenge. Benchmarks, regression tests, tracing, and
151	profiling revealed the analyzer's practical behavior, including precision losses	142	profiling revealed the analyzer's practical behavior, including precision losses
152	and needless solver work, which the soundness theorem does not address.	143	and needless solver work, which the soundness theorem does not address.
153		144	
154	== Relevance to Goblint and limitations <sec:limitations>	145	== Relevance to Goblint and limitations <sec:limitations>
155		146	
156	The proofs do not show that Goblint is correct, since they concern	147	The proofs concern Voblint's definitions, and the correspondence with Goblint
157	Voblint's definitions and the correspondence with Goblint is architectural	148	is architectural (@sec:rel-goblint), so they do not establish that Goblint is
158	(@sec:rel-goblint). They show, however, that Goblint's architecture admits a	149	correct. They show that Goblint's architecture admits a soundness proof that
159	soundness proof that composes along its own boundaries, provided each analysis	150	composes along its own boundaries, provided each analysis and each context
160	and each context policy meets its obligations.	151	policy meets its obligations.
161		152	
162	A documented Goblint defect can be replayed as a regression	153	The formalization can also serve Goblint's development directly. A documented

163	fixture and as a counterexample theorem, as the congruence remainder was	154	Goblint defect can be replayed as a regression fixture and as a counterexample
164	(@sec:eval-1161). A proposed domain operation with a counterpart in Voblint	155	theorem (@sec:eval-1161). A proposed domain operation with a counterpart in Voblint
165	could be checked against the obligation that its counterpart meets before the	156	could be checked against the obligation that its counterpart meets before the
166	change lands in Goblint. Conversely, a verified component exported as	157	change lands in Goblint. Conversely, a verified component exported as
167	OCaml could replace its unverified counterpart in Goblint.	158	OCaml could replace its unverified counterpart in Goblint.
168		159	
169	For Goblint's developers, however, the current formalization is still largely	160	For Goblint's developers, however, the current formalization is still largely
170	a proof of concept. It lacks many concepts that their work depends on, such as	161	a proof of concept. It lacks many concepts that their work depends on, such as
171	pointers, a heap, threads, machine integers, and most of Goblint's analyses, so its	162	pointers, a heap, threads, machine integers, and most of Goblint's analyses
172	immediate applicability to Goblint development is limited, although it already	163	(@sec:outlook-extending). Among the limitations identified in @sec:intro-scope,
173	provides a reference for selected components and proof obligations	164	the gap between VIMP and C is the largest. Because VIMP lacks these features and
174	(@sec:outlook-extending). Among the limitations identified in @sec:intro-scope, the gap between VIMP and	165	computes with unbounded integers, a verdict about a VIMP program
175	C is the largest. VIMP has no pointers, heap, or threads and	166	does not carry over to a C program with the same text (@sec:vimp-vs-c). The
176	computes with unbounded integers, so a verdict about a VIMP program does not	167	adequacy of #isaconst("pstep") itself is argued rather than proved. Relating VIMP to AutoCorres2 @brecknell24autocorres2, Isabelle/C
177	carry over to a C program with the same text (@sec:vimp-vs-c), and the	168	@tuong19isabellec, CH2O @krebbers15phd, or Cerberus @memarian23phd would require
178	adequacy of #isaconst("pstep") itself is argued rather than proved. As far as the author is aware, no commonly	169	a verified translation for a common fragment.
179	accepted C11 semantics exists in Isabelle against which #isaconst("pstep")		
180	could be checked @brecknell24autocorres2 @tuong19isabellec @krebbers15phd		
181	@memarian23phd.		
182		170	
183	A second limitation concerns the evaluation of precision. No precision or completeness theorem is proved, so the evidence for precision	171	A second limitation concerns the evaluation of precision. No general precision or completeness theorem is proved, so the evidence for precision
184	rests on examples. The regression corpus is small and its expected verdicts were set by hand (@sec:eval-corpus),	172	rests on examples. The regression corpus is small and its expected verdicts were set by hand (@sec:eval-corpus),
185	and beyond the small comparison of @sec:eval-flavors, no agreement rate with	173	and beyond the small comparison of @sec:eval-flavors, no agreement rate with
186	Goblint is measured (@sec:rel-goblint).	174	Goblint is measured (@sec:rel-goblint).
187		175	
188	== Extending Voblint towards C and Goblint <sec:outlook-extending>	176	== Extensions towards C and Goblint <sec:outlook-extending>
189		177	
190	Addressing these limitations requires extending Voblint towards the language	178	Addressing these limitations requires extending Voblint towards the language
191	features and analyses that Goblint supports. Some analyses and concepts could be added to the current proof of concept	179	features and analyses that Goblint supports. Some analyses and concepts could be added to the current proof of concept
192	without much change, since each soundness proof stays within its own	180	without much change, since each soundness proof stays within its own
193	obligations. For example, relational domains such as octagons or polyhedra @mine17	181	obligations. For example, relational domains such as octagons or polyhedra @mine17
194	could follow the order analysis (@sec:relational), and path-sensitive	182	could follow the order analysis (@sec:relational), and path-sensitive
195	analyses could split a call into several entry pairs, which the interface	183	analyses could split a call into several entry pairs, which the interface
196	already admits (@sec:calls). Other features of Goblint, such as machine	184	already admits (@sec:calls). Other features of Goblint, such as machine
197	integers, pointers, or threads, would require extensive changes to Voblint. Machine integers illustrate the scale of such an extension.	185	integers, pointers, or threads, would require extensive changes to Voblint. Machine integers illustrate the scale of such an extension.
198	#link("https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis/pull/357")[Pull request 357],	186	#link("https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis/pull/357")[Pull request 357],

199	developed separately from the version this thesis describes, introduces	187	developed separately from the version this thesis describes, introduces
200	machine-integer kinds from `i8` to `u64` and a typed core language elaborated	188	machine-integer kinds from `i8` to `u64` and a typed core language translated from VIMP. Integrating it requires revisiting several layers and their
201	from VIMP. Integrating it requires revisiting several layers and their		
202	soundness proofs, which is left as future work. With machine integers, Goblint's bitfield domain, which tracks each	189	soundness proofs, which is left as future work. With machine integers, Goblint's bitfield domain, which tracks each
203	bit separately and which the author co-developed	190	bit separately and which the author co-developed
204	(#link("https://github.com/goblint/analyzer/pull/1623")[Goblint pull request 1623]),	191	(#link("https://github.com/goblint/analyzer/pull/1623")[Goblint pull request 1623]),
205	is a natural next analysis.	192	is a natural next analysis.
206		193	
207	Whether a further C feature needs such a rewrite depends on how deeply it	194	Whether a further C feature needs such a rewrite depends on how deeply it
208	reaches into the formalization. Several central results are independent of the details of VIMP. The solver certificate is stated over unknowns and right-hand sides, and	195	reaches into the formalization. Several central results are independent of the details of VIMP. The solver certificate is stated over unknowns and right-hand sides, and
209	the coverage contract refers to the language only through a few step functions	196	the coverage contract refers to the language only through a few step functions
210	(@sec:contract), so a feature that changes only these step functions leaves	197	(@sec:contract), so a feature that changes only these step functions leaves
211	the rest of the proof intact. The trace semantics, however, assumes that a callee cannot touch its caller's	198	the rest of the proof intact. The trace semantics, however, assumes that a callee cannot touch its caller's
212	locals, so a return can	199	locals, so a return can
213	resume the caller with the locals it had before the call (@sec:calls).	200	resume the caller with the locals it had before the call (@sec:calls).
214	Features that keep this assumption, such as machine integers, change the	201	Features that keep this assumption, such as machine integers, change the
215	step functions and the proofs that unfold them, while the contract and its	202	step functions and the proofs that unfold them, while the contract and its
216	composition keep their shape. Pointers, shared mutable memory, and threads would invalidate this assumption. With	203	composition keep their shape. A heap could be modeled as shared state with new transfers and proofs.
217	a pointer into the caller's frame or into the heap, a callee can modify data the	204	Pointers into caller frames break the separation of locals @sotin11; threads
218	caller reads after the return, and with threads, other code can run between a	205	add interference @schwarz21. These features require revisiting the trace
219	call and its return. Such features	206	semantics and its soundness argument.
220	would require extending the trace semantics @sotin11 @schwarz21, and the		
221	soundness arguments that depend on its current separation of caller and callee		
222	state would need substantial revision.		
223		207	
224	Revisions of this scale may become more feasible as AI-assisted tools improve. Such tools	208	Revisions of this scale may become more feasible as AI-assisted tools improve.
225	have already made large-scale mathematical formalization practical	209	Such tools have already made large-scale mathematical formalization practical
226	@hales26lean[Sec. "Autoformalization is a practical reality"] and played an important role	210	@hales26lean[Sec. "Autoformalization is a practical reality"], and they produced
227	in the formalization presented in this thesis. Over the course of this project, the models	211	many of the proofs in this development. Over the course of
228	behind these tools, including Anthropic's Claude, OpenAI's ChatGPT and Codex, and Cursor,	212	this project, the models behind these tools handled the formalization
229	handled the formalization consistently better, and the surrounding	213	consistently better, and the surrounding Isabelle tooling improved as well.
230	Isabelle tooling improved as well. Continued progress in both areas may make ambitious formalization projects	214	Continued progress in both areas may bring the revision of the trace semantics
231	and large-scale refactorings increasingly practical.	215	for pointers and threads within reach.

thesis/content/abstract.typ

77e07947 → 0edeac81

64 old lines, 52 new lines

12% similar lines **+45** **-57** **6 changed blocks**

Old	77e07947	New	0edeac81
1	#import "../lib/code.typ": site		
2	#import "../lib/stats.typ": stat	1	#import "../lib/stats.typ": stat
3	#import "../lib/figures.typ": verdict		
4		2	
5	= Abstract <abstract>	3	= Abstract <abstract>
6		4	
7	An abstract interpreter over-approximates a program's executions, so any	5	A static analyzer predicts properties of a program without running it, for
8	property that holds for the over-approximation holds in every run. Goblint, an	6	example, whether a division can divide by zero, whether an array index stays
9	abstract interpreter for C, builds a control-flow graph of the program,	7	within bounds, or whether an assertion always holds. Such an analyzer is _sound_ if
10	translates the graph into a system of equations over abstract states, and solves	8	its answers hold in every execution of the program. Developers and verification
11	that system with a generic fixpoint solver @vojdani16 @seidl26. Its guarantee	9	tools rely on this guarantee, but commonly used analyzers are themselves complex programs that
12	therefore depends on four conditions: a control-flow graph that simulates the	10	may contain bugs. If such a bug makes the analyzer report a property that does not
13	source program, equations that over-approximate its executions in every calling	11	hold, its users miss exactly the error they wanted to find. Machine-checked
14	context, a solver result that is a post-solution of the equations, and reported	12	soundness proofs drastically reduce the likelihood of such bugs.
15	verdicts that follow from that result. For Goblint's top-down solver, only the third		
16	condition had been verified @stade24 @tilscher26.		
17		13	
18	Voblint is an Isabelle/HOL formalization of a context-sensitive	14	This thesis presents Voblint, short for verified Goblint, a static analyzer
19	interprocedural analyzer. Its soundness proof covers all four conditions for	15	modeled on the Goblint analyzer for C programs. Like Goblint, Voblint translates the program into a
20	VIMP, a small C-like language with global and local integer variables and	16	control-flow graph, derives a system of equations from the graph, and solves
21	recursive procedures with parameters and return values. The end-to-end soundness theorem states	17	the system with a generic solver. Previous work proved a formalization of the solver algorithm
22	that every report the analyzer returns covers every store that a	18	partially correct @stade24 @tilscher26, so whenever it terminates, its result
23	finite execution of the program reaches from an initial store, and that every	19	satisfies the equations. This alone does not make the analyzer sound. The graph
24	#verdict("PROVED") or #verdict("REFUTED") verdict at the corresponding program	20	also has to cover every execution of the program, and the equations have to
25	point holds for that store. This is partial correctness, since termination of	21	account for these executions in every calling context. Finally, the reported
26	the solver is not proved. A check reported #verdict("DEAD") is also proved	22	answers have to follow from the solver's result. Voblint proves all of these
27	unreachable. At a point without an arithmetic warning or error, no divisor is	23	properties together for VIMP, a small C-like language with integer variables and
28	zero, and a divisor reported as an error is zero wherever its point is reached.	24	recursive procedures, defined for this work. The proof, developed in the proof assistant
29	To the author's knowledge, no prior mechanized analyzer has a soundness proof that	25	Isabelle/HOL, thus covers the analysis from the parsed program to the reported
30	connects a source semantics, through a context-indexed collecting semantics, to	26	answers. The main theorem states that the reported answers hold in every state
31	a side-effecting constraint system and its verified solver.	27	that a finite execution of the program reaches. For example, if the analyzer proves an
		28	assertion, the assertion holds whenever an execution reaches it. Likewise, the
		29	analyzer is proved to flag every program point where a division can divide by
		30	zero. This guarantee applies whenever the analyzer finishes, since
		31	termination of the solver is not proved for every program.
32		32	
33	The main difficulty is modeling context sensitivity. A context-sensitive	33	Voblint distinguishes calls to the same procedure by their calling contexts,
34	analyzer computes a separate abstract state for each program point and calling	34	avoiding the loss of precision that would result from merging their values. To prove
35	context. Soundness proofs usually compare them with the collecting semantics,	35	these per-context results sound, Voblint introduces _activation traces_, a
36	the stores reachable at each program point. This semantics, however, merges the	36	semantics that distinguishes individual procedure activations and relates them

Old	77e07947	New	0edeac81
37	stores of all calls to a procedure, so it cannot say which stores a given	37	to their calling contexts. This
38	context must cover. Voblint therefore introduces activation traces, inspired by	38	construction fits into a modular soundness proof that follows Goblint's
39	the local traces of Schwarz et al. @schwarz21. Each trace follows one procedure	39	architecture. Abstract domains, analyses,
40	activation, and a context policy decides to which contexts it belongs. If the	40	and context policies each prove their own soundness properties, and a single
41	policy assigns at least one context to every call that an execution reaches,	41	theorem combines them with the solver's correctness. The resulting guarantee
42	every execution belongs to some context, and the per-context sets of stores	42	holds for every analysis configuration that Voblint offers. To our knowledge, no
43	together equal the collecting semantics.	43	earlier mechanized analyzer combines recursive procedures, calling contexts,
		44	and a verified solver in one soundness proof.
44		45	
45	The proof is modular and follows Goblint's architecture. Domains, context	46	The verified analyzer is exported to executable code, runs in a command-line
46	policies, analyses, and the solver each have their own proof obligations, and	47	tool and a browser playground, and is tested on #stat("corpus.cases") regression
47	one soundness theorem composes them. A numeric domain only has to implement a	48	programs. Proofs on concrete programs show that the assumptions of the main
48	shared interface and prove its abstract operations sound. The framework builds	49	theorem can be met, so the theorem does not hold vacuously. The proof does not
49	transfer functions from these operations and proves them sound once for all	50	cover parsing, presentation of results, or code generation. Because the theorem
50	domains, as in Nipkow and Klein's generic abstract interpreter	51	concerns VIMP rather than C, the thesis also documents where the two languages
51	@nipkow14[§13.5]. This theorem holds for every offered choice of analysis,	52	differ.
52	context policy, and placement of program globals.		
53			
54	The analysis function is exported to OCaml, runs in a command-line tool and		
55	a		
56	#link(site("playground.html"))[browser		
57	playground], and is tested on #stat("corpus.cases") regression fixtures. Proofs		
58	by evaluation show that the theorem is non-vacuous on concrete programs.		
59			
60	The analyzer runs inside unverified handwritten OCaml glue and driver code,		
61	including the parser and the rendering. This code, Isabelle's code		
62	generator, and the OCaml and WebAssembly toolchains are trusted and tested		
63	conventionally using unit and regression tests. Because the theorem concerns VIMP, VIMP's differences from		
64	C11 @iso-c11 are documented.		

thesis/content/ai-use.typ

77e07947 → 0edeac81

128 old lines, 122 new lines

50% similar lines +59 -65 **12 changed blocks**

Old	77e07947	New	0edeac81
1	#import "../lib/stats.typ": _grouped as _n	1	#import "../lib/stats.typ": _grouped as _n
2		2	
3	// Aggregates only, written by tools/ai_use_stats.py from local session logs.	3	// Aggregates only, written by tools/ai_use_stats.py from local session logs.
4	#let _ai = json("/shared/generated/ai-use-stats.json")	4	#let _ai = json("/shared/generated/ai-use-stats.json")
5		5	
6	= Use of Generative AI <ai-use>	6	= Use of Generative AI <ai-use>
7		7	

Old	77e07947	New	0edeac81
8	The Leiden Declaration on Artificial Intelligence and Mathematics recommends	8	The Leiden Declaration on Artificial Intelligence and Mathematics recommends
9	that authors transparently disclose the automated tools they use, including	9	that authors transparently disclose the automated tools they use, including
10	large language models @leiden26. This section follows that recommendation.	10	large language models @leiden26. This section follows that recommendation.
11		11	
12	Substantial parts of this work were carried out with generative AI tools,	12	Substantial parts of this work were carried out with generative AI tools,
13	primarily Anthropic's Claude, OpenAI's ChatGPT and Codex, and Cursor. Under the	13	primarily Anthropic's Claude, OpenAI's ChatGPT and Codex, and Cursor. Under the author's
14	author's supervision, these tools contributed a large share of the	14	supervision, the tools produced a large share of the implementation-level
15	implementation-level material. The author chose the research direction,	15	material.
16	selected and revised the designs, and approved the definitions and theorem		
17	statements.		
18		16	
19	In the Isabelle development, the tools generated many proofs, carried out	17	The author's own contribution lies mainly in the design of the formalization
		18	and in the review of what it proves. On the design side, this
		19	includes modeling the overall architecture on Goblint's and adapting the local
		20	traces of Schwarz et al. @schwarz21 to activation traces. It also includes
		21	splitting soundness into independent obligations for the abstract operations,
		22	the calling contexts, and the solver. The review concerns the definitions,
		23	assumptions, and theorem statements, which Isabelle cannot check against their
		24	intended meaning.
		25	
		26	In the Isabelle development, AI agents generated many proofs, carried out
20	refactorings across theories, and transformed tactic-style proofs and	27	refactorings across theories, and transformed tactic-style proofs and
21	Sledgehammer output into structured Isar. They proposed designs and drafted	28	Sledgehammer output into structured Isar. Once the definitions and locale
22	planning documents for larger changes. For the thesis, they assisted with	29	boundaries had stabilized, many of the remaining obligations followed from the
23	literature searches, checked citations and claims against cited sources, and	30	invariants in a few steps, and agents discharged them with little guidance.
24	drafted and revised the chapter structure, text, and figures.	31	Agents also proposed designs and drafted planning documents for larger changes,
		32	which the author accepted, changed, or rejected. For the thesis, agents assisted with literature searches,
		33	checked citations and claims against the cited sources, and drafted and revised
		34	the chapter structure, text, and figures under the author's review. Agents also
		35	wrote the command-line tool and the project site.
25		36	
26	Separately, Grammarly was used to check spelling and grammar in the thesis text.	37	Separately, Grammarly was used to check spelling and grammar in the thesis text.
27	The Isabelle interfaces of AutoCorrode @autocorrode and the Isabelle PIDE MCP server	38	The Isabelle interfaces of AutoCorrode @autocorrode and the Isabelle PIDE MCP server
28	@pidemcp served as proof-development infrastructure, and the proof work followed the human-guided workflow proposed	39	@pidemcp served as proof-development infrastructure, and the proof work followed the human-guided workflow proposed
29	by Kappelmann et al. @kappelmann26.	40	by Kappelmann et al. @kappelmann26.
30		41	
31	The assistance also extended to design. To formalize calling contexts, the	42	For the analysis framework, agents inspected Goblint's OCaml sources to
32	concrete semantics is based on traces instead of states, inspired by the local	43	identify interfaces the formalization could approximate (@sec:rel-goblint).
33	traces of Schwarz et al. @schwarz21. Language models helped adapt the	44	Examples are local and global unknowns, unknowns indexed by node and context,
34	thread-local setting of that work to procedure activations. The requirements for	45	and the enter/combine protocol at calls. The source language and its
35	the activation traces of @sec:activation-trace were evaluated and refined in	46	compilation, the activation traces, the coverage contract, and the

Old	77e07947	New	0edeac81
36	collaboration with agents: how traces are generated, that a callee trace begins	47	corresponding proofs have no direct counterpart in Goblint and were developed
37	only at a call, that a trace retains its call history, and that calling contexts	48	for this work.
38	are derived from traces.		
39		49	
40	For the Isabelle development of the analysis framework, agents inspected	50	Throughout the project, the author also investigated how to detect and correct errors
41	Goblint's OCaml sources to identify interfaces the formalization could approximate, such as local and global	51	introduced by agents in a formal development of this size. Agent output
42	unknowns, unknowns indexed by node and context, and the enter/combine protocol	52	was checked before it was accepted, by Isabelle, by tests, or by manual review,
43	at calls (@sec:rel-goblint). The source language and its compilation, the	53	depending on the kind of change. Agents
44	activation traces, the coverage contract, and the corresponding proofs have no	54	proposed incorrect lemmas. In one case, an agent introduced a premise into the
45	direct counterpart in Goblint and were developed for this work.	55	end-to-end soundness theorem that no program satisfied. The theorem was
		56	therefore vacuous until the solver run was redesigned. The
		57	author's audits of theorem statements and full batch builds exposed these
		58	errors. The non-vacuity instances of @sec:nonvacuity now show that the premises
		59	of the end-to-end soundness theorem and its companions can be satisfied.
46		60	
47	A major part of the development effort concerned finding suitable definitions,	61	To detect errors and check consistency across the project, the author directed
48	interfaces, invariants, and locale boundaries rather than discharging individual	62	the development of extensive tooling, much of it implemented by agents (@sec:voblint-repo). A regression corpus, including
49	proof steps. During the first weeks, a small prototype mirrored Goblint's	63	ports of Goblint regression tests, records the expected analyzer output, and
50	structure while leaving proof obligations as `sorry`. Once the definitions and	64	`isar-tools` finds duplicated and unused lemmas in the proofs. Automated checks
51	locale boundaries had stabilized, many of the remaining proof obligations were	65	compare the theories, the generated material, the analyzer, the project site,
52	short and were discharged with substantial agent assistance.	66	and the text of this thesis with each other. These checks exposed errors that
		67	manual reading had missed.
53		68	
54	Throughout the project, the author also explored how such tools can be used	69	#let _cc = _ai.claude_code
55	responsibly and correctly in a formal development of this size. Agent output was	70	#let _cx = _ai.codex
56	treated as a proposal until Isabelle, a test or the author's review confirmed it. This		
57	caution was necessary, since the assistants' output was not reliable on its own. Agents proposed incorrect		
58	lemmas and, in one case, introduced a premise into the end-to-end soundness theorem that		
59	no program satisfied, so the theorem was vacuous until the solver run was		
60	redesigned. Audits of theorem statements and full batch builds exposed these		
61	errors. The non-vacuity instances of @sec:nonvacuity now demonstrate that the		
62	premises of the end-to-end soundness theorem and its companions can be satisfied.		
63			
64	Agents also wrote much of the tooling around the formalization, such as the		
65	command-line tool, the project site, and the regression corpus with its ports of		
66	Goblint regression tests. They also wrote the checks that keep the theories, the		
67	generated material, the analyzer, the project site, and this thesis consistent		
68	(@sec:voblint-repo). These checks proved especially useful because they replaced		
69	repeated manual inspection with conditions that every change had to satisfy, and		

Old	77e07947	New	0edeac81
70	they exposed errors that manual reading had missed.		
71		71	
72	@fig:ai-sessions quantifies the recorded part of this assistance using	72	@fig:ai-sessions quantifies the recorded part of this assistance using
73	session-log counts, following Bryant et al. @bryant26munkres[Section 5]. The counts	73	session-log counts, following Bryant et al. @bryant26munkres[Section 5]. The counts
74	measure interaction with the tools, not authorship or the proportion of the work	74	measure interaction with the tools, not authorship or the proportion of the work
75	attributable to them.	75	attributable to them. Earlier Claude Code sessions, ChatGPT in the browser, and Cursor left no
		76	local logs and are not counted. The author sent #_n(_cc.top_level.human_messages)
		77	messages to Claude Code and #_n(_cx.top_level.human_messages) to Codex.
		78	Co-author lines in commit messages were not kept consistently and are not used
		79	as a measure.
76		80	
77	#let _cc = _ai.claude_code		
78	#let _cx = _ai.codex		
79	#let _row(label, key, codex: true) = (	81	#let _row(label, key, codex: true) = (
80	label,	82	label,
81	..(_cc.top_level, _cc.subagents).map(s => _n(s.at(key))),	83	..(_cc.top_level, _cc.subagents).map(s => _n(s.at(key))),
	19 unchanged lines hidden		
101	table.hline(),	103	table.hline(),
102	),	104	),
103	kind: table,	105	kind: table,
104	caption: [Recorded automated-assistant activity during development, counted on #_ai.snapshot from the	106	caption: [Recorded assistant activity, counted on #_ai.snapshot from local session
105	local session logs (#_cc.top_level.first_day to #_cc.top_level.last_day for	107	logs (#_cc.top_level.first_day to #_cc.top_level.last_day for Claude Code,
106	Claude Code, #_cx.top_level.first_day to #_cx.top_level.last_day for Codex).	108	#_cx.top_level.first_day to #_cx.top_level.last_day for Codex). Rows count tool
107	No local logs are available for earlier Claude Code sessions, ChatGPT in	109	calls, not distinct changes or successful builds. Shell edits count as shell calls,
108	the browser, or Cursor, so these are not counted. The author sent	110	theory edits through the Isabelle server as server calls.],
109	#_n(_cc.top_level.human_messages) messages to Claude Code and		
110	#_n(_cx.top_level.human_messages) to Codex. Shell edits count as shell calls.		
111	Interactions through the Isabelle server, including theory edits, count as		
112	server calls. Co-author lines in commit messages were not kept consistently		
113	and are not used as a measure.],		
114	) <fig:ai-sessions>	111	) <fig:ai-sessions>
115		112	
116	Isabelle/HOL accepts every theorem in the development. Proofs by evaluation,	113	Isabelle/HOL accepts every theorem in the development. Proofs by evaluation,
117	used for some facts about fixed programs, additionally rely on Isabelle's code	114	used for some facts about fixed programs, additionally rely on Isabelle's code
118	generator (@fig:intro-trust). This establishes the stated propositions	115	generator. This establishes the stated propositions
119	relative to their definitions and assumptions, but does not establish that those	116	relative to their definitions and assumptions, but does not establish that those
120	definitions faithfully capture the intended language or analysis. Definitions	117	definitions faithfully capture the intended language or analysis. Definitions
121	and theorem statements were therefore also checked against example programs, the	118	and theorem statements were therefore also checked against example programs, the
122	regression corpus, and the documented differences between VIMP and C11, much as a human	119	regression corpus, and the documented differences between VIMP and C11, much as a human
123	audit checks the statements of AI-generated formal proofs @hales26lean[Sec. "Is Lean reliable?"].	120	audit checks the statements of AI-generated formal proofs @hales26lean[Sec. "Is Lean reliable?"].

124	Related-work claims were checked against the cited sources, often with agent	121	The author reviewed the related-work claims against the cited sources, often after a first check by an agent. Responsibility for the final claims, framing, and treatment of related work lies
125	assistance and subsequent author review.		
126			
127	Responsibility for the final claims, framing, and treatment of related work lies		
128	with the author alone.	122	with the author alone.

thesis/content/appendix-tooling.typ

77e07947 → 0edeac81

266 old lines, 265 new lines

90% similar lines **+25** **-26** **9 changed blocks**

Old	77e07947	New	0edeac81
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/code.typ": isa, isacmd, isaconst, isalocale, isathm, repo-blob, site	2	#import "../lib/code.typ": isa, isacmd, isaconst, isalocale, isathm, repo-blob, site
3	#import "../lib/theme.typ": vb	3	#import "../lib/theme.typ": vb
4	#import "../lib/stats.typ": stat	4	#import "../lib/stats.typ": stat
5		5	
6	= Repository and Tooling <sec:voblint-repo>	6	= Repository and Tooling <sec:voblint-repo>
7		7	
8	The theories, the exported analyzer, the project site, and this thesis quote	8	The theories, the exported analyzer, the project site, and this thesis quote
9	one another. The analyzer is exported from the theories, the thesis cites	9	one another. The analyzer is exported from the theories, the thesis cites
10	their names and statements, and both the site and the thesis quote the	10	their names and statements, and both the site and the thesis quote the
11	analyzer's output. Every change to the theories can therefore make another	11	analyzer's output. Every change to the theories can therefore make another
12	part stale, and keeping them consistent is engineering work outside the proof.	12	part stale, and keeping them consistent is engineering work outside the proof.
13	Isabelle offers comparatively little project-level tooling for this beyond its	13	For this project, the distribution's tools were supplemented with checks
14	distribution. Users have pointed this out as well @paulson26lean	14	connecting the theories, analyzer output, and document. A reader commenting on
		15	Paulson's discussion of proof assistants @paulson26lean also criticized
		16	Isabelle's build and contribution tooling
15	(#link("https://github.com/lawrencecpaulson/lawrencecpaulson.github.io/issues/62#issuecomment-4333878708")[reader comment]). The need grew with AI assistance. Agents wrote many proofs	17	(#link("https://github.com/lawrencecpaulson/lawrencecpaulson.github.io/issues/62#issuecomment-4333878708")[reader comment]). The need grew with AI assistance. Agents wrote many proofs
16	and much of the surrounding code (#link(<ai-use>)[statement on the use of	18	and much of the surrounding code (#link(<ai-use>)[statement on the use of
17	generative AI]), and a reviewer cannot read every line they produce. A change	19	generative AI]), and a reviewer cannot read every line they produce. A change
18	therefore counts as done only when deterministic checks pass, so the checks,	20	therefore counts as done only when deterministic checks pass, so the checks,
19	not the agent, decide whether the repository is consistent.	21	not the agent, decide whether the repository is consistent.
20		22	
21	== Repository contents and the vendored solver	23	== Repository contents and the vendored solver
22		24	
23	One repository holds everything this thesis describes: the Isabelle theories,	25	One repository holds everything this thesis describes: the Isabelle theories,
24	a submodule reference to the vendored TD solver, the exported analyzer with its	26	a submodule reference to the vendored TD solver, the exported analyzer with its
25	command-line tool and playground, the regression corpus, and the sources of	27	command-line tool and playground, the regression corpus, and the sources of
26	this thesis. Each release is archived on Zenodo with a citable DOI @voblint,	28	this thesis. Released snapshots are archived on Zenodo with a citable DOI @voblint,
27	and the project site hosts the playground, the explainer, and the rendered	29	and the project site hosts the playground, the explainer, and the rendered
28	theories that every Isabelle name in this thesis links to.	30	theories that every Isabelle name in this thesis links to.

Old	77e07947	New	0edeac81
29		31	
30	The vendored TD solver of Tilscher et al. @tilscher26 carries four changes made	32	The vendored TD solver of Tilscher et al. @tilscher26 carries four changes made
31	for this thesis, each proposed upstream and none changing the algorithm. They	33	for this thesis, each proposed upstream and none changing the algorithm.
32	restore the build under Isabelle2025-2	34	#link("https://github.com/stilscher/td-verification/pull/15")[PR 15] restores
33	(#link("https://github.com/stilscher/td-verification/pull/15")[PR 15]), remove the stabilization	35	the build under Isabelle2025-2.
34	assumptions from the classes #isalocale("widening") and	36	#link("https://github.com/stilscher/td-verification/pull/13")[PR 13] removes the
		37	stabilization assumptions from the classes #isalocale("widening") and
35	#isalocale("narrowing"), which only termination proofs use	38	#isalocale("narrowing"), which only termination proofs use
36	(#link("https://github.com/stilscher/td-verification/pull/13")[PR 13], @sec:domain-carrier-laws), rename short field and constructor names that	39	(@sec:domain-carrier-laws).
37	clashed with downstream theories	40	#link("https://github.com/stilscher/td-verification/pull/14")[PR 14] renames
38	(#link("https://github.com/stilscher/td-verification/pull/14")[PR 14]), and add facts about the solver's	41	short field and constructor names that clashed with downstream theories, and
39	definitions, such as #isathm("solve_dom_of_solve_c"), that	42	#link("https://github.com/stilscher/td-verification/pull/16")[PR 16] adds facts
40	#isathm("td_certified_solver") uses	43	about the solver's definitions, such as #isathm("solve_dom_of_solve_c"), that
41	(#link("https://github.com/stilscher/td-verification/pull/16")[PR 16], @sec:certificate). At the time of	44	#isathm("td_certified_solver") uses (@sec:certificate). At the time of
42	writing, all four pull requests await review. The solver's repository is	45	writing, all four pull requests await review. The upstream solver has a public artifact @tilscher26artifact. Voblint's
43	private, so the development cannot yet be built from public sources alone,	46	modified fork remains private, so reproducing this development still
44	and there are plans to publish the solver in the Archive of Formal Proofs.	47	requires access to that exact dependency.
45		48	
46	== OCaml packages on conda-forge	49	== OCaml packages on conda-forge
47		50	
	34 unchanged lines hidden		
82	(@sec:instances-supply). The product state of	85	(@sec:instances-supply). The product state of
83	@ch:cooperation is written by hand, with one field per analysis, each of a	86	@ch:cooperation is written by hand, with one field per analysis, each of a
84	different type. A dependently typed logic could express such a heterogeneous	87	different type. A dependently typed logic could express such a heterogeneous
85	structure once @chlipala13cpdt[ch. 9], but HOL's simple types cannot, so adding	88	structure directly @chlipala13cpdt[ch. 9]. With the chosen HOL encoding,
86	an analysis adds a field and one case to each function over the analyses.	89	adding an analysis adds a field and a case to each function over the analyses.
87		90	
88	The most important derived file is the analyzer itself. Isabelle's code	91	The most important derived file is the analyzer itself. Isabelle's code
89	generator exports #isaconst("run_voblint") as OCaml, and the repository commits	92	generator exports #isaconst("run_voblint") as OCaml, and the repository commits
	102 unchanged lines hidden		
192	== Maintaining the formalization: `isar-tools` <sec:isar-tools>	195	== Maintaining the formalization: `isar-tools` <sec:isar-tools>
193		196	
194	Isabelle checks that every proof goes through, but a successful build says	197	Isabelle checks that every proof goes through, but a successful build says
195	little about how well a development is maintained. A lemma that no proof	198	little about how well a development is maintained. The build stays green with a lemma no proof cites, an import that another import already brings in, a constant written out next to the notation the project gave it, or a locale assumption no proof uses. Documentation that quotes formal statements or
196	cites, an import that another import already brings in, a constant written out		

197	next to the notation the project gave it, or a locale assumption that no proof		
198	uses all leave the build green. Documentation that quotes formal statements or		
199	links to the rendered theories can also fall out of date without affecting the	199	links to the rendered theories can also fall out of date without affecting the
200	build. In a development of #stat("isabelle.theories") theories, such problems	200	build. In a development of #stat("isabelle.theories") theories, such problems
201	accumulate.	201	accumulate.
	9 unchanged lines hidden		
211	#link("https://github.com/ott2/isabelle-query")[`isabelle-query`] answers	211	#link("https://github.com/ott2/isabelle-query")[`isabelle-query`] answers
212	source-level queries about declarations, dependencies, and unfinished proofs.	212	source-level queries about declarations, dependencies, and unfinished proofs.
213		213	
214	The tool has three parts. `isar fmt` changes only layout. `isar check` offers	214	The tool consists of `isar fmt`, `isar check`, and `isar project`. `isar fmt` changes only layout. `isar check` offers
215	about fifty checks in groups. Checks for unfinished proofs and unreached	215	about fifty checks in groups. Checks for unfinished proofs and unreached
216	theories run by default, and checks for unused or duplicated lemmas, unused	216	theories run by default, and checks for unused or duplicated lemmas, unused
217	locale assumptions, or terms written out next to their notation run when	217	locale assumptions, or terms written out next to their notation run when
	36 unchanged lines hidden		
254	inspect and does not prove an error.	254	inspect and does not prove an error.
255		255	
256	A second kind of finding concerns build time. A session inherits the heaps of	256	A second kind of finding concerns build time. A session inherits the heaps of
257	its parent sessions, but a library theory outside that ancestry is elaborated	257	its parent sessions, but a library theory outside that ancestry is processed again in every session that imports it. `isar stats build` reads the time each
258	again in every session that imports it. `isar stats build` reads the time each		
259	theory took from a verbose build log and reports such repetitions, and	258	theory took from a verbose build log and reports such repetitions, and
260	continuous integration enforces a budget per library session. In this	259	continuous integration enforces a budget per library session. In this
261	repository, repeated elaboration once cost 19% of a clean build.	260	repository, repeated processing once took 19% of a clean build.
262		261	
263	All of these checks work on the sources and complement Isabelle. They point to	262	All of these checks work on the sources and complement Isabelle. They point to
264	suspicious patterns and structural problems, and they may report false	263	suspicious patterns and structural problems, and they may report false
265	positives or miss problems. Whether a theory is formally correct is still	264	positives or miss problems. Whether a theory is formally correct is still
266	decided by the Isabelle build alone.	265	decided by the Isabelle build alone.

thesis/content/reading-guide.typ

77e07947 → 0edeac81

91 old lines, 94 new lines

78% similar lines +22 -19 **6 changed blocks**

Old	77e07947	New	0edeac81
1	#import "../lib/math.typ": *	1	#import "../lib/math.typ": *
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": (	3	#import "../lib/code.typ": (
4	fixture, isaconst, isalocale, isathm, isatype, oblig, site, site-url, thy-badge,	4	fixture, isaconst, isalocale, isathm, isatype, oblig, site, site-url, thy-badge,
5	)	5	)
6	#import "../lib/figures.typ": verdict	6	#import "../lib/figures.typ": verdict
7	#import "../lib/claims.typ": claim-ref	7	#import "../lib/claims.typ": claim-ref
8		8	
9	= Reading This Thesis <reading-guide>	9	= Reading This Thesis <reading-guide>

Old	77e07947	New	0edeac81
10		10	
11	The thesis can be read on paper, but it is designed to be read on screen. It closely	11	The thesis can be read on paper, but it is designed to be read on screen. It closely
12	follows a machine-checked Isabelle/HOL development, and most of its definitions and	12	follows the machine-checked Isabelle/HOL development. Definitions
13	theorems link to the #link(site("Voblint/index.html"))[HTML rendering of the theories].	13	and theorems link to the #link(site("Voblint/index.html"))[rendered theories],
14	A single #link(site("formalization.pdf"))[PDF export of the whole formalization] is available as	14	also available as a #link(site("formalization.pdf"))[single PDF].
15	well. The project site at #link(site-url)[#site-url.trim("https://", at: start)] also	15	The project site hosts an animated #link(site("index.html"))[explainer],
16	hosts an #link(site("index.html"))[explainer] that animates several of the constructions,	16	a #link(site("proof-map.html"))[proof map] of the declarations,
17	a #link(site("profiling.html"))[profiling page] with the full benchmark data and CPU	17	#link(site("profiling.html"))[benchmark data and CPU profiles], and a
18	profiles, and the #link(site("playground.html"))[playground], which runs the analyzer in	18	#link(site("playground.html"))[playground] for running VIMP programs.
19	the browser on any VIMP program.		
20		19	
21	Each kind of object the text refers to has its own marker, and most markers are also links.	20	Each kind of object has its own marker, usually also a link. Isabelle names open
22	An Isabelle name opens its definition, a program opens the playground, and a fixture opens	21	their declarations, programs open the playground, and test programs open their
23	its file in the repository.	22	repository files.
24		23	
25	@sec:isabelle explains how to read the Isabelle listings, and @app:notation lists the	24	@sec:isabelle explains how to read the Isabelle listings, and @app:notation lists the
26	mathematical notation shared by the theories and the text.	25	mathematical notation shared by the theories and the text.
	20 unchanged lines hidden		
47	[Teal small capitals mark an obligation of the coverage contract	46	[Teal small capitals mark an obligation of the coverage contract
48	(@sec:contract), linked to the locale assumption that states it.],	47	(@sec:contract), linked to the locale assumption that states it.],
49	[#ctor("FunctionEntry"), #keyw("while")],	48	[#ctor("FunctionEntry"), #keyw("while")],
50	[Purple sans-serif text marks a datatype constructor in notation, and	49	[Purple sans-serif text marks a constructor of an Isabelle datatype, and
51	bold maroon text a keyword of the source language VIMP.],	50	bold maroon text a keyword of the source language VIMP.],
52	[Section title #thy-badge("Voblint_CLI", "Analysis_Run")],	51	[Section title #thy-badge("Voblint_CLI", "Analysis_Run")],
53	[A small blue #raw("thy") box beside a heading marks a section that	52	[A small blue #raw("thy") box beside a heading marks a section that
	20 unchanged lines hidden		
74	[A coloured monospace word on a tinted background is a verdict as the	73	[A coloured monospace word on a tinted background is a verdict as the
75	analyzer reports it (@sec:verdict-meaning).],	74	analyzer reports it (@sec:verdict-meaning).],
76	[#claim-ref("intro-answers")],	75	[#claim-ref("intro-answers")],
77	[An underlined blue name marks a _claim_, a registered analyzer run. Every	76	[An underlined blue name marks a _claim_, an analyzer run recorded with
78	verdict and state of Voblint quoted in the text is read from a claim,	77	the thesis sources. Every verdict and state of Voblint quoted in the text
79	which the build repeats, and the name opens the run in the playground.],	78	comes from a claim. A check reruns each claim and fails if the analyzer
		79	output differs from the quoted one. The name opens the run in the
		80	playground.],
80	[#fixture(	81	[#fixture(
81	"21-context-sensitivity/01-unbounded_context_chain_diverges.vimp",	82	"21-context-sensitivity/01-unbounded_context_chain_diverges.vimp",
82	label: "01-unbounded_…",	83	label: "01-unbounded_…",
83	)],	84	)],
84	[A blue monospace file name marks a fixture of the regression corpus,	85	[A blue monospace file name marks a _fixture_, one test program of the
85	linked to its file in the repository.],	86	regression corpus, which records each program with its expected analyzer
		87	output. The name links to the file in the repository.],

86	[#signval("+"), $absval(ivl(1, 9))$, #absval(raw("1+2ℤ"))],	88	[#signval("+"), $absval(ivl(1, 9))$, #absval(raw("1+2ℤ"))],
87	[A light purple background marks an abstract value, as opposed to a	89	[A light purple background marks an abstract value, which describes a
88	concrete integer or set of stores.],	90	set of concrete integers or of _stores_, the maps from variables to
		91	their integer values.],
89	table.hline(),	92	table.hline(),
90	)	93	)
91	}	94	}

thesis/thesis.typ

77e07947 → 0edeac81

123 old lines, 123 new lines

98% similar lines +3 -3 **3 changed blocks**

Old	77e07947	New	0edeac81
1	#import "lib/tum.typ": front-chapter, thesis	1	#import "lib/tum.typ": front-chapter, thesis
2	#import "lib/theme.typ": vb	2	#import "lib/theme.typ": vb
3	#import "lib/code.typ": isabelle-scripts	3	#import "lib/code.typ": isabelle-scripts
	80 unchanged lines hidden		
84	#include "content/01-introduction.typ"	84	#include "content/01-introduction.typ"
85	#include "content/02-background.typ"	85	#include "content/02-background.typ"
86		86	
87	#part("What Must Be Over-Approximated", lbl: <part:over-approx>)	87	#part("Semantics and the Coverage Contract", lbl: <part:over-approx>)
88	#include "content/03-program-model.typ"	88	#include "content/03-program-model.typ"
89	#include "content/04-traces.typ"	89	#include "content/04-traces.typ"
90		90	
91	#part("Building Analyses", lbl: <part:analyses>)	91	#part("Domains and Analyses", lbl: <part:analyses>)
92	#include "content/05-domains.typ"	92	#include "content/05-domains.typ"
93	#include "content/06-analysis-interface.typ"	93	#include "content/06-analysis-interface.typ"
94	#include "content/07-cooperation.typ"	94	#include "content/07-cooperation.typ"
95		95	
96	#part("Computing Sound Results", lbl: <part:verdicts>)	96	#part("Equations, Solver, and Verdicts", lbl: <part:verdicts>)
97	#include "content/08-equations.typ"	97	#include "content/08-equations.typ"
98	#include "content/09-solving.typ"	98	#include "content/09-solving.typ"
99	#include "content/10-results.typ"	99	#include "content/10-results.typ"
	21 unchanged lines hidden		
121	#include "content/appendix-notation.typ"	121	#include "content/appendix-notation.typ"
122	#include "content/appendix-tooling.typ"	122	#include "content/appendix-tooling.typ"
123		123	